

RTL Design Sherpa

RTL AMBA AXI5 Module Reference — Rev 0.92

October 2026

Table of Contents

1 AXI5 (Advanced eXtensible Interface - AMBA 5) Modules....25
1.1 Overview....26
1.1.1 Scope of This Implementation....26
1.2 AXI5 Features Implemented....27
1.3 AXI5 Features Not Implemented....29
1.4 AXI4 vs AXI5....31
1.5 Module Categories....33
1.5.1 Master Components....33
1.5.2 Slave Components....33
1.5.3 Monitor Components....34
1.6 Key Features....35
1.6.1 AXI5 Channel Support....35
1.6.2 Advanced Features....35
1.6.3 Power Management....35
1.6.4 Monitoring and Debug....35
1.7 Protocol Details....36
1.7.1 AXI5 Channel Overview....36
1.7.2 AXI5 Sideband Signals Carried by These Modules....36
1.7.3 Burst Types....37
1.8 Usage Example....38
1.8.1 Using AXI5 Read Master....38
1.8.2 Using AXI5 Write Master....39

1.8.3 Using Integrated Monitor Modules....40
1.9 Design Notes....41
1.9.1 FUB Interface Pattern....41
1.9.2 Migration from AXI4....41
1.9.3 Monitor Integration....41
1.9.4 Performance Characteristics....41
1.10 Related Modules....42
1.11 Testing....43
1.12 References....44
1.12.1 Specifications....44
1.12.2 Papers and textbooks....44
1.12.3 Source Code....44
1.13 Navigation....45
2 AXI5 Atomic Filter....46
2.1 Overview....47
2.1.1 Key Features....48
2.2 Parameters....49
2.2.1 Derived Parameters (do not override)....49
2.3 Ports....50
2.4 Functional Description....52
2.4.1 Block diagram....52
2.5 Timing Characteristics....54
2.6 Usage Examples....55
2.6.1 Integration....56

2.6.2 Design Notes and Constraints....56
2.7 Design Notes....57
2.8 Testing....58
2.9 Related Modules....59
2.10 References....60
2.11 Navigation....61
3 AXI5 Atomic Read-Return Tracker....62
3.1 Overview....63
3.1.1 Key Features....63
3.2 Parameters....64
3.2.1 Derived Parameters (do not override)....64
3.3 Ports....65
3.4 Functional Description....66
3.4.1 Block diagram....66
3.5 Timing Characteristics....67
3.6 Usage Examples....68
3.7 Verification....69
3.8 Related....70
3.9 References....71
4 AXI5 Master Read....72
4.1 Overview....73
4.1.1 Key Features....73
4.2 Parameters....74
4.2.1 Derived Parameters....75

4.3 Ports....76
4.3.1 Clock and Reset....76
4.3.2 FUB AXI5 Interface (Slave Side - Input)....76
4.3.3 Master AXI5 Interface (Output Side)....78
4.3.4 Status Outputs....78
4.4 Functional Description....79
4.4.1 Architecture....79
4.4.2 AXI5 Enhancements Over AXI4....79
4.4.3 Signal Packing Logic....80
4.5 Timing Characteristics....81
4.5.1 Basic Read Transaction....81
4.5.2 Chunked Read Transaction....81
4.5.3 Memory Tagging Extension (MTE)....81
4.6 Usage Examples....82
4.7 Design Notes....84
4.7.1 AXI5 vs AXI4 Differences....84
4.7.2 Memory Tagging Extension (MTE)....84
4.7.3 Chunking Support....84
4.7.4 Feature Enable Strategy....84
4.8 Related Modules....86
4.9 Testing....87
4.10 References....88
4.11 Navigation....90
5 AXI5 Master Read with Clock Gating....91

5.1 Overview....92
5.1.1 Key Features....92
5.2 Parameters....93
5.2.1 Derived Parameters....94
5.3 Ports....95
5.3.1 Clock and Reset....95
5.3.2 Clock Gating Configuration....95
5.3.3 FUB AXI5 Interface (Slave Side - Input)....95
5.3.4 Master AXI5 Interface (Output Side)....97
5.3.5 Clock Gating Status Outputs....98
5.4 Functional Description....99
5.4.1 Architecture....99
5.4.2 Clock Gating Operation....99
5.4.3 Ready Deassertion and Wake Behavior....100
5.4.4 Integration Caveat: fub_axi_rready in the Activity Term....100
5.4.5 Idle Counter Configuration....101
5.4.6 Power Savings Analysis....101
5.5 Timing Characteristics....102
5.5.1 Clock Gating Activation....102
5.5.2 Clock Gating Wake-up....102
5.6 Usage Examples....103
5.7 Design Notes....105
5.7.1 Clock Gating vs. Always-On....105
5.7.2 Configuration Strategies....105

5.7.3 Integration with System Power Management....105
5.7.4 Verification Considerations....106
5.8 Related Modules....107
5.9 Testing....108
5.10 References....109
5.11 Navigation....111
6 AXI5 Master Read with Monitor....112
6.1 Overview....113
6.1.1 Key Features....113
6.2 Parameters....115
6.2.1 Derived Parameters (do not override)....119
6.3 Ports....120
6.3.1 Clock and Reset....120
6.3.2 FUB AXI5 Interface (Slave Side - Input)....120
6.3.3 Master AXI5 Interface (Output Side)....120
6.3.4 Monitor Configuration....120
6.3.5 AXI Protocol Filtering Configuration....122
6.3.6 Monitor Bus Output....122
6.3.7 Status Outputs....123
6.3.8 Packet filters....123
6.4 Functional Description....125
6.4.1 Architecture....125
6.4.2 Monitor Backpressure (block_ready)....125
6.4.3 Address-Range Checker....126

6.4.4 Monitor Bus Packet Format....127
6.4.5 Filtering Hierarchy (two active levels, one reserved)....127
6.4.6 Error Detection Events....128
6.4.7 Configuration Conflict Detection....129
6.4.8 Performance Monitoring....129
6.5 Timing Characteristics....132
6.5.1 Monitored Read Transaction with Error....132
6.5.2 Timeout Detection....132
6.5.3 Performance Monitoring....132
6.6 Usage Examples....133
6.7 Design Notes....136
6.7.1 Configuration Best Practices....136
6.7.2 Transaction Table Sizing....136
6.7.3 Filtering Strategy....136
6.7.4 Monitor Bus Backpressure....137
6.8 Related Modules....138
6.9 Testing....139
6.10 References....140
6.11 Navigation....141
7 AXI5 Master Read with Monitor and Clock Gating....142
7.1 Overview....143
7.1.1 Key Features....143
7.2 Parameters....144
7.2.1 Filter configuration (forwarded)....148

7.2.2 Derived Parameters (do not override)....148
7.3 Ports....150
7.3.1 Clock and Reset....150
7.3.2 Clock Gating Configuration....150
7.3.3 FUB AXI5 Interface (Slave Side - Input)....150
7.3.4 Master AXI5 Interface (Output Side)....150
7.3.5 Monitor Configuration....150
7.3.6 Monitor Bus Output....150
7.3.7 Status Outputs....151
7.4 Functional Description....152
7.4.1 Architecture....152
7.4.2 Combined Monitoring and Power Management....152
7.4.3 Clock Gating with Monitor Active....153
7.4.4 Ready Signal Control During Gating....153
7.4.5 Performance Monitoring....154
7.5 Timing Characteristics....155
7.5.1 Clock Gating with Monitor Activity....155
7.5.2 Wake-up with Immediate Monitoring....155
7.6 Usage Examples....156
7.7 Design Notes....160
7.7.1 When to Use This Module....160
7.7.2 Configuration Strategy Matrix....160
7.7.3 Monitor + Clock Gating Interaction....160
7.7.4 Verification Recommendations....161

7.8 Related Modules..... 162
7.9 Testing..... 163
7.10 References..... 164
7.11 Navigation..... 165
8 AXI5 Master Write..... 166
8.1 Overview..... 167
8.1.1 Key Features..... 167
8.2 Parameters..... 168
8.2.1 Derived Parameters..... 169
8.3 Ports..... 170
8.3.1 Clock and Reset..... 170
8.3.2 FUB AXI5 Interface (Slave Side - Input)..... 170
8.3.3 Master AXI5 Interface (Output Side)..... 173
8.3.4 Status Outputs..... 173
8.4 Functional Description..... 174
8.4.1 Architecture..... 174
8.4.2 AXI5 Enhancements Over AXI4..... 174
8.4.3 Atomic Transactions..... 175
8.5 Timing Characteristics..... 177
8.5.1 Basic Write Transaction..... 177
8.5.2 Atomic Operation..... 177
8.5.3 Memory Tagging Extension (MTE)..... 177
8.6 Usage Examples..... 178
8.7 Design Notes..... 180

8.7.1 Atomic Operation Support....180
8.7.2 Memory Tagging Extension (MTE)....180
8.7.3 Poison Support....180
8.7.4 Feature Enable Strategy....180
8.8 Related Modules....181
8.9 Testing....182
8.10 References....183
8.11 Navigation....185
9 AXI5 Master Write with Clock Gating....186
9.1 Overview....187
9.1.1 Key Features....187
9.2 Parameters....188
9.2.1 Derived Parameters....189
9.3 Ports....190
9.3.1 Clock and Reset....190
9.3.2 Clock Gating Configuration....190
9.3.3 FUB AXI5 Interface (Slave Side - Input)....190
9.3.4 Master AXI5 Interface (Output Side)....193
9.3.5 Clock Gating Status Outputs....193
9.4 Functional Description....194
9.4.1 Architecture....194
9.4.2 Clock Gating Operation....194
9.4.3 Ready Deassertion and Wake Behavior....195
9.4.4 Integration Caveat: fub_axi_bready in the Activity Term....195

9.4.5 AXI5 Write-Specific Features....195
9.4.6 Idle Counter Configuration....196
9.5 Timing Characteristics....197
9.5.1 Clock Gating with Write Burst....197
9.5.2 Atomic Operation with Clock Gating....197
9.6 Usage Examples....198
9.7 Design Notes....201
9.7.1 Write vs. Read Clock Gating....201
9.7.2 Write-Specific Gating Considerations....201
9.8 Related Modules....202
9.9 Testing....203
9.10 References....204
9.11 Navigation....206
10 AXI5 Master Write with Monitor....207
10.1 Overview....208
10.1.1 Key Features....208
10.1.2 Module Architecture....209
10.2 Parameters....210
10.2.1 Derived Parameters (do not override)....214
10.3 Ports....215
10.3.1 Clock and Reset....215
10.3.2 FUB AXI5 Interface (Slave Side)....215
10.3.3 Master AXI5 Interface (Output Side)....215
10.3.4 Monitor Configuration....215

10.3.5 Monitor Bus Output....215
10.3.6 Status Outputs....216
10.3.7 Packet filters....217
10.4 Functional Description....218
10.4.1 Monitor Bus Packet Format....218
10.4.2 Write-Specific Monitoring Events....218
10.4.3 Atomic Operation Monitoring....219
10.4.4 Filtering Hierarchy (two active levels, one reserved)....219
10.4.5 Monitor Backpressure (block_ready)....219
10.4.6 Address-Range Checker....220
10.4.7 Performance Monitoring....220
10.5 Timing Characteristics....224
10.5.1 Monitored Write Burst with Error....224
10.5.2 Atomic Operation Monitoring....224
10.5.3 Write Timeout Detection....224
10.6 Usage Examples....225
10.7 Design Notes....229
10.7.1 W-Channel Wiring into the Monitor Core....229
10.7.2 Write vs. Read Monitoring Differences....229
10.7.3 Write Transaction Table Management....229
10.7.4 Write Latency Measurement....229
10.7.5 Configuration Best Practices....229
10.8 Related Modules....231
10.9 Testing....232

10.10 References....233
10.11 Navigation....234
11 AXI5 Master Write with Monitor and Clock Gating....235
11.1 Overview....236
11.1.1 Key Features....236
11.1.2 Module Architecture....237
11.2 Parameters....238
11.2.1 Derived Parameters (do not override)....242
11.3 Ports....243
11.3.1 Filter configuration (forwarded)....243
11.3.2 Clock and Reset....243
11.3.3 Clock Gating Configuration....243
11.3.4 FUB AXI5 Interface (Slave Side)....244
11.3.5 Master AXI5 Interface (Output Side)....244
11.3.6 Monitor Configuration....244
11.3.7 Monitor Bus Output....244
11.3.8 Status Outputs....244
11.4 Functional Description....246
11.4.1 Combined Write Monitoring and Power Management....246
11.4.2 Clock Gating with Write Monitor Active....246
11.4.3 Ready Signal Control During Gating....246
11.4.4 Performance Monitoring....247
11.5 Timing Characteristics....248
11.5.1 Clock Gating with Write Burst and Monitoring....248

11.5.2 Atomic Operation with Monitor + Clock Gating....248
11.6 Usage Examples....249
11.7 Design Notes....253
11.7.1 When to Use This Module....253
11.7.2 Write-Specific Configuration Strategy....253
11.7.3 Monitor + Clock Gating Interaction for Writes....253
11.7.4 Performance Analysis with Write Monitoring....254
11.8 Related Modules....255
11.9 Testing....256
11.9.1 Verification Recommendations for Write Path....256
11.10 References....257
11.11 Navigation....258
12 AXI5 Slave Read....259
12.1 Overview....260
12.1.1 Key Features....260
12.1.2 Module Architecture....261
12.2 Parameters....262
12.2.1 Derived Parameters....263
12.3 Ports....264
12.3.1 Clock and Reset....264
12.3.2 Slave AXI5 Interface (External Master Input)....264
12.3.3 FUB Interface (Backend Memory/Logic Output)....266
12.3.4 Status Outputs....266
12.4 Functional Description....267

12.5 Timing Characteristics....268
12.5.1 Basic Read Transaction....268
12.6 Usage Examples....269
12.7 Design Notes....270
12.8 Related Modules....271
12.9 Testing....272
12.10 References....273
12.11 Navigation....275
13 AXI5 Slave Read with Clock Gating....276
13.1 Overview....277
13.1.1 Key Features....277
13.1.2 Module Architecture....278
13.2 Parameters....279
13.2.1 Derived Parameters....280
13.2.2 Derived Parameters (do not override)....280
13.3 Ports....282
13.3.1 Clock and Reset....282
13.3.2 Clock Gating Configuration....282
13.3.3 Slave AXI5 Interface....282
13.3.4 FUB Interface....282
13.3.5 Clock Gating Status....282
13.4 Functional Description....283
13.4.1 Clock Gating Behavior....283
13.4.2 Ready Deassertion and Wake Behavior....283

13.4.3 Integration Caveat: s_axi_rready in the Activity Term....284
13.4.4 Clock Gating Configuration....284
13.5 Timing Characteristics....286
13.5.1 Clock Gating Sequence....286
13.6 Usage Examples....287
13.7 Design Notes....289
13.7.1 Power Savings Estimation....289
13.7.2 Clock Gating Overhead....289
13.7.3 When to Use Clock Gating....289
13.7.4 Integration with Power Management....289
13.8 Related Modules....291
13.9 Testing....292
13.10 References....293
13.11 Navigation....295
14 AXI5 Slave Read Monitor....296
14.1 Overview....297
14.1.1 Key Features....297
14.1.2 Module Architecture....298
14.2 Parameters....299
14.2.1 Derived Parameters (do not override)....303
14.3 Ports....304
14.3.1 Clock and Reset....304
14.3.2 Slave AXI5 Interface....304
14.3.3 FUB Interface....304

14.3.4 Monitor Configuration....304
14.3.5 Monitor Bus Output....306
14.3.6 Status Outputs....307
14.3.7 Packet filters....307
14.4 Functional Description....309
14.4.1 Monitor Packet Format....309
14.4.2 Event Types....309
14.4.3 Monitor Backpressure (block_ready)....310
14.4.4 Address-Range Checker....311
14.4.5 Performance Monitoring....312
14.5 Timing Characteristics....315
14.6 Usage Examples....316
14.7 Design Notes....319
14.8 Related Modules....320
14.9 Testing....321
14.10 References....322
14.11 Navigation....323
15 AXI5 Slave Read Monitor with Clock Gating....324
15.1 Overview....325
15.1.1 Key Features....325
15.1.2 Module Architecture....326
15.2 Parameters....327
15.2.1 Derived Parameters (do not override)....331
15.3 Ports....332

15.3.1 Filter configuration (forwarded)....332
15.3.2 Clock and Reset....332
15.3.3 Clock Gating Configuration....332
15.3.4 Slave AXI5 Interface....333
15.3.5 FUB Interface....333
15.3.6 Monitor Configuration....333
15.3.7 Monitor Bus Output....333
15.3.8 Status Outputs....333
15.4 Functional Description....335
15.4.1 Clock Gating Behavior....335
15.4.2 Performance Monitoring....335
15.5 Timing Characteristics....336
15.6 Usage Examples....337
15.7 Design Notes....341
15.7.1 Power Optimization....341
15.7.2 Observability....341
15.7.3 When to Use This Module....341
15.7.4 Configuration Recommendations....341
15.7.5 Area and Timing Impact....341
15.8 Related Modules....343
15.9 Testing....344
15.10 References....345
15.11 Navigation....346
16 AXI5 Slave Write....347

16.1 Overview....348
16.1.1 Key Features....348
16.1.2 Module Architecture....349
16.2 Parameters....350
16.2.1 Derived Parameters....351
16.3 Ports....352
16.3.1 Clock and Reset....352
16.3.2 Slave AXI5 Interface (External Master Input)....352
16.3.3 FUB Interface (Backend Memory/Logic Output)....354
16.3.4 Status Outputs....355
16.4 Functional Description....356
16.5 Timing Characteristics....357
16.5.1 Basic Write Transaction....357
16.5.2 Write with Poison Indicator....357
16.6 Usage Examples....358
16.7 Design Notes....360
16.8 Related Modules....361
16.9 Testing....362
16.10 References....363
16.11 Navigation....365
17 AXI5 Slave Write with Clock Gating....366
17.1 Overview....367
17.1.1 Key Features....367
17.1.2 Module Architecture....368

17.2 Parameters....369
17.2.1 Derived Parameters....370
17.2.2 Derived Parameters (do not override)....370
17.3 Ports....372
17.3.1 Clock and Reset....372
17.3.2 Clock Gating Configuration....372
17.3.3 Slave AXI5 Interface....372
17.3.4 FUB Interface....372
17.3.5 Clock Gating Status....372
17.4 Functional Description....373
17.4.1 Clock Gating Behavior....373
17.4.2 Ready Deassertion and Wake Behavior....373
17.4.3 Integration Caveat: s_axi_bready in the Activity Term....374
17.4.4 Clock Gating Configuration....374
17.5 Timing Characteristics....376
17.5.1 Clock Gating During Write Burst....376
17.6 Usage Examples....377
17.7 Design Notes....379
17.7.1 Power Savings Estimation....379
17.7.2 Write-Specific Considerations....379
17.7.3 Clock Gating Overhead....379
17.7.4 When to Use Clock Gating....379
17.8 Related Modules....380
17.9 Testing....381

17.10 References....382
17.11 Navigation....384
18 AXI5 Slave Write Monitor....385
18.1 Overview....386
18.1.1 Key Features....386
18.1.2 Block Diagram....387
18.2 Parameters....388
18.2.1 Derived Parameters (do not override)....392
18.3 Ports....393
18.3.1 Clock and Reset....393
18.3.2 Slave AXI5 Interface....393
18.3.3 FUB Interface....393
18.3.4 Monitor Configuration....393
18.3.5 Monitor Bus Output....395
18.3.6 Status Outputs....396
18.3.7 Packet filters....396
18.4 Functional Description....398
18.4.1 Monitor Backpressure (block_ready)....398
18.4.2 Address-Range Checker....399
18.4.3 Monitor Packet Format....399
18.4.4 Event Types....400
18.4.5 Performance Monitoring....401
18.5 Timing Characteristics....404
18.6 Usage Examples....405

18.7 Design Notes....409
18.7.1 W-Channel Wiring into the Monitor Core....409
18.7.2 Write-Specific Monitoring....409
18.7.3 Best Practices....409
18.8 Related Modules....410
18.9 Testing....411
18.10 References....412
18.11 Navigation....413
19 AXI5 Slave Write Monitor with Clock Gating....414
19.1 Overview....415
19.1.1 Key Features....415
19.1.2 Block Diagram....416
19.2 Parameters....417
19.2.1 Derived Parameters (do not override)....421
19.3 Ports....422
19.3.1 Filter configuration (forwarded)....422
19.3.2 Clock and Reset....422
19.3.3 Clock Gating Configuration....423
19.3.4 Slave AXI5 Interface....423
19.3.5 FUB Interface....423
19.3.6 Monitor Configuration....423
19.3.7 Monitor Bus Output....423
19.3.8 Status Outputs....424
19.4 Functional Description....425

19.4.1 Clock Gating Behavior....425
19.4.2 Performance Monitoring....425
19.5 Timing Characteristics....427
19.6 Usage Examples....428
19.7 Design Notes....432
19.7.1 When to Use This Module....432
19.7.2 Configuration Recommendations....432
19.7.3 Area and Timing Impact....432
19.7.4 Power Optimization....433
19.7.5 Observability....433
19.7.6 Write-Specific Considerations....433
19.8 Related Modules....434
19.9 Testing....435
19.10 References....436
19.11 Navigation....437

1 AXI5 (Advanced eXtensible Interface - AMBA 5) Modules

Location: `rtl/amba/axi5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXI5 subsystem provides read/write master and slave channel modules for the ARM AMBA 5 AXI (Advanced eXtensible Interface) protocol, plus clock-gated variants and combined module+monitor variants for efficient system integration. Read *Scope of This Implementation* below before you treat these as a full AXI5 protocol stack — they aren't one, and that distinction will save you a redesign.

AXI5 extends AXI4 with significant enhancements for modern high-performance SoC designs: atomic transactions, memory tagging, memory partitioning, and data poisoning.

1.1.1 Scope of This Implementation

These modules are **channel-transport blocks**: they carry a full AXI5 signal set across configurable SKID buffers between a FUB (Functional Unit Block) interface and an external AXI5 interface. They're the AXI5 equivalents of the `axi4_master_*` / `axi4_slave_*` modules — same job, wider signal set.

What the RTL does:

- Transports every implemented AXI5 sideband signal end-to-end without modification.
- Packs and unpacks channel payloads conditionally, so disabled features cost zero area.
- Provides a `busy` status output for clock gating and power management.

What the RTL does not do:

- It does not *execute* AXI5 semantics. `AWATOP` is transported, but no atomic read-modify-write is performed; `AxTAGOP`/`RTAG`/`WTAG` are transported, but no tag checking is performed; chunk fields are transported, but no chunk reassembly is performed. Those behaviors belong to the endpoint (memory controller, interconnect, or FUB logic) on either side.
- It does not track or limit outstanding transactions. Depth is bounded by the endpoints, not by these modules.

Consult the per-module pages for the exact signal set each module carries.

1.2 AXI5 Features Implemented

AXI5 feature	Signals	Status in `rtl/amba/axi5/`
Non-secure access identifier	ARNSAID / AWNSAID	Transported (`ENABLE_NSAID`)
Trace	ARTRACE / AWTRACE / RTRACE / BTRACE	Transported (`ENABLE_TRACE`)
Memory partitioning and monitoring (MPAM)	ARMPAM / AWMPAM	Transported (`ENABLE_MPAM`)
Memory encryption context	ARMECID / AWMECID	Transported (`ENABLE_MECID`)
Unique ID indicator	ARIDUNQ / AWIDUNQ / RIDUNQ / BIDUNQ	Transported (`ENABLE_UNIQUE`)
Read data chunking	ARCHUNKEN / RCHUNKV / RCHUNKNUM / RCHUNKSTRB	Transported (`ENABLE_CHUNKING`, read path only)
Memory Tagging Extension (MTE)	ARTAGOP / AWTAGOP / WTAG / WTAGUPDATE / RTAG / BTAGMATCH[1:0]	Transported (`ENABLE_MTE`)
Data poison	RPOISON / WPOISON	Transported (`ENABLE_POISON`)
Atomic transactions	AWATOP	Signal transported (`ENABLE_ATOMIC`); atomic execution is the endpoint's

AXI5 feature	Signals	Status in `rtl/amba/axi5/`
		responsibility
Interface wake-up	AWAKEUP	Transported (`ENABLE_WAKEUP`) and held while an accepted request waits on its READY (§A14.1.1); a `_cg` wrapper counts it as activity, so the clock is running when the first request after idle arrives

1.3 AXI5 Features Not Implemented

AXI5 feature	Signals	Notes
Loopback signalling	ARLOOP / AWLOOP	No ports; not carried
Separate MTE completion response	BCOMP	No port. A Match write must therefore be answered with a single combined response (BTAGMATCH 0b10/0b11, §A12.2); BTAGMATCH 0b01 “Separate” is carried through but has no BCOMP partner here
QoS acceptance	ARQOSACCEPT / AWQOSACCEPT	No ports; not carried
Untranslated transactions (SMMU)	AxMMUSID / AxMMUSSID / AxMMUSSIDV / AxMMUATST / AxMMUFLOW	No ports; not carried
Cache stash and CMO transactions	AWSTASHNID / AWSTASHLPID, CMO opcodes	Not implemented
Region identifier	ARREGION / AWREGION	Not implemented. AxREGION is still a valid optional AXI5 signal; it is simply omitted

AXI5 feature	Signals	Notes
		here. Route or decode by address instead

1.4 AXI4 vs AXI5

Feature	AXI4	AXI5
Outstanding transactions	No architectural limit; bounded by implementation	Unchanged; no architectural limit
Atomic transactions	Not supported	AWATOP: AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare
Memory tagging	Not supported	MTE: AxTAGOP, WTAG/RTAG, WTAGUPDATE, BTAGMATCH
Security identification	ARPROT/AWPROT[1] non-secure bit	AxPROT retained, plus AxNSAID for finer-grained security domains
QoS	ARQOS/AWQOS	AxQOS retained, plus optional AxQOSACCEPT
Memory partitioning	Not supported	MPAM (PartID + PMG)
Encryption context	Not supported	MECID
Trace	Not supported	AxTRACE / RTRACE / BTRACE
Read data chunking	Not supported	ARCHUNKEN, RCHUNKV, RCHUNKNUM, RCHUNKSTRB
Data poison	Not supported	RPOISON, WPOISON
Region identifier	ARREGION/ AWREGION (optional)	ARREGION/ AWREGION retained (optional)

1.5 Module Categories

1.5.1 Master Components

Module	Description	Documentation	Status
axi5_master_rd	AXI5 read master with skid buffers and burst support	axi5_master_rd.md	Documented
axi5_master_rd_cg	Clock-gated AXI5 read master	axi5_master_rd_cg.md	Documented
axi5_atomic_filter	Read-return atomic termination (store-class passes, load-class DECERRs)	axi5_atomic_filter.md	Documented
axi5_atomic_rr_tracker	Per-ID R routing for read-return atomics where a read path exists	axi5_atomic_rr_tracker.md	Documented
axi5_master_wr	AXI5 write master with address/data coordination	axi5_master_wr.md	Documented
axi5_master_wr_cg	Clock-gated AXI5 write master	axi5_master_wr_cg.md	Documented

1.5.2 Slave Components

Module	Description	Documentation	Status
axi5_slave_rd	AXI5 read slave with configurable response handling	axi5_slave_rd.md	Documented
axi5_slave_rd_cg	Clock-gated AXI5 read slave	axi5_slave_rd_cg.md	Documented
axi5_slave_wr	AXI5 write slave with write response generation	axi5_slave_wr.md	Documented
axi5_slave_wr_cg	Clock-gated AXI5 write slave	axi5_slave_wr_cg.md	Documented

1.5.3 Monitor Components

Module	Description	Documentation	Status
axi5_master_rd_mon	AXI5 read master with integrated transaction monitor	axi5_master_rd_mon.md	Documented
axi5_master_rd_mon_cg	Clock-gated read master with monitor	axi5_master_rd_mon_cg.md	Documented
axi5_master_wr_mon	AXI5 write master with integrated transaction monitor	axi5_master_wr_mon.md	Documented
axi5_master_wr_mon_cg	Clock-gated write master with monitor	axi5_master_wr_mon_cg.md	Documented
axi5_slave_rd_mon	AXI5 read slave with integrated transaction monitor	axi5_slave_rd_mon.md	Documented
axi5_slave_rd_mon_cg	Clock-gated read slave with monitor	axi5_slave_rd_mon_cg.md	Documented
axi5_slave_wr_mon	AXI5 write slave with integrated transaction monitor	axi5_slave_wr_mon.md	Documented
axi5_slave_wr_mon_cg	Clock-gated write slave with monitor	axi5_slave_wr_mon_cg.md	Documented

1.6 Key Features

1.6.1 AXI5 Channel Support

- **Full AXI5 signal set** for the features listed in *AXI5 Features Implemented* above
- **Burst types:** INCR, FIXED, and WRAP transported unmodified
- **ID transport:** Full `AXI_ID_WIDTH` transaction IDs; ordering and reordering are left to the endpoints
- **Transaction depth:** Not limited by these modules; set by the endpoints
- **Atomic transactions:** AWATOP transported for AtomicStore, AtomicLoad, AtomicSwap, and AtomicCompare

1.6.2 Advanced Features

- **Memory Tagging (MTE):** AxTAGOP, WTAG/RTAG, WTAGUPDATE, BTAGMATCH
- **Memory partitioning:** MPAM (PartID + PMG) and MECID encryption context
- **Poison support:** RPOISON/WPOISON for error propagation
- **Chunking:** ARCHUNKEN and the RCHUNK* response fields on the read path

1.6.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Idle Detection:** Automatic clock gate when interface is idle
- **Low-Power Modes:** Support for system low-power state integration

1.6.4 Monitoring and Debug

- **Integrated Monitors:** Combined master/slave + monitor modules
- **Transaction Tracking:** Complete transaction lifecycle monitoring
- **Performance Metrics:** Bandwidth, latency, and utilization
- **128-bit Monitor Bus:** Standardized packet format (+ 64-bit side-band timestamp)

1.7 Protocol Details

1.7.1 AXI5 Channel Overview

AXI5 maintains the five-channel architecture from AXI4:

Channel	Direction	Purpose
AR (Address Read)	Master to Slave	Read address and control
R (Read Data)	Slave to Master	Read data and response
AW (Address Write)	Master to Slave	Write address and control
W (Write Data)	Master to Slave	Write data
B (Write Response)	Slave to Master	Write response

1.7.2 AXI5 Sideband Signals Carried by These Modules

Signal	Channel	Description
AWATOP	AW	Atomic transaction type (see `axi5_master_wr.md` for the encoding)
ARNSAID / AWNSAID	AR / AW	Non-secure access identifier
ARTRACE / AWTRACE	AR / AW	Request trace marker
ARMPAM / AWMPAM	AR / AW	Memory partitioning and monitoring (PartID + PMG)
ARMECID / AWMECID	AR / AW	Memory encryption context identifier
ARIDUNQ / AWIDUNQ	AR / AW	Unique ID indicator (IHI 0022L §A5.2)
RIDUNQ / BIDUNQ	R / B	Unique ID indicator on the response, behind the same `ENABLE_UNIQUE`
ARCHUNKEN	AR	Read data chunking enable
ARTAGOP /	AR / AW	MTE tag operation

Signal	Channel	Description
AWTAGOP		
WPOISON	W	Write data poisoned indicator
WTAG / WTAGUPDATE	W	MTE write data tags and update mask
RPOISON	R	Read data poisoned indicator
RTRACE / BTRACE	R / B	Response trace marker
RCHUNKV / RCHUNKNUM / RCHUNKSTRB	R	Read chunk valid, number, and strobe
RTAG	R	MTE read data tags
BTAGMATCH[1:0]	B	MTE tag match result (Table A12.14)
AWAKEUP	AR / AW side	Wake-up request (§A14.1.1), behind `ENABLE_WAKEUP`; in a `_cg` wrapper it also wakes the gated clock

Signals absent from this list (ARLOOP/AWLOOP, ARQOSACCEPT/AWQOSACCEPT, ARREGION/AWREGION, the SMMU `AxMMU*` group, and stash/CMO encodings) have no ports on these modules. See *AXI5 Features Not Implemented* above.

1.7.3 Burst Types

ARBURST/AWBURST	Type	Description
2’b00	FIXED	Fixed address for FIFO access
2’b01	INCR	Incrementing address burst
2’b10	WRAP	Wrapping burst for cache line
2’b11	Reserved	Not used

1.8 Usage Example

The examples below are abbreviated: `.m_axi_ar*` style wildcards stand in for the full port list, and the AXI5 sideband ports are omitted for brevity. All AXI5 sideband ports exist on the module unconditionally — the `ENABLE_*` parameters control whether a signal is packed through the SKID buffer, not whether the port is present — so a real instantiation must connect or explicitly tie off every one of them. See the per-module pages for complete, connect-every-port examples.

1.8.1 Using AXI5 Read Master

```
axi5_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128),
    .AXI_USER_WIDTH(4)
) u_axi5_rd_master (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB (Functional Unit Block) interface
    .fub_axi_arid       (fub_arid),
    .fub_axi_araddr     (fub_araddr),
    .fub_axi_arlen      (fub_arlen),
    .fub_axi_arsize     (fub_arsize),
    .fub_axi_arburst    (fub_arburst),
    .fub_axi_arlock     (fub_arlock),
    .fub_axi_arcache    (fub_arcache),
    .fub_axi_arprot     (fub_arprot),
    .fub_axi_arqos      (fub_arqos),
    .fub_axi_aruser     (fub_aruser),
    .fub_axi_arvalid    (fub_arvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_arready    (fub_arready),

    .fub_axi_rid        (fub_rid),
    .fub_axi_rdata      (fub_rdata),
    .fub_axi_rresp      (fub_rresp),
    .fub_axi_rlast      (fub_rlast),
    .fub_axi_ruser      (fub_ruser),
    .fub_axi_rvalid     (fub_rvalid),
    .fub_axi_rready     (fub_rready),

    // AXI5 master interface
    .m_axi_ar*          (mem_ar*),
    .m_axi_r*           (mem_r*)
);
```

1.8.2 Using AXI5 Write Master

```
axi5_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128),
    .AXI_USER_WIDTH(4)
) u_axi5_wr_master (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB interface
    .fub_axi_awid       (fub_awid),
    .fub_axi_awaddr     (fub_awaddr),
    .fub_axi_awlen      (fub_awlen),
    .fub_axi_awsize     (fub_awsize),
    .fub_axi_awburst    (fub_awburst),
    .fub_axi_awlock     (fub_awlock),
    .fub_axi_awcache    (fub_awcache),
    .fub_axi_awprot     (fub_awprot),
    .fub_axi_awqos      (fub_awqos),
    .fub_axi_awuser     (fub_awuser),
    .fub_axi_awvalid    (fub_awvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_awready    (fub_awready),

    .fub_axi_wdata      (fub_wdata),
    .fub_axi_wstrb      (fub_wstrb),
    .fub_axi_wlast      (fub_wlast),
    .fub_axi_wuser      (fub_wuser),
    .fub_axi_wvalid     (fub_wvalid),
    .fub_axi_wready     (fub_wready),

    .fub_axi_bid        (fub_bid),
    .fub_axi_bresp      (fub_bresp),
    .fub_axi_buser      (fub_buser),
    .fub_axi_bvalid     (fub_bvalid),
    .fub_axi_bready     (fub_bready),

    // AXI5 master interface
    .m_axi_aw*          (mem_aw*),
    .m_axi_w*           (mem_w*),
    .m_axi_b*           (mem_b*)
);
```

1.8.3 Using Integrated Monitor Modules

```
// AXI5 master with built-in transaction monitoring
axi5_master_rd_mon #(
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(64),
    .AXI_DATA_WIDTH(128)
) u_axi5_rd_master_mon (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB and AXI interfaces (same as non-monitor version)
    .fub_axi_ar*        (...),
    .fub_axi_r*         (...),
    .m_axi_ar*          (...),
    .m_axi_r*           (...),

    // Monitor bus output
    .monbus_valid          (rd_mon_valid),
    .monbus_ready          (rd_mon_ready),
    .monbus_packet           (rd_mon_data)
);
```

1.9 Design Notes

1.9.1 FUB Interface Pattern

All AXI5 modules use the FUB (Functional Unit Block) interface pattern: - **fub_axi_*** signals connect to internal logic - **m_axi_*** or **s_axi_*** signals connect to external AXI bus - Skid buffers between FUB and external interfaces for timing closure

1.9.2 Migration from AXI4

These modules carry an AXI4-compatible core signal set, so an AXI4 agent can drive them: - Core protocol unchanged (5-channel architecture) - Every AXI5 sideband port can be tied off to its default value, and the matching `ENABLE_*` parameter set to 0 so the field costs no SKID buffer area - AWATOP tied to `6'b000000` (NonAtomic) makes every write a normal AXI4 write - User signal widths can match AXI4 configurations - **Exception:** AXI4's ARREGION/AWREGION have no port here. An AXI4 agent that relies on AxREGION must decode by address instead, or use the `axi4_master_*` / `axi4_slave_*` modules

1.9.3 Monitor Integration

The `*_mon` variants combine master/slave with integrated monitor: - Single instantiation for both data path and monitoring - Shared clock and reset - Monitor bus output for transaction visibility - Reduced resource usage vs. separate monitor instantiation

1.9.4 Performance Characteristics

These modules are SKID-buffer pipelines, so their cost is a small, fixed latency adder and their throughput ceiling is simply the bus bandwidth. No synthesis has been run against a specific ASIC node; the frequency figure below is an FPGA-oriented design target, not a signed-off characterization result.

Metric	Value
Maximum frequency	Design target; not characterized against a specific technology node
Added latency (per channel)	1 clock cycle per SKID stage
Throughput	One beat per clock, sustained (`AXI_DATA_WIDTH` bits/cycle)
Peak bandwidth, 128-bit bus at 500 MHz	8 GB/s per direction
Outstanding transactions	Not limited by these modules; bounded by the endpoints

1.10 Related Modules

- **AXI4 Modules** - AXI4 protocol components
- **APB5 Modules** - APB5 protocol components
- **AXIS5 Modules** - AXI5-Stream components
- **GAXI Modules** - Generic AXI utilities
- **Shared Infrastructure** - Common components

1.11 Testing

All AXI5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all AXI5 tests
pytest val/amba/test_axi5*.py -v

# Run specific module tests
pytest val/amba/test_axi5_master_rd.py -v
pytest val/amba/test_axi5_master_wr.py -v
pytest val/amba/test_axi5_slave_rd.py -v
pytest val/amba/test_axi5_slave_wr.py -v
```

`val/amba/test_axi5_spec_signals.py` checks the signal set itself, pin by pin, on every wrapper: no AWTAG, BTAG or RTAGMATCH; a 2-bit BTAGMATCH carried with all four Table A12.14 codes; AWIDUNQ/ARIDUNQ forward and BIDUNQ/RIDUNQ back; AWAKEUP passed through and held while an accepted request waits on its READY; and, on each `_cg` wrapper, AWAKEUP alone waking the gated clock so the next request is accepted on its first cycle. The feature-OFF builds check that every one of those outputs ties to 0.

1.12 References

1.12.1 Specifications

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest — the current AXI5 definition: §A2.3 “Valid-Ready transport”, §A6.4 “Atomic transactions”, §A12.2 “Memory Tagging Extension (MTE)”, §A16.1 “Data protection using Poison”, §A5.2 “Unique ID indicator”, §A5.6.2 “Read data chunking”, §A4.6 “Memory Encryption Contexts”.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h — ch. C1 “AMBA AXI5” and ch. E1 “Additional Features in AMBA 5” / E2 “Interface and data protection”, and the AXI4 base interface the AXI5 family extends.

1.12.2 Papers and textbooks

- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — register slices as relay stations.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC ’06)*, July 2006, pp. 657-662. DOI: 10.1145/1146909.1147077 — the elastic (skid) buffer under every wrapper.
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

1.12.3 Source Code

- RTL: `rtl/amba/axi5/`
- Tests: `val/amba/test_axi5*.py`
- Framework: `bin/TBClasses/components/axi4/`

Last Updated: 2026-07-19

1.13 Navigation

- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

2 AXI5 Atomic Filter

Module: `axi5_atomic_filter.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

2.1 Overview

The AXI5 Atomic Filter terminates **read-return atomic transactions** at a boundary whose downstream fabric can transport store-class atomics but cannot route the read data that load-class atomics return. It sits on the AW/W/B **control path**, plus an R-channel merge for the read data it owes; address and data payload buses route around it.

AXI5 `AWATOP[5:0]` encodes four transaction classes:

AWATOP	Class	Response	Filter action
`6'b000000`	Non-atomic write	B	**Forward**
`6'b01xxxx`	AtomicStore	B only	**Forward**
`6'b10xxxx`	AtomicLoad	B + original data on **R**	**Swallow** + **local DECERR on B and R**
`6'b11000x`	AtomicSwap / AtomicCompare	B + original data on **R**	**Swallow** + **local DECERR on B and R**

The load-class R response uses the AW's ID on the **read** channel. A fabric that splits write and read paths (such as the bridge crossbar, where every R-return tracker learns only about ARs) would never route that response — the transaction would hang. The filter guarantees it never reaches the fabric: the AW is accepted upstream but not forwarded, its W burst is consumed and discarded, a local `DECERR` B response returns with the AW's ID, and the read data transfers the atomic owes are returned on R, each `DECERR`, with `RID` = `AWID` and `RLAST` on the last one.

IHI 0022L §A6.4.8 requires those R transfers: "A Subordinate must send the correct number of read data transfers, even if the write response is DECERR or SLVERR", and a compliant manager waits for them. The count is §A6.4.7's:

Atomic	R transfers
AtomicLoad, AtomicSwap	`AWLEN + 1`
AtomicCompare, `AWLEN = 0`	1
AtomicCompare, `AWLEN > 0`	`(AWLEN + 1) / 2` (the compare and swap values share the write data; the original value is half as long)

Both Compare rows reduce to `(AWLEN >> 1) + 1`, which is what the RTL computes. A filter that answered on B only (the module before GitHub #102) left a compliant manager waiting for R forever.

2.1.1 Key Features

- Discriminates purely on `AWATOP[5]` — one wire, no decode table
- Route queue (push per AW, pop at WLAST) steers or sinks each W burst, so forwarded and swallowed bursts interleave correctly in AW order
- Read-return queue (`AWID` + beat count per swallowed atomic) drains local DECERR R beats whenever the downstream R channel is idle. Entries are visible as soon as the AW is accepted: §A6.4.7 lets a Subordinate send all read data before accepting any write data, and requires the Manager to take it.
- **R bursts never interleave.** Once a burst's first beat is accepted, the R source is held until `RLAST`, so a downstream read burst and a local read-return burst cannot mix on `s_r*`. Downstream wins only at a burst boundary.
- Response queue drains local DECERRs whenever the downstream B channel is idle. Downstream takes priority only when the filter is free to switch: once a beat is presented on `s_bvalid`, the chosen source is **held** until the master accepts it, and `m_bready` is qualified by that choice. Without the hold, a downstream B arriving under a stalled `s_bvalid` would swap `s_bid`/`s_bresp` mid-beat, which AXI forbids
- Control-plane only: `id`, `atop`, `awlen`, `wlast`, `rlast` and the eight handshake pairs (`s_aw`/`s_w`/`s_b`/`s_r` and `m_aw`/`m_w`/`m_b`/`m_r`). All other AW/W payload passes around the filter; the B payload (`bid`/`bresp`) and the R control payload (`rid`/`rresp`/`rlast`) are the filter's output muxes. `RDATA`, `RUSER` and the AXI5 R sidebands route around it too: on a local error beat they carry whatever the downstream R bus holds, which a DECERR makes don't-care
- Single clock domain, no protocol state machine — two small FIFOs and combinational routing

2.2 Parameters

Parameter	Default	Description
AXI_ID_WIDTH	4	Width of s_awid / s_bid / m_bid
AXI_ATOP_WIDTH	6	AWATOP width (the spec value; only bit [5] is examined)
DEPTH_LG2	3	log2 of the route, B-response and read-return queue depth (default 8 entries)

2.2.1 Derived Parameters (do not override)

These are declared as parameter so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
IW	AXI_ID_WIDTH
DEPTH	1 << DEPTH_LG2

2.3 Ports

Upstream (`s_*`) faces the boundary wrapper's FUB side; downstream (`m_*`) faces the fabric. Only control signals pass through:

Group	Signals
AW	`s_awvalid/awready/awid/awatop/awlen` → `m_awvalid/awready`
W	`s_wvalid/wready/wlast` → `m_wvalid/wready`
B	`m_bvalid/bready/bid/bresp` → `s_bvalid/bready/bid/bresp`
R	`m_rvalid/rready/rid/rresp/rlast` → `s_rvalid/rready/rid/rresp/rlast` (merged with the local read-return beats)

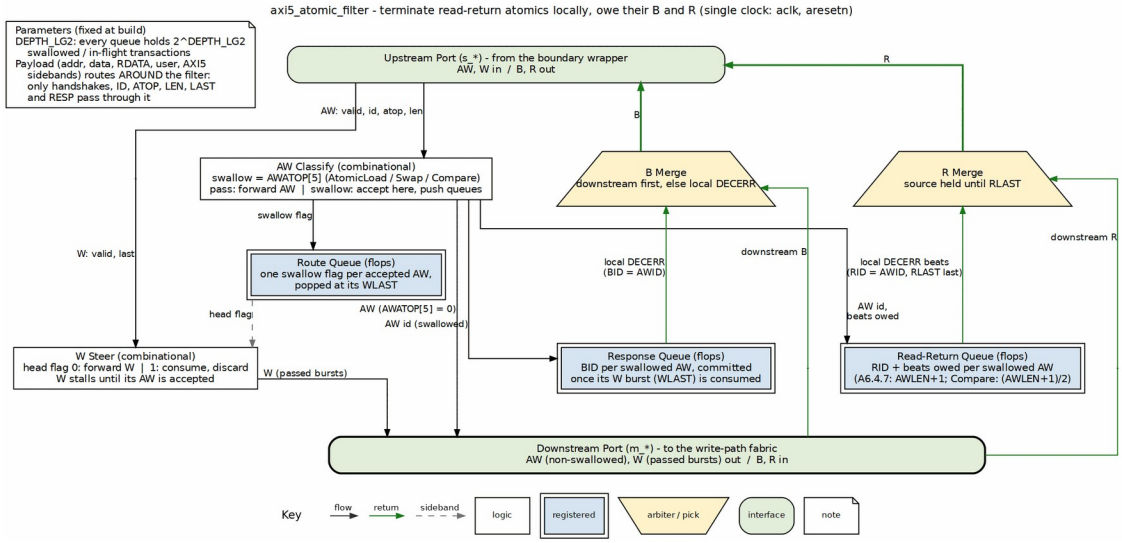
The ports added for the R path (GitHub #102 item 2):

Port	Width	Direction	Description
`s_awlen`	8	Input	AWLEN of the upstream AW; sets the R beat count of a swallowed atomic (§A6.4.7)
`s_rvalid`	1	Output	Merged read data valid toward the manager
`s_rready`	1	Input	Manager read data ready
`s_rid`	IW	Output	`m_rid`, or the swallowed atomic's `AWID`
`s_rresp`	2	Output	`m_rresp`, or `2'b11` (DECERR) on a local beat

Port	Width	Direction	Description
s_rlast	1	Output	m_rlast, or the last local beat of the atomic
m_rvalid	1	Input	Downstream read data valid
m_rready	1	Output	s_rready while the downstream source is selected
m_rid	IW	Input	Downstream RID
m_rresp	2	Input	Downstream RRESP
m_rlast	1	Input	Downstream RLAST
Clock/reset	aclk, aresetn (active-low async) – single domain, shared with both sides		

2.4 Functional Description

2.4.1 Block diagram



axi5_atomic_filter block diagram

The upstream port is at the top and the downstream (write-path fabric) port is the bar. W runs down the left through the W steer; AW comes down the center into the classifier. A read-return atomic (`AWATOP[5]` = 1: AtomicLoad, AtomicSwap, AtomicCompare) is swallowed: its AW is accepted but not forwarded, the route queue marks its W burst to be consumed and discarded, and two queues record what is owed – a DECERR B response (committed only once the W burst has been consumed) and the A6.4.7 read beats. Responses climb the right through two merges, which are the picks: downstream first, else the local DECERR, with the source held until the beat is accepted (B) or until RLAST (R). Their wide edge faces down because the contenders arrive from below. Address and data payload route around the filter.

The filter sits on the write path and decides, per AW, whether the burst is forwarded downstream or swallowed. An unsupported atomic (`AWATOP` naming a load, swap or compare) is swallowed: its W beats are discarded, a local DECERR is returned on B with the AW's ID, and its read data transfers are returned on R as DECERR beats with the same ID. Everything else passes through.

A route queue pushed per AW and popped at WLAST steers each W burst, so forwarded and swallowed bursts interleave correctly in AW order. A response queue holds the locally generated DECERRs, and the B output selects between that queue and the downstream response. A read-return queue holds `{AWID, beats-1}` per swallowed atomic plus a beat counter for its head, and the R output selects between it and the downstream R channel, holding the choice for a whole burst.

There is no protocol state machine: three small FIFOs, a beat counter and combinational routing.

2.5 Timing Characteristics

This module is **sequential**: it contains 3 `always_ff` block(s), clocked on `aclk` with active-low asynchronous reset `aresetn`. Outputs derived in those blocks are registered and therefore appear one clock after the inputs that produced them.

Per-path cycle counts are not enumerated here; read the block that drives the signal you care about. No synthesis frequency or area figures are quoted – none have been measured against a target device.

2.6 Usage Examples

Every parameter and port below is taken from the module declaration.

```
axi5_atomic_filter #(
    .AXI_ID_WIDTH          (4),
    .AXI_ATOP_WIDTH        (6),
    .DEPTH_LG2             (3),
    .IW                    (AXI_ID_WIDTH),
    .DEPTH                 (1 << DEPTH_LG2)
) u_axi5_atomic_filter (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .s_awvalid             (s_awvalid),
    .s_awready             (s_awready),
    .s_awid                (s_awid),
    .s_awatop              (s_awatop),
    .s_awlen               (s_awlen),
    .s_wvalid              (s_wvalid),
    .s_wready              (s_wready),
    .s_wlast               (s_wlast),
    .s_bvalid              (s_bvalid),
    .s_bready              (s_bready),
    .s_bid                 (s_bid),
    .s_bresp               (s_bresp),
    .s_rvalid              (s_rvalid),
    .s_rready              (s_rready),
    .s_rid                 (s_rid),
    .s_rresp               (s_rresp),
    .s_rlast               (s_rlast),
    .m_awvalid             (m_awvalid),
    .m_awready             (m_awready),
    .m_wvalid              (m_wvalid),
    .m_wready              (m_wready),
    .m_bvalid              (m_bvalid),
    .m_bready              (m_bready),
    .m_bid                 (m_bid),
    .m_bresp               (m_bresp),
    .m_rvalid              (m_rvalid),
    .m_rready              (m_rready),
    .m_rid                 (m_rid),
    .m_rresp               (m_rresp),
    .m_rlast               (m_rlast)
);
```

2.6.1 Integration

The bridge generator (bridge TASK-002 (was BRIDGE-002) A5-3a) inserts this filter automatically in the master adapter of any AXI5 master port with `atomic` in its `axi5_features`, between the `axi5_slave_wr` boundary wrapper (`pref_axi_*`) and the fabric-facing `fub_axi_*` namespace. Handshakes and the B payload go through the filter; every other signal gets a `pref → fub` pass-through assign. Swallowed transactions never assert `m_awvalid`, so stale pass-through payload is inert.

Standalone integrations follow the same recipe: route payload around, handshakes through.

2.6.2 Design Notes and Constraints

- **W stalls until its AW is queued.** `s_wready` is held low while the route queue is empty. AW acceptance never depends on W, so this cannot deadlock; it simply serializes W-before-AW masters at this boundary.
- **Same-ID B and R ordering.** A local DECERR (B or R) can pass an in-flight forwarded transaction's response. The AXI atomic rules already require atomics not to share an ID with outstanding transactions, so a compliant master never observes the reorder.
- **DECERR, not SLVERR.** An unsupported atomic at this boundary is "no such capability at this address path" — the decode-class error.
- The filter is intentionally not parameterized to pass load-class atomics through: a fabric that CAN route read returns (A5-3b's shared per-ID tracker) simply omits the filter.

2.7 Design Notes

The B selection must be HELD once presented. AXI requires BID and BRESP to be stable from BVALID until the handshake. Selecting combinationally between the downstream response and a queued DECERR lets a downstream B arriving under a stalled `s_bvalid` change the payload mid-beat; the module latches which source is presented and releases only on accept, and qualifies `m_bready` by that choice so a downstream response is not consumed while the master is still looking at a DECERR.

DECERR, not SLVERR. An unsupported atomic at this boundary is “no such capability here”, which is a decode error.

The R channel passes through it. The filter has eight handshake pairs – `s_aw`/`s_w`/`s_b`/`s_r` and `m_aw`/`m_w`/`m_b`/`m_r`. AR does not pass through it; only R does, because that is where a swallowed atomic’s read data must appear. The R selection is held twice over: under backpressure (payload stable from RVALID to the handshake, as for B) and for a whole burst (no interleaving of local and downstream bursts).

2.8 Testing

`val/amba/test_axi5_atomic_filter.py` — direct cocotb driving: mixed forward/swallow traffic, a multi-beat swallowed burst, DECERR ID/order checks, and forwarded-set assertions.

`atomic_filter_read_return_r_test` covers the R path: AtomicLoad, AtomicSwap and AtomicCompare at several `AWLEN`s, each checked for its exact §A6.4.7 beat count, `RID`, DECERR on every beat and `RLAST` on the last one, while a downstream R stream shares the channel under random manager backpressure. It also checks that bursts from the two sources never interleave and that the presented R payload holds from RVALID to the handshake. Each of three RTL mutations (no burst hold, no backpressure hold, Compare not halved) fails it; on the pre-#102 module it fails at once, because there is no R channel.

Until GitHub #102 this file passed its tests to cocotb as a comma-separated `testcase=` list. Under cocotb 2.x that matched no test at all, so the file ran zero tests and still reported PASS; it now runs every test in the module.

Most phases use an always-ready downstream model, which cannot exercise the B selection hold: with `s_bready` high throughout, no beat is ever presented and unaccepted, so the stability window never opens. A dedicated phase therefore stalls the master, queues a local DECERR, then raises `m_bvalid` underneath it and asserts `s_bid`/`s_bresp` hold for eight cycles. Removing the hold from the RTL fails that phase at cycle 0; nothing else in the suite notices. End-to-end behavior (real ATOP values through a generated bridge, memory side-effect checks) is covered by `projects/components/fabric-gen-ip/bridge/dv/tests/test_bridge_1x2_wr_axi5a_atomics.py`.

2.9 Related Modules

- axi5_slave_wr — the boundary wrapper upstream of the filter in bridge master adapters
- axi5_atomic_rr_tracker — the alternative for a fabric that CAN route read-return data
- AXI5 index — scope statement for the AXI5 family

2.10 References

- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h – §E1.1.8 “Response signaling”: an error response is permitted from a component that does not support atomics, but for AtomicLoad, AtomicSwap and AtomicCompare “a slave must send the correct number of read data beats, even if the write response is DECERR or SLVERR”. This filter does exactly that: a DECERR on B and the full count of DECERR read beats on R (Issue L §A6.4.8; GitHub #102 item 2).
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h – §E1.1.4 “ID use for Atomic transactions” (Issue L §A6.4.4, “unique-in-flight”): why a local DECERR passing a same-ID B is invisible to a compliant master; §A3.3.1 “Dependencies between channel handshake signals”: B only after the last W beat, which is why the local DECERR is committed at WLAST.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A6.4.2 “Atomic transaction operations” and §A6.4.6 “Atomic transaction signaling” (the AWATOP encodings; bit 5 set marks the read-return classes); §A6.4.7 “Transaction structure” (the number of read data transfers: AWLEN+1, halved for a multi-transfer AtomicCompare); §A6.4.8 “Response signaling” (the read transfers are owed even when the write response is an error).
- Cummings, C.E. “Simulation and Synthesis Techniques for Asynchronous FIFO Design.” *SNUG San Jose*, 2002 – the wrap-bit full/empty test used by the route and response queues.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/cdc/references/`: `AsyncFifoGrayPointers__Cummings2002.pdf`)

2.11 Navigation

- ← Back to axi5 index
- ← Back to rtl-amba index

3 AXI5 Atomic Read-Return Tracker

Module: `axi5_atomic_rr_tracker.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

3.1 Overview

The AXI5 Atomic Read-Return Tracker gives a fabric the one piece of state it is missing when a **read-return atomic** passes through it: which read data channel, and which requester, an R beat belongs to when no AR was ever issued for it.

AXI5 `AWATOP[5:0]` splits atomics into two classes by bit 5:

AWATOP	Class	Response
`6'b01xxxx`	AtomicStore	B only
`6'b10xxxx`	AtomicLoad	B, **plus** the location's original data on **R**
`6'b11000x`	AtomicSwap / AtomicCompare	B, **plus** the original data on **R**

The read-return classes answer on the **read** data channel using the **AW's ID**. A fabric that splits its write and read paths (the bridge crossbar is one) routes R beats with trackers that learn only from AR handshakes, so the atomic's R beat matches nothing and is never delivered. `axi5_atomic_filter` solves that at a boundary with no read path by refusing the classes. This block solves it where a read path exists: it records the atomic at its AW handshake and answers the R beat's routing question from its ID.

3.1.1 Key Features

- Keyed on ID alone. AXI5 forbids an atomic from sharing its ID with any transaction outstanding from the same Manager, so within one requester's traffic a live ID is unambiguous
- Combinational lookup from the presented `RID`: `hit` says the beat is a tracked atomic's, `hit_data` says where it goes
- Allocate and release may land in the same cycle; they can never target the same slot, because release needs a live entry and allocate needs a free one
- `full` is a registered view. The caller must not allocate while it is set; in the bridge the slave adapter holds the atomic AW's `awready` on it
- No protocol state machine: a valid vector, two small arrays and two priority encoders

3.2 Parameters

Parameter	Default	Description
AXI_ID_WIDTH	4	Width of alloc_id / lookup_id
DATA_WIDTH	1	Width of the routing tag stored per entry (the bridge stores its master index)
DEPTH	4	Number of read-return atomics that may be outstanding at once. The bridge's slave adapter builds it with 8, so a bridge slave port holds at most 8 in flight and stalls the ninth AW

3.2.1 Derived Parameters (do not override)

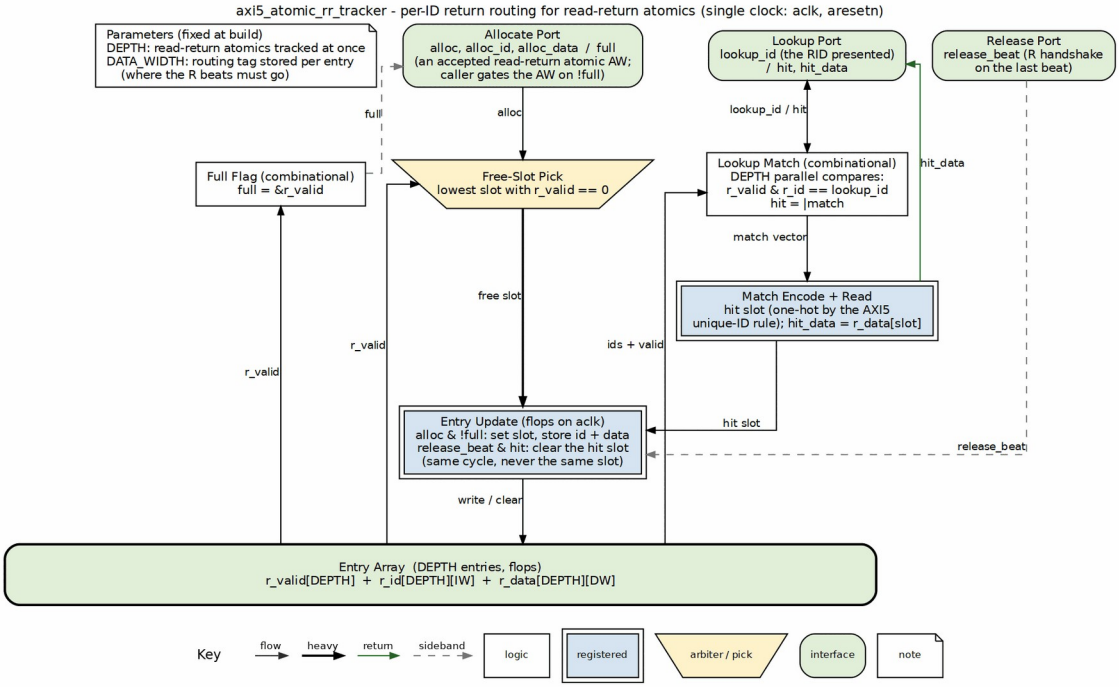
Derived parameter	Default expression
IW	AXI_ID_WIDTH
DW	DATA_WIDTH

3.3 Ports

Group	Signals	Direction	Meaning
Allocate	`alloc`, `alloc_id`, `alloc_data`	in	An accepted read-return atomic AW: its ID and routing tag
	`full`	out	Every slot live; do not allocate
Lookup	`lookup_id`	in	The `RID` being presented
	`hit`, `hit_data`	out	This beat belongs to a tracked atomic, and its routing tag
Release	`release_beat`	in	The R handshake's last beat; frees the matching entry when `hit` is set, ignored otherwise
Clock/reset	`aclk`, `aresetn`	in	Single domain, active-low asynchronous reset

3.4 Functional Description

3.4.1 Block diagram



axi5_atomic_rr_tracker block diagram

A small CAM with three request flows, one column each. Allocate takes the lowest free slot (the trapezium) and stores the atomic's AW ID and its routing tag; `full` goes back to the caller, which gates the AW on it. Lookup compares the RID being presented against every live entry in parallel; live IDs are unique by the AXI5 atomic rule, so the match is at most one-hot and returns `hit` and `hit_data` the same cycle. Release clears the hit slot on the R handshake of the last beat. Allocate and release can share a cycle but never a slot.

At a slave-side boundary the adapter connects `alloc` to the AW handshake qualified by `AWATOP[5]`, `alloc_id` to `AWID`, and `alloc_data` to the bridge's requester index for that AW. On the read side it presents `RID` on `lookup_id`. When `hit` is set the adapter routes the beat by `hit_data` and does **not** advance its in-order AR tracker; every other beat is routed by that tracker exactly as before. `release_beat` is the R handshake on its last beat, and the tracker frees the entry only if the beat was one of its own.

Two requesters may present the same ID at one Subordinate. That aliasing is a property of the surrounding fabric, not of this block: the generated bridge adapter reports it in simulation when an R beat matches both a tracked atomic and the head of the in-order tracker.

3.5 Timing Characteristics

This module is **sequential**: one `always_ff` block clocked on `aclk` with active-low asynchronous reset `aresetn`. `hit` and `hit_data` are combinational from `lookup_id` and the stored entries, so a beat is routable in the cycle it is presented. An allocation is visible to lookup on the cycle after its handshake.

3.6 Usage Examples

Every parameter and port below is taken from the module declaration. This is how the bridge's slave adapter instantiates it beside its in-order read FIFO:

```
axi5_atomic_rr_tracker #(
    .AXI_ID_WIDTH (ID_WIDTH),
    .DATA_WIDTH   (BRIDGE_ID_WIDTH),
    .DEPTH        (8)
) u_atom_rd (
    .aclk         (aclk),
    .aresetn      (aresetn),
    .alloc        (xbar_awvalid && xbar_awready && xbar_awatop[5]),
    .alloc_id     (xbar_awid),
    .alloc_data   (xbar_bridge_id_aw),
    .full         (atom_full),
    .lookup_id    (xbar_rid),
    .hit          (atom_hit),
    .hit_data     (atom_bridge_id),
    .release_beat (xbar_rvalid && xbar_rready && xbar_rlast)
);

assign rid_bridge_id = atom_hit ? atom_bridge_id : rd_fifo_head;
assign rid_valid     = atom_hit || rd_fifo_nonempty;
```

3.7 Verification

`val/amba/test_axi5_atomic_rr_tracker.py`: directed allocate / lookup / release / fill-to-full / same-cycle allocate-and-release, then a random phase against a dictionary model checked on every round, scaled by TEST_LEVEL. The end-to-end path is exercised by the bridge fixture `bridge_1x2_rw_axi5a` and its hand-written `test_bridge_1x2_rw_axi5a_atomics.py`.

3.8 Related

- `axi5_atomic_filter`: the other half of the story, for a boundary that has no read path and must refuse the classes instead
- Bridge MAS, AMBA5 boundary chapter (bridge TASK-002 (was BRIDGE-002) A5-3b)

3.9 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A6.4.4 “ID use for Atomic transactions”: one ID for the request, write response and read data, and the ID unique-in-flight, which is what makes an ID-only key sound; §A6.4.8 “Response signaling” (the read-return classes answer on R). Issue H: §E1.1.4 and §E1.1.8.
- Pagiamtzis, K., Sheikholeslami, A. “Content-Addressable Memory (CAM) Circuits and Architectures: A Tutorial and Survey.” *IEEE J. Solid-State Circuits*, 41(3), Mar. 2006. DOI: 10.1109/JSSC.2005.864128 – the parallel compare of every valid entry against the presented RID.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `ContentAddressableMemory__Pagiamtzis2006.pdf`)

4 AXI5 Master Read

Module: `axi5_master_rd.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

4.1 Overview

The AXI5 Master Read module is the master-side AR/R channel transport block. It carries a full AXI5 read signal set — including the AXI5 sideband extensions — between a FUB (Functional Unit Block) interface and an external AXI5 master interface, with a configurable SKID buffer on each channel for timing closure and flow control.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no outstanding-transaction tracking. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

4.1.1 Key Features

- Carries the full AXI5 signal set listed below, unmodified, across the SKID buffers
- **ARNSAID:** Non-secure access identifier for security domains
- **ARTRACE:** Trace signal for debug and performance monitoring
- **ARMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID:** Memory Encryption Context ID for secure memory
- **ARIDUNQ / RIDUNQ:** Unique ID indicator on the request and on every read data beat (IHI 0022L §A5.2)
- **ARCHUNKEN:** Read data chunking enable for partial data transfers
- **ARTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **RTRACE:** Read data trace signal
- **RPOISON:** Data poison indicator for corrupted data detection
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking control signals
- **RTAG:** Memory tags returned with read data (MTE)
- Configurable SKID buffer depths for AR and R channels
- Busy signal for power management and clock gating
- ARREGION not implemented (see *Design Notes*)

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	R channel SKID buffer depth (legal range 2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID

Parameter	Type	Default	Description
			indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low

4.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `rtag` field
CHUNK_STRB_WIDTH	max(AXI_DATA_WIDTH / 128, 1)	Width of `rchunkstrb`, one bit per 128-bit granule
ARSize	Sum of the enabled AR fields	AR SKID buffer payload width
RSize	Sum of the enabled R fields	R SKID buffer payload width

Note on AXI_WSTRB_WIDTH: this module declares `AXI_WSTRB_WIDTH` (default `AXI_DATA_WIDTH/8`) and its short alias `SW`, but neither is used. A read module has no W channel and therefore no write strobes. The parameter is retained only so the read and write module parameter lists line up; overriding it has no effect.

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

4.3.2 FUB AXI5 Interface (Slave Side - Input)

4.3.2.1 AR Channel

Port	Width	Direction	Description
fub_axi_arid	IW	Input	Read address ID
fub_axi_araddr	AW	Input	Read address
fub_axi_arlen	8	Input	Burst length
fub_axi_arsize	3	Input	Burst size
fub_axi_arburst	2	Input	Burst type
fub_axi_arlock	1	Input	Lock type
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_aruser	UW	Input	User-defined signal
fub_axi_arvalid	1	Input	Read address valid
fub_axi_aw	1	Input	Wake-up request,

Port	Width	Direction	Description
akeup			AWAKEUP (IHI 0022L §A14.1.1; ENABLE_WAKEUP)
fub_axi_arready	1	Output	Read address ready

4.3.2.2 AXI5 AR Extensions

Port	Width	Direction	Description
fub_axi_arnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_artrace	1	Input	Trace signal
fub_axi_armpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_armecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_aridunq	1	Input	Unique ID indicator, ARIDUNQ (IHI 0022L §A5.2; ENABLE_UNIQUE)
fub_axi_archunken	1	Input	Chunking enable
fub_axi_artagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

4.3.2.3 R Channel

Port	Width	Direction	Description
fub_axi_rid	IW	Output	Read data ID
fub_axi_rdata	DW	Output	Read data
fub_axi_rresp	2	Output	Read response
fub_axi_rlast	1	Output	Last transfer in burst
fub_axi_ruser	UW	Output	User-defined signal
fub_axi_rvalid	1	Output	Read data valid

Port	Width	Direction	Description
fub_axi_rready	1	Input	Read data ready

4.3.2.4 AXI5 R Extensions

Port	Width	Direction	Description
fub_axi_rtrace	1	Output	Trace signal
fub_axi_riduq	1	Output	Unique ID indicator on every read data beat, RIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_rpoison	1	Output	Data poison indicator
fub_axi_rchunkv	1	Output	Chunk valid
fub_axi_rchunknum	AXI_CHUNKNUM_WIDTH	Output	Chunk number
fub_axi_rchunkstrb	CHUNK_STRB_WIDTH	Output	Chunk strobe (128-bit granules)
fub_axi_rtag	TW	Output	Memory tags (NUM_TAGS × TAG_WIDTH)

4.3.3 Master AXI5 Interface (Output Side)

Same port list as FUB interface but with `m_axi_*` prefix and reversed directions.

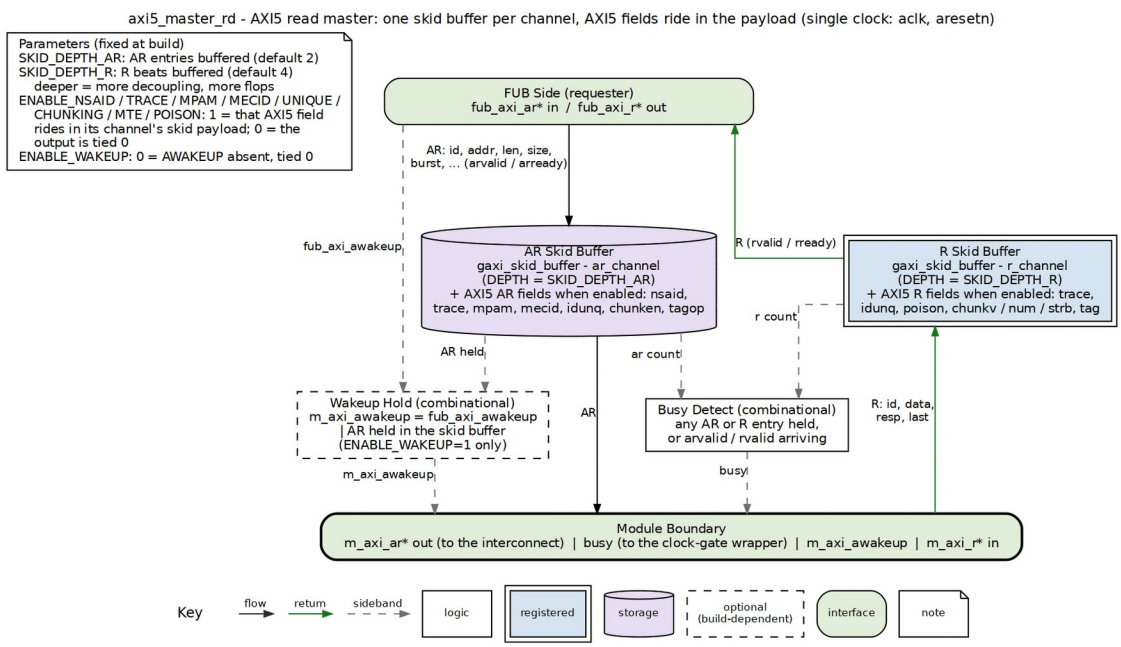
4.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator for clock gating

4.4 Functional Description

4.4.1 Architecture

Two channels, two SKID buffers, conditional packing in between.



axi5_master_rd block diagram

The requester (FUB) side is at the top and the module boundary is the bar. AR goes down the left lane through its skid buffer and R comes back up the right lane through its own. The AXI5 fields each channel carries (NSAID, trace, MPAM, MECID, unique-ID, MTE tags, poison, chunking) ride in that channel's skid payload when their `ENABLE_*` parameter is set; a disabled field's output is tied to 0. The busy detect feeds the clock-gate wrapper, and the wakeup hold keeps `m_axi_awakeup` asserted while an accepted request is still waiting in a skid buffer, not only while `fub_axi_awakeup` is high (IHI 0022L A14.1.1).

4.4.2 AXI5 Enhancements Over AXI4

Security and Isolation: - **ARNSAID:** Identifies security domain for non-secure accesses - **ARMECID:** Provides encryption context for secure memory regions

Performance and Monitoring: - **ARTRACE:** Enables performance trace and debug capabilities - **ARMPAM:** Supports memory bandwidth partitioning and QoS

Advanced Features: - **ARIDUNQ / RIDUNQ:** Unique ID indicator on the request and on every read data beat (IHI 0022L §A5.2) - **ARCHUNKEN:** Enables partial data transfer (chunking) - **ARTAGOP/RTAG:** Memory Tagging Extension for security

Data Integrity: - **RPOISON:** Flags corrupted or invalid data -
RCHUNKV/RCHUNKNUM/RCHUNKSTRB: Controls chunked data transfers

Not implemented: - **ARREGION:** No port on this module. AxREGION is not deprecated by AXI5; it remains a valid optional signal and is simply omitted here. Decode or route by address instead, or use `axi4_master_rd`

4.4.3 Signal Packing Logic

Packing is dynamic, driven by the feature enables:

```
// AR channel packed conditionally
if (ENABLE_NSAID)    → pack ARNSAID
if (ENABLE_TRACE)    → pack ARTRACE
if (ENABLE_MPAM)     → pack ARMPAM
if (ENABLE_MECID)    → pack ARMECID
if (ENABLE_UNIQUE)   → pack ARIDUNQ
if (ENABLE_CHUNKING) → pack ARCHUNKEN
if (ENABLE_MTE)      → pack ARTAGOP

// R channel unpacked conditionally
if (ENABLE_TRACE)    → unpack RTRACE
if (ENABLE_POISON)   → unpack RPOISON
if (ENABLE_CHUNKING) → unpack RCHUNKV/RCHUNKNUM/RCHUNKSTRB
if (ENABLE_MTE)      → unpack RTAG
```

Disabled signals default to 0.

4.5 Timing Characteristics

4.5.1 Basic Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- ARID, ARADDR, ARLEN, ARSIZE
- ARVALID, ARREADY
- ARNSAID, ARTRACE, ARMPAM (AXI5 extensions)
- RID, RDATA, RRESP, RLAST
- RVALID, RREADY
- RTRACE, RPOISON (AXI5 extensions)

4.5.2 Chunked Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- ARCHUNKEN assertion
- RCHUNKV, RCHUNKNUM, RCHUNKSTRB
- Partial data transfer pattern

4.5.3 Memory Tagging Extension (MTE)

Timing diagram pending. The signals and sequence this scenario exercises:

- ARTAGOP encoding
- RTAG delivery with data

4.6 Usage Examples

A full-featured instantiation with every AXI5 extension enabled:

```
axi5_master_rd #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    // Enable AXI5 features
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_master_rd (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // FUB interface (slave side)
    .fub_axi_arid       (fub_arid),
    .fub_axi_araddr     (fub_araddr),
    .fub_axi_arlen      (fub_arlen),
    .fub_axi_arsize     (fub_arsize),
    .fub_axi_arburst    (fub_arburst),
    .fub_axi_arlock     (fub_arlock),
    .fub_axi_arcache    (fub_arcache),
    .fub_axi_arprot     (fub_arprot),
    .fub_axi_arqos      (fub_arqos),
    .fub_axi_aruser     (fub_aruser),
    .fub_axi_arvalid    (fub_arvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_arready    (fub_arready),

    // AXI5 AR extensions
    .fub_axi_arnsaid    (fub_arnsaid),
    .fub_axi_artrace    (fub_artrace),
    .fub_axi_armpam     (fub_armpam),
    .fub_axi_armecid    (fub_armecid),
    .fub_axi_aridunq   (fub_aridunq),
    .fub_axi_archunken  (fub_archunken),
    .fub_axi_artagop    (fub_artagop),
```

```
    // R channel
    .fub_axi_rid        (fub_rid),
    .fub_axi_rdata      (fub_rdata),
    .fub_axi_rresp      (fub_rresp),
    .fub_axi_rlast      (fub_rlast),
    .fub_axi_ruser      (fub_ruser),
    .fub_axi_rvalid     (fub_rvalid),
    .fub_axi_rready     (fub_rready),

    // AXI5 R extensions
    .fub_axi_rtrace     (fub_rtrace),
    .fub_axi_ridunq     (fub_ridunq),
    .fub_axi_rpoison    (fub_rpoison),
    .fub_axi_rchunkv    (fub_rchunkv),
    .fub_axi_rchunknum  (fub_rchunknum),
    .fub_axi_rchunkstrb (fub_rchunkstrb),
    .fub_axi_rtag       (fub_rtag),

    // Master interface (output side)
    .m_axi_arid         (m_axi_arid),
    .m_axi_araddr       (m_axi_araddr),
    // Every remaining m_axi_* port mirrors the fub_axi_* list above,
    // same names and widths, opposite directions. All must be
connected.

    // Status
    .busy               (master_rd_busy)
);
```

4.7 Design Notes

4.7.1 AXI5 vs AXI4 Differences

Feature	AXI4	AXI5
Security domains	None	NSAID
Trace support	None	TRACE signals
Memory partitioning	None	MPAM
Encryption context	None	MECID
Unique ID indicator	None	ARIDUNQ / RIDUNQ
Data chunking	None	CHUNKING
Memory tagging	None	MTE (TAGOP, RTAG)
Poison indication	None	POISON
Region field	ARREGION (optional)	ARREGION retained by the spec; no port on this module

4.7.2 Memory Tagging Extension (MTE)

When ENABLE_MTE=1: - **ARTAGOP:** Specifies tag operation (Match/Insert/Fetch) - **RTAG:** Delivers memory tags (1 tag per 16 bytes of data) - **NUM_TAGS:** Calculated as (DATA_WIDTH / 128), minimum 1

Example for 128-bit data bus: - NUM_TAGS = 1 - TW = AXI_TAG_WIDTH × 1 = 4 bits

4.7.3 Chunking Support

When ENABLE_CHUNKING=1: - **ARCHUNKEN:** Master requests chunked response - **RCHUNKV:** Indicates chunk is valid - **RCHUNKNUM:** Identifies chunk within transaction - **RCHUNKSTRB:** Byte strobes at 128-bit granularity

4.7.4 Feature Enable Strategy

Disable unused features to reduce area:

```
.ENABLE_NSAID    (0),  // No security domains
.ENABLE_TRACE    (0),  // No trace capability
.ENABLE_MPAM     (0),  // No memory partitioning
.ENABLE_MECID    (0),  // No encryption
.ENABLE_UNIQUE   (0),  // No unique-ID indicator
.ENABLE_CHUNKING (0),  // No chunking
.ENABLE_MTE      (0),  // No memory tagging
.ENABLE_POISON   (0)   // No poison detection
```

4.8 Related Modules

- **AXI5 Master Write** - Master write interface
- **AXI5 Slave Read** - Slave read interface
- **AXI5 Master Read CG** - Clock-gated variant
- **AXI5 Master Read Monitor** - With integrated monitoring
- **AXI4 Master Read** - AXI4 version for comparison

4.9 Testing

`val/amba/test_axi5_master_rd.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_rd.py -v
```

4.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest — §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A5.6.2 “Read data chunking” (ARCHUNKEN, RCHUNKV/NUM/STRB), §A16.1 “Data protection using Poison” (RPOISON), §A12.2 “Memory Tagging Extension (MTE)” (ARTAGOP, RTAG), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (ARMECID). The ports follow the specification’s names (GitHub #102): there is no RTAGMATCH, the Unique ID indicator is ARIDUNQ with RIDUNQ on every read data beat, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h — the same AMBA 5 features as first numbered: ch. C1 “AMBA AXI5”, §E1.11 “Read data chunking”, §E1.15 “Memory tagging”, §E2.1 “Poison”, §E1.13 “Unique ID indicator”. MECID is not in Issue H.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — relay stations: a channel can be pipelined by any number of back-pressured stages without changing its behavior, which is why a skid buffer per channel is transparent to both ends.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC ’06)*, July 2006, pp. 657-662. DOI: 10.1145/1146909.1147077 — the elastic buffer with valid/stop back-pressure, the structure a skid buffer implements.
- Chipressian. *Crack the Hardware Interview: from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (forward and backward register slices) — textbook treatment of the valid/ready slice (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

4.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

5 AXI5 Master Read with Clock Gating

Module: `axi5_master_rd_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

5.1 Overview

The AXI5 Master Read with Clock Gating module wraps the standard `axi5_master_rd` core with intelligent clock gating for power optimization. When no AXI activity is detected for a configurable number of idle cycles, the clock gates off automatically — no software involvement, no protocol changes.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no outstanding-transaction tracking. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

5.1.1 Key Features

- Carries the same AXI5 signal set as `axi5_master_rd`, which it wraps
- **ARNSAID:** Non-secure access identifier for security domains
- **ARTRACE:** Trace signal for debug and performance monitoring
- **ARMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID:** Memory Encryption Context ID for secure memory
- **ARIDUNQ / RIDUNQ:** Unique ID indicator on the request and on every read data beat (IHI 0022L §A5.2)
- **ARCHUNKEN:** Read data chunking enable for partial data transfers
- **ARTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **RTRACE:** Read data trace signal
- **RPOISON:** Data poison indicator for corrupted data detection
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking control signals
- **RTAG:** Memory tags returned with read data (MTE)
- **Automatic clock gating** based on activity detection
- **Configurable idle count** before clock gating activates
- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Status outputs** for clock gating state monitoring
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the

Parameter	Type	Default	Description
			responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

5.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `rtag` field
CHUNK_STRB_WIDTH	max(AXI_DATA_WIDTH / 128, 1)	Width of `rchunkstrb`, one bit per 128-bit granule
ARSize	Sum of the enabled AR fields	AR SKID buffer payload width
RSize	Sum of the enabled R fields	R SKID buffer payload width

Note on AXI_WSTRB_WIDTH: this module declares `AXI_WSTRB_WIDTH` (default `AXI_DATA_WIDTH/8`) and its short alias `SW`, but neither is used. A read module has no W channel and therefore no write strobes. The parameter is retained only so the read and write module parameter lists line up; overriding it has no effect.

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

5.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

5.3.3 FUB AXI5 Interface (Slave Side - Input)

5.3.3.1 AR Channel

Port	Width	Direction	Description
fub_axi_arid	IW	Input	Read address ID
fub_axi_araddr	AW	Input	Read address
fub_axi_arlen	8	Input	Burst length
fub_axi_arsize	3	Input	Burst size
fub_axi_arburst	2	Input	Burst type
fub_axi_arlock	1	Input	Lock type
fub_axi_arcache	4	Input	Cache attributes
fub_axi_ar	3	Input	Protection attributes

Port	Width	Direction	Description
prot			
fub_axi_arqos	4	Input	Quality of Service
fub_axi_aruser	UW	Input	User-defined signal
fub_axi_arvalid	1	Input	Read address valid
fub_axi_awakeup	1	Input	Wake-up request, AWAKEUP (IHI 0022L §A14.1.1; `ENABLE_WAKEUP`)
fub_axi_arready	1	Output	Read address ready (forced 0 when gating)

5.3.3.2 AXI5 AR Extensions

Port	Width	Direction	Description
fub_axi_arnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_artrace	1	Input	Trace signal
fub_axi_armpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_armecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_aridunq	1	Input	Unique ID indicator, ARIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_archunken	1	Input	Chunking enable
fub_axi_artagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

5.3.3.3 R Channel

Port	Width	Direction	Description
fub_axi_rid	IW	Output	Read data ID

Port	Width	Direction	Description
fub_axi_rdata	DW	Output	Read data
fub_axi_rresp	2	Output	Read response
fub_axi_rlast	1	Output	Last transfer in burst
fub_axi_ruser	UW	Output	User-defined signal
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

5.3.3.4 AXI5 R Extensions

Port	Width	Direction	Description
fub_axi_rtrace	1	Output	Trace signal
fub_axi_ridunq	1	Output	Unique ID indicator on every read data beat, RIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_rpoison	1	Output	Data poison indicator
fub_axi_rchunkv	1	Output	Chunk valid
fub_axi_rchunknum	AXI_CHUNKNUM_WIDTH	Output	Chunk number
fub_axi_rchunkstrb	CHUNK_STRB_WIDTH	Output	Chunk strobe (128-bit granules)
fub_axi_rtag	TW	Output	Memory tags (NUM_TAGS × TAG_WIDTH)

5.3.4 Master AXI5 Interface (Output Side)

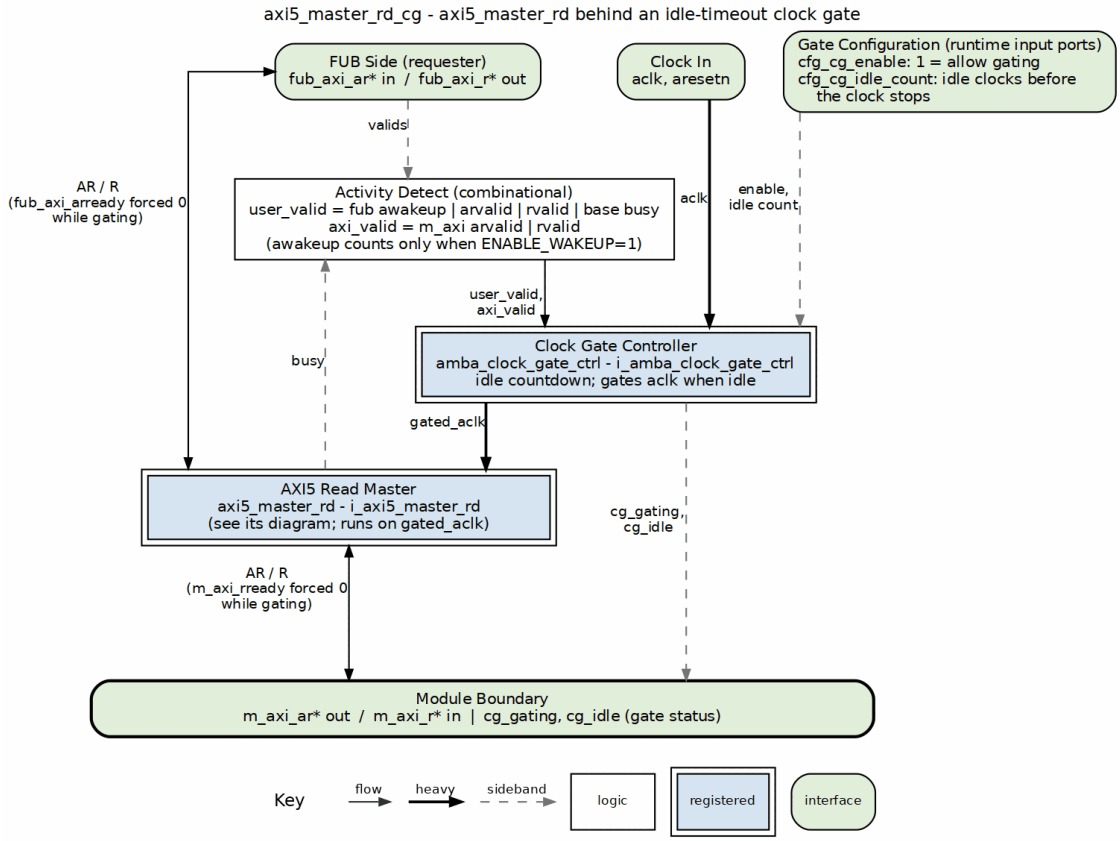
Same port list as FUB interface but with `m_axi_*` prefix and reversed directions.

5.3.5 Clock Gating Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Clock gating active (1=clock gated, 0=clock running)
cg_idle	1	Output	Interface idle (1=no activity detected)

5.4 Functional Description

5.4.1 Architecture



axi5_master_rd_cg block diagram

The base `axi5_master_rd` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `fub_axi` valids, the child's busy, and `fub_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `m_axi` valids.

5.4.2 Clock Gating Operation

The clock gating controller monitors activity on both user and AXI sides:

User-side activity detection:

```
user_valid = (ENABLE_WAKEUP && fub_axi_awakeup) || fub_axi_arvalid ||
fub_axi_rvalid || int_busy  // peer VALID (post-fix)
```

AXI-side activity detection:

```
axi_valid = m_axi_arvalid || m_axi_rvalid
```

Clock gating state machine: 1. **ACTIVE:** Clock running, transactions in progress 2. **IDLE:** No activity detected, idle counter incrementing 3. **GATING:** Idle count reached, clock gated (power saving) 4. **WAKE:** Activity detected, clock ungating

Ready signal control during gating:

```
fub_axi_aready = cg_gating ? 1'b0 : int_aready;
m_axi_rready = cg_gating ? 1'b0 : int_rready;
```

5.4.3 Ready Deassertion and Wake Behavior

While `cg_gating` is asserted, the ready outputs listed above are forced low.

This is legal AXI. A READY output may be deasserted for any number of cycles, and neither side may assume READY asserts within a bounded time, so holding it low is ordinary backpressure rather than a protocol violation. It cannot deadlock either: the same VALID being held off also feeds the activity detector, so gating releases and READY returns to the core's value.

Two consequences are worth designing around:

- **Wake latency adds to first-transfer latency.** Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. The AXI5 `_cg` wrappers drive the activity terms combinationally, so there is 1 register stage and the first usable gated-clock edge arrives 2 cycles after activity asserts. Size `cfg_cg_idle_count` so that cost is amortized over the traffic burst that follows.
- **READY is a combinational function of `cg_gating`.** The path from the idle counter to the `fub_axi_aready` output may need attention during timing closure, particularly at high frequency.

5.4.4 Integration Caveat: `fub_axi_rready` in the Activity Term

The activity detector uses the corresponding VALID (`fub_axi_rvalid`), never the peer's READY. (Historically this term was `fub_axi_rready`, which meant a peer parking its ready high – a common and entirely correct style – pinned the block permanently awake and defeated gating; that defect was fixed family-wide, matching the rule the mon_cg wrappers always carried.)

An always-ready consumer no longer prevents gating.

5.4.5 Idle Counter Configuration

The `cfg_cg_idle_count` parameter sets how many consecutive idle cycles are required before clock gating activates. Gating engages `cfg_cg_idle_count + 1` clocks after the internal wakeup

deasserts, which is `cfg_cg_idle_count + 2` clocks after the last bus activity on AXI5 (one extra for the `r_wakeup` flop). The “Idle Cycles” column below counts from the internal wakeup deassertion:

cfg_cg_idle_count	Idle Cycles	Use Case
0	1 cycle	Aggressive power saving
1-3	2-4 cycles	Balanced (typical)
4-7	5-8 cycles	Low-latency systems
8-15	9-16 cycles	Minimize gating overhead

Recommendation: Start with `cfg_cg_idle_count = 3` (4 idle cycles) for most applications.

5.4.6 Power Savings Analysis

Clock gating effectiveness: - Dynamic power of the gated logic falls roughly in proportion to the gated fraction of time - Order-of-magnitude expectation: tens of percent of this module’s dynamic power under bursty traffic; most of it in a mostly-idle system

Trade-offs: - Wake latency: 1 register stage; first usable gated-clock edge 2 clock cycles after activity - Area increase: a few percent (clock gate cell plus idle counter) - Best suited for: Intermittent traffic patterns

All power figures on this page are first-order estimates derived from duty cycle, not measured results. No power analysis (gate-level or otherwise) has been run against these modules. Actual savings depend on the gated logic’s share of total design power, the technology library’s clock-gate cell, and leakage, which clock gating does not reduce at all. Treat these numbers as a sizing aid and characterize your own instance before quoting a savings figure.

5.5 Timing Characteristics

5.5.1 Clock Gating Activation

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- user_valid, axi_valid activity
- Idle counter incrementing
- cg_idle assertion
- cg_gating activation
- Clock stops after idle count

5.5.2 Clock Gating Wake-up

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK resuming
- Activity detection (arvalid assertion)
- cg_gating deactivation
- arready response after wake
- Normal AXI transaction proceeds

5.6 Usage Examples

```
axi5_master_rd_cg #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    // Enable AXI5 features
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1),
    // Clock gating configuration
    .CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_rd_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable      (power_save_enable),
    .cfg_cg_idle_count  (4'd3),  // Gate after 4 idle cycles

    // FUB interface (slave side)
    .fub_axi_arid       (fub_arid),
    .fub_axi_araddr     (fub_araddr),
    .fub_axi_arlen      (fub_arlen),
    .fub_axi_arsize     (fub_arsize),
    .fub_axi_arburst    (fub_arburst),
    .fub_axi_arlock     (fub_arlock),
    .fub_axi_arcache    (fub_arcache),
    .fub_axi_arprot     (fub_arprot),
    .fub_axi_arqos      (fub_arqos),
    .fub_axi_aruser     (fub_aruser),
    .fub_axi_arvalid    (fub_arvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_arready    (fub_arready),

    // AXI5 AR extensions
    .fub_axi_arnsaid    (fub_arnsaid),
    .fub_axi_artrace    (fub_artrace),
    .fub_axi_armpam     (fub_armpam),
    .fub_axi_armecid    (fub_armecid),
```

```
    .fub_axi_aridunq   (fub_aridunq),
    .fub_axi_archunken  (fub_archunken),
    .fub_axi_artagop    (fub_artagop),

    // R channel
    .fub_axi_rid        (fub_rid),
    .fub_axi_rdata      (fub_rdata),
    .fub_axi_rresp      (fub_rresp),
    .fub_axi_rlast      (fub_rlast),
    .fub_axi_ruser      (fub_ruser),
    .fub_axi_rvalid     (fub_rvalid),
    .fub_axi_rready     (fub_rready),

    // AXI5 R extensions
    .fub_axi_rtrace     (fub_rtrace),
    .fub_axi_ridunq     (fub_ridunq),
    .fub_axi_rpoison    (fub_rpoison),
    .fub_axi_rchunkv    (fub_rchunkv),
    .fub_axi_rchunknum  (fub_rchunknum),
    .fub_axi_rchunkstrb (fub_rchunkstrb),
    .fub_axi_rtag       (fub_rtag),

    // Master interface (output side)
    .m_axi_arid         (m_axi_arid),
    .m_axi_araddr       (m_axi_araddr),
    // Every remaining m_axi_* port mirrors the fub_axi_* list above,
    // same names and widths, opposite directions. All must be
connected.

    // Clock gating status
    .cg_gating          (clock_gated),
    .cg_idle            (interface_idle)
);

// Use status signals for power monitoring
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n)
        power_cycles_saved <= '0;
    else if (clock_gated)
        power_cycles_saved <= power_cycles_saved + 1;
end
```

5.7 Design Notes

5.7.1 Clock Gating vs. Always-On

Aspect	Always-On	Clock Gated
Power (idle)	100%	10-30%
Wake latency	0 cycles	1 register stage; first usable edge 2 cycles after activity
Area overhead	0%	~5%
Best for	Continuous traffic	Bursty traffic

5.7.2 Configuration Strategies

Aggressive Power Saving:

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd0)   // Gate immediately after idle
```

Balanced (Recommended):

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd3)   // Gate after 4 idle cycles
```

Low-Latency:

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd7)   // Gate after 8 idle cycles (count+1; a
LITERAL count)
```

Disabled (Always-On):

```
.cfg_cg_enable      (1'b0),
.cfg_cg_idle_count  (4'd0)   // Ignored when disabled
```

5.7.3 Integration with System Power Management

The clock gating status outputs can integrate with broader system power management:

```
// Aggregate clock gating status
assign system_power_state = {
    axi_rd_cg.cg_gating,
    axi_wr_cg.cg_gating,
    apb_cg.cg_gating
    // ... other clock-gated modules
};

// Enter deep sleep when all interfaces gated
assign deep_sleep_ok = &system_power_state;
```

5.7.4 Verification Considerations

When verifying clock-gated designs: 1. Test with `cfg_cg_enable = 0` first (baseline) 2. Enable clock gating and verify identical functional behavior 3. Check wake latency meets timing requirements 4. Verify no protocol violations during gating/ungating 5. Measure power savings in representative traffic scenarios

---

5.8 Related Modules

- **AXI5 Master Read** - Non-gated base module
- **AXI5 Master Write CG** - Write with clock gating
- **AXI5 Master Read Monitor CG** - With monitoring + clock gating
- **AMBA Clock Gate Controller** - Clock gating controller spec

5.9 Testing

`val/amba/test_axi5_master_rd_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_rd_cg.py -v
```

5.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A5.6.2 “Read data chunking” (ARCHUNKEN, RCHUNKV/NUM/STRB), §A16.1 “Data protection using Poison” (RPOISON), §A12.2 “Memory Tagging Extension (MTE)” (ARTAGOP, RTAG), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (ARMECID). The ports follow the specification’s names (GitHub #102): there is no RTAGMATCH, the Unique ID indicator is ARIDUNQ with RIDUNQ on every read data beat, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

5.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

6 AXI5 Master Read with Monitor

Module: `axi5_master_rd_mon.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

6.1 Overview

The AXI5 Master Read with Monitor module combines the standard `axi5_master_rd` core with an integrated `axi_monitor_filtered` for comprehensive transaction monitoring and error detection. You get real-time visibility into AXI5 read operations, with configurable packet filtering so the monitor bus only carries what you actually care about.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

6.1.1 Key Features

- Carries the full AXI5 signal set unmodified (wraps `axi5_master_rd`) – transport, not semantics; see Scope above
- **ARNSAID:** Non-secure access identifier for security domains
- **ARTRACE:** Trace signal for debug and performance monitoring
- **ARMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **ARMECID:** Memory Encryption Context ID for secure memory
- **ARIDUNQ / RIDUNQ:** Unique ID indicator on the request and on every read data beat (IHI 0022L §A5.2)
- **ARCHUNKEN:** Read data chunking enable for partial data transfers
- **ARTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **RTRACE:** Read data trace signal
- **RPOISON:** Data poison indicator for corrupted data detection
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking control signals
- **RTAG:** Memory tags returned with read data (MTE)
- **Integrated AXI monitor** with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)
- **Error detection:** Protocol violations, SLVERR, DECERR
- **Timeout monitoring:** Stuck transactions, stalled channels
- **Performance metrics:** Latency, throughput, outstanding transactions
- **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
- **Configuration validation:** Detects filter conflicts

6.2 Parameters

A lite-monitor version of this wrapper exists as `axi5_master_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	10	Monitor agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when cfg_threshold_enable=1. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. **>1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise

Parameter	Type	Default	Description
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
ACLK_MHZ	int	100	Clock frequency in MHz – keeps the 1 us tick exact off-100MHz
CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
`ENABLE_FILTERING`	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor (latency vs. timing)
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared

Parameter	Type	Default	Description
			allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the `axi_monitor_timeout` instance.
ENABLE_COMPL_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGIC	bit	1	Compile-in the REPORTER's legacy perf-packet cone and the two lifetime counters (error_count / transaction_count). The window FSM and the bucket/beat/byte/burst counters are UNCONDITIONAL module-level logic – always compiled, always live.
ENABLE_DEBUG_LOGIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six `ENABLE_*_LOGIC` parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero

area (classic cones default on, debug off). Inside the wrapper's `axi_monitor_filtered` instance the perf/debug master switches are fixed (`ENABLE_PERF_PACKETS = 1`, `ENABLE_DEBUG_MODULE = 0`). The former `CAM_PIPELINE` / `TRANS_CAM_PIPELINE` parameters were removed — the transaction CAM is now always pipelined.

6.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS
CHUNK_STRB_WIDTH	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1

6.3 Ports

6.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

6.3.2 FUB AXI5 Interface (Slave Side - Input)

Same as `axi5_master_rd` - see AXI5 Master Read for complete port listing.

6.3.3 Master AXI5 Interface (Output Side)

Same as `axi5_master_rd` - see AXI5 Master Read for complete port listing.

6.3.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Master runtime gate. 0 = monitor inert: no allocation, transaction CAM held clear, and the upstream ready is never stalled (a disabled monitor cannot block the datapath). 1 = normal operation
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packet generation
cfg_timeout_enable	1	Input	Enable timeout packet generation
cfg_perf_e	1	Input	Enable performance

Port	Width	Direction	Description
nable			packet generation (see Performance Monitoring)
cfg_compl_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Unified coarse timeout control, a MICROSECOND count passed through at FULL 16-bit width: 1..65535 us per phase; 0 = 16'hFFFF (~65 ms, effectively never). One value drives all three phase counts (addr/data/resp). The old 4-bit squash that saturated >= 16 at 15 us is retired
cfg_latency_threshold	32	Input	Latency threshold for performance alerts
cfg_freq_sel	4	Input	`counter_freq_invariant` LUT index scaling the 1 us timer tick

Detection-cone enables: `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4) and `cfg_debug_mask` (16) inputs are tied to `4'h0` / `16'h0`; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`). None are wrapper ports.

6.3.5 AXI Protocol Filtering Configuration

Port	Width	Direction	Description
cfg_axi_pkt_mask	16	Input	Packet type filter (Level 1)
cfg_axi_err_select	16	Input	Error routing configuration (Level 2)
cfg_axi_error_mask	16	Input	Error event filter (Level 3)
cfg_axi_timeout_mask	16	Input	Timeout event filter (Level 3)
cfg_axi_compl_mask	16	Input	Completion event filter (Level 3)
cfg_axi_thresh_mask	16	Input	Threshold event filter (Level 3)
cfg_axi_perf_mask	16	Input	Performance event filter (Level 3)
cfg_axi_addr_mask	16	Input	Address event filter (Level 3)
cfg_axi_debug_mask	16	Input	Debug event filter (Level 3)

6.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	`monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_block_ready	1	Output	Observability tap for the block_ready gating net (drives nothing internally;

Port	Width	Direction	Description
			leave unconnected if unused)
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

6.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding transactions
error_count	16	Output	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
transaction_count	32	Output	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
cfg_conflict_error	1	Output	Configuration conflict detected

6.3.8 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

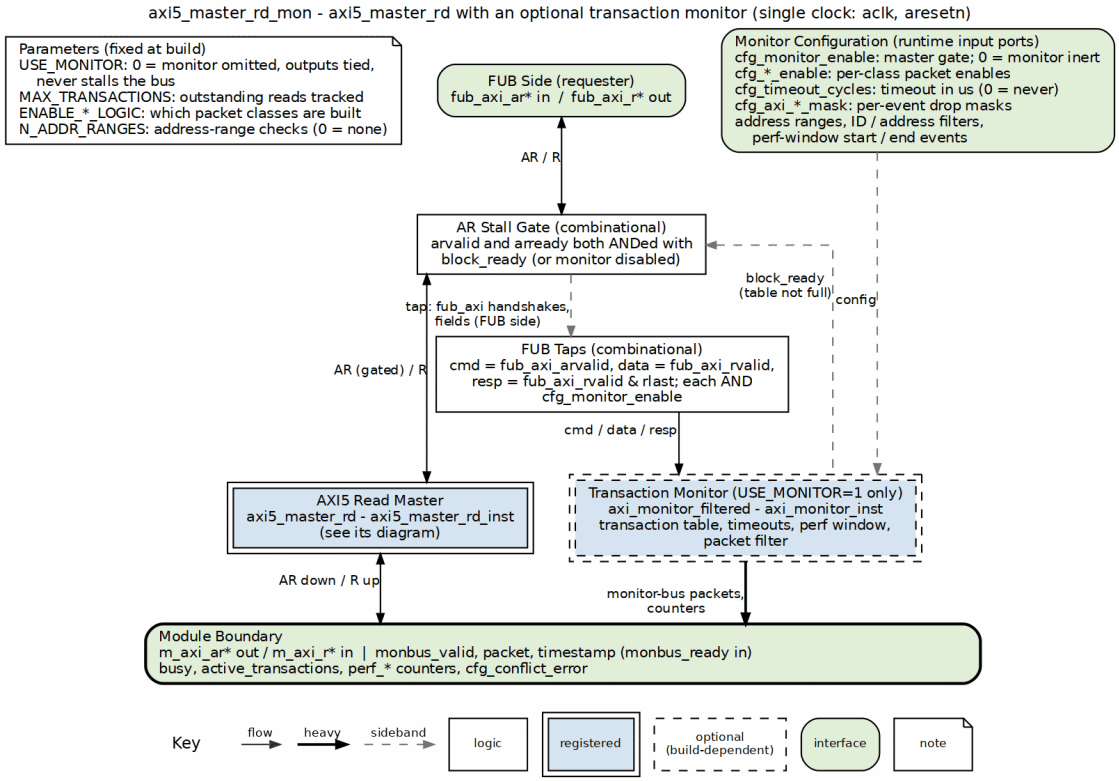
Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
`cfg_id_filter_enable`	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
`cfg_id_match_base`	`ID_WIDTH`	First ID owned
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

6.4 Functional Description

6.4.1 Architecture



axi5_master_rd_mon block diagram

The base `axi5_master_rd` is one box (see its own diagram). Traffic runs down the left through the AR stall gate, which ANDs `fub_axi_arvalid` and its ready with the `block_ready` the monitor raises while its transaction table has room. The taps watch the FUB-side (`fub_axi`) handshake – the same point the stall gate acts on, so a falling `block_ready` stops new events with no skid lag – and feed `axi_monitor_filtered`, which is built only when `USE_MONITOR=1`; its monitor-bus packets are the heavy edge out. With `cfg_monitor_enable` low the monitor is inert and never stalls the bus.

6.4.2 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axi_arready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axi_arready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, `ap_disabled_never_stalls`; this module has no `ifdef FORMAL` block of its own, so here the guarantee rests on the gate expression above, not a proof).
- **Tap point**: every monitor taps the FUB-side handshake, for timing. On this master that is also where block_ready gates, so new events stop the cycle block_ready goes low. On the slave variants block_ready gates the upstream bus handshake while the tap sits on the backend side, so there is a skid-depth lag (`SKID_DEPTH_AR`) that the `cmd_entry_reserve` margin covers.

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `ocelus_common_pkg`), and `block_ready` re-asserts at occupancy `< MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

6.4.3 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper does not expose ADDR_RANGE_IS_ERROR; all ranges default to the DEBUG (match) flavor.

Config inputs (active only when N_ADDR_RANGES > 0): - cfg_addr_check_enable — master on/off for the checker. - cfg_addr_range_enable[N-1:0] — per-range enable bit. - cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0] — inclusive bounds.

Event encoding: event_data[63:60] = range_index (matching DEBUG range, or 4'hF sentinel on an ERROR miss); event_data[59:0] = full address. **Filtering:** AddrMatch is dropped by cfg_axi_addr_mask[1], the ADDR_RANGE error by cfg_axi_error_mask[13]. See the checker page for coalescing + formal properties.

6.4.4 Monitor Bus Packet Format

The 128-bit monbus_packet (paired with the 64-bit monbus_timestamp side-band signal) follows the standardized AMBA monitor bus format:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, DECERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished)
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits – AXI ID or channel index)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits – full address, latency, etc.)
```

6.4.5 Filtering Hierarchy (two active levels, one reserved)

Level 1: Packet Type Mask (cfg_axi_pkt_mask)

```
// SET BIT = DROP that packet type (pkt_drop =
cfg_axi_pkt_mask[pkt_type]),
// and bits index the NUMERIC packet type from the format table:
cfg_axi_pkt_mask[0]  = 1 → DROP ERROR packets   (type 0x0)
cfg_axi_pkt_mask[1]  = 1 → DROP COMPL packets   (type 0x1)
cfg_axi_pkt_mask[2]  = 1 → DROP THRESH packets  (type 0x2)
cfg_axi_pkt_mask[3]  = 1 → DROP TIMEOUT packets (type 0x3)
cfg_axi_pkt_mask[4]  = 1 → DROP PERF packets    (type 0x4)
cfg_axi_pkt_mask[8]  = 1 → DROP ADDR_MATCH packets (type 0x8)
cfg_axi_pkt_mask[15] = 1 → DROP DEBUG packets   (type 0xF)
```

```
// e.g. 16'hFFF4 passes only ERROR|COMPL|TIMEOUT -- matching this
page's examples
```

Level 2: cfg_axi_err_select (reserved – no routing)

Its ONLY use is the conflict check `cfg_conflict_error = \|(cfg_axi_pkt_mask & cfg_axi_err_select)`; no error-routing logic exists anywhere in the monitor chain. Setting it to "reroute errors into completions" changes nothing except possibly tripping cfg_conflict_error.

Level 3: Event Masks (cfg_axi_*_mask)

Fine-grained control over specific events within each packet type: - `cfg_axi_error_mask`: SLVERR, DECERR, orphan detection, etc. - `cfg_axi_timeout_mask`: AR timeout (EVT_CMD_TIMEOUT), R timeout (EVT_DATA_TIMEOUT). The response-timeout bit exists in the mask but cannot fire on a read: EVT_RESP_TIMEOUT needs `data_completed && ! resp_received`, and on a read `data_last` sets `data_completed` and `TRANS_COMPLETE` in the same cycle (or `TRANS_ERROR` on a bad RRESP) – both states are excluded from the timeout block. - `cfg_axi_compl_mask`: Normal completion, error completion - `cfg_axi_thresh_mask`: Outstanding transaction count, latency threshold - `cfg_axi_perf_mask`: Average latency, peak bandwidth, utilization - `cfg_axi_addr_mask`: Address range tracking - `cfg_axi_debug_mask`: Internal state, debug events

6.4.6 Error Detection Events

The monitor detects and reports:

Protocol Errors: none on a read monitor.

`axi_monitor_trans_mgr` emits `EVT_PROTOCOL` (response-before-data ordering) and `EVT_RESP_ORPHAN` (response with no tracked transaction) only from inside its `if (!IS_READ && resp_valid && resp_ready)` block. A read monitor elaborates with `IS_READ = 1`, so neither event can be produced here – both describe the write B channel, which this module does not observe.

(Handshake-stability, ID-width, burst-length and address-alignment checks are NOT implemented either – nothing in the monitor chain inspects those.)

Response Errors: - SLVERR (slave error response) - DECERR (decode error response) - Orphaned read data (no matching AR) – an R beat whose ID matches no live entry

There is no RID-versus-ARID comparison. `axi_monitor_trans_mgr` attributes an R beat to the OLDEST live entry sharing that ID, so a slave returning the RID of a *different* outstanding read is silently mis-attributed with no error. An RID matching nothing is the orphan case above.

Timeout Errors – per phase, measured as DURATION not stall: - AR phase outstanding too long – `EVT_CMD_TIMEOUT` - R phase outstanding too long – `EVT_DATA_TIMEOUT`

(`EVT_RESP_TIMEOUT` is write-only; see Protocol Errors above.)

axi_monitor_timeout runs three per-phase timers (addr, data, resp) and there is no whole-transaction timer. Each timer is zeroed only while its phase is **not pending** (`if (!w_data_pending[idx]) r_data_timer[idx] <= '0;`) – it is NOT reset by a beat handshake. So these are phase-duration limits, not stall detectors: a long multi-beat read burst that is making steady progress still trips EVT_DATA_TIMEOUT once the R phase as a whole exceeds cfg_timeout_cycles. Size the threshold for your longest legitimate burst, not for your worst tolerable inter-beat gap.

Threshold Violations: - Outstanding transaction count > threshold - Transaction latency > cfg_latency_threshold

6.4.7 Configuration Conflict Detection

The cfg_conflict_error output flags invalid configurations:

```
// A packet type dropped by cfg_axi_pkt_mask must not also be selected
// for error routing by cfg_axi_err_select:
assign cfg_conflict_error = |(cfg_axi_pkt_mask & cfg_axi_err_select);
```

Recommended: enable only ONE high-traffic packet type (completions or perf) at a time; the monitor bus sustains at most one packet per two cycles.

6.4.8 Performance Monitoring

When performance tracking is compiled in (ENABLE_PERF_LOGIC = 1, the wrapper default, with ENABLE_PERF_PACKETS fixed to 1 inside the monitor instance) and cfg_perf_enable is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (window_active = 1) and hold their values between windows so the host can read a completed window's totals.

6.4.8.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by cfg_start_event_sel / cfg_end_event_sel (3-bit; e.g. 3'b010 selects the cfg_perf_enable edge) and can also be fired directly by the cfg_start_trigger / cfg_end_trigger pulses (from an engine or CSR write). cfg_window_force_close is a software override that closes the window immediately. While the window is open:

- window_active is high.
- window_cycles free-runs, counting every clock elapsed inside the window.

6.4.8.2 Utilization buckets (R data channel)

Every cycle inside the window is classified by the **R** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
perf_prod_cycles	32	rvalid && rready	productive beat transferred
perf_bp_cycles	32	rvalid && !rready	back-pressure (data offered, sink not ready)
perf_starv_cycles	32	!rvalid && rready	starvation (sink ready, no data)
perf_idle_cycles	32	!rvalid && !rready	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); the one-count skew is negligible for long windows.

6.4.8.3 Throughput counters

Output	Width	Meaning
perf_beat_count	32	R data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats x (1 << latched ARSIZE), using the ARSIZE captured at the most recent AR address phase
perf_burst_count	32	AR address-phase handshakes

Average burst length is `perf_beat_count / perf_burst_count`.

6.4.8.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window **start** event source select
cfg_end_ev	3	Input	Window **end** event source

Port	Width	Direction	Description
ent_sel			select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	`rvalid && rready` cycles
perf_bp_cycles	32	Output	`rvalid && !rready` cycles (back-pressure)
perf_starv_cycles	32	Output	`!rvalid && rready` cycles (starvation)
perf_idle_cycles	32	Output	`!rvalid && !rready` cycles
perf_beat_count	32	Output	R data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AR address-phase handshakes

When `USE_MONITOR = 0` all perfmon outputs are tied to 0. With `USE_MONITOR = 1` and `ENABLE_PERF_LOGIC = 0`, only error_count / transaction_count read 0 – window_active, window_cycles and all bucket/throughput outputs remain LIVE.

6.5 Timing Characteristics

6.5.1 Monitored Read Transaction with Error

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- AR channel: ARID, ARADDR, ARVALID, ARREADY
- R channel: RID, RDATA, RRESP (SLVERR), RVALID, RREADY
- Monitor bus: monbus_valid, monbus_packet showing ERROR packet
- Event sequence: AR → R error → ERROR packet generated

6.5.2 Timeout Detection

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- AR channel: ARVALID asserted, ARREADY stuck low
- Timeout counter incrementing
- cfg_timeout_cycles threshold
- Monitor bus: monbus_valid, monbus_packet showing TIMEOUT packet

6.5.3 Performance Monitoring

Timing diagram pending. The signals and sequence this scenario exercises:

- Multiple read transactions
- Latency measurement (AR to RLAST)
- Monitor bus: PERF packets with latency data
- Threshold comparison with cfg_latency_threshold

6.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_master_rd_mon #(
    .SKID_DEPTH_AR         (2),
    .SKID_DEPTH_R          (4),
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (32),
    .AXI_USER_WIDTH        (1),
    .AXI_NSAID_WIDTH       (4),
    .AXI_MPAM_WIDTH        (11),
    .AXI_MECID_WIDTH       (16),
    .AXI_TAG_WIDTH         (4)
) u_axi5_master_rd_mon (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cam_clear             (cam_clear),
    .fub_axi_arid          (fub_axi_arid),
    .fub_axi_araddr        (fub_axi_araddr),
    .fub_axi_arlen         (fub_axi_arlen),
    .fub_axi_arsize        (fub_axi_arsize),
    .fub_axi_arburst       (fub_axi_arburst),
    .fub_axi_arlock        (fub_axi_arlock),
    .fub_axi_arcache       (fub_axi_arcache),
    .fub_axi_arprot        (fub_axi_arprot),
    .fub_axi_arqos         (fub_axi_arqos),
    .fub_axi_aruser        (fub_axi_aruser),
    .fub_axi_arvalid       (fub_axi_arvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_arready       (fub_axi_arready),
    .fub_axi_arnsaid       (fub_axi_arnsaid),
    .fub_axi_artrace       (fub_axi_artrace),
    .fub_axi_armpam        (fub_axi_armpam),
    .fub_axi_armecid       (fub_axi_armecid),
    .fub_axi_aridunq      (fub_axi_aridunq),
    .fub_axi_archunken     (fub_axi_archunken),
    .fub_axi_artagop       (fub_axi_artagop),
    .fub_axi_rid           (fub_axi_rid),
    .fub_axi_rdata         (fub_axi_rdata),
    .fub_axi_rresp         (fub_axi_rresp),
    .fub_axi_rlast         (fub_axi_rlast),
    .fub_axi_ruser         (fub_axi_ruser),
    .fub_axi_rvalid        (fub_axi_rvalid),
    .fub_axi_rready        (fub_axi_rready),
    .fub_axi_rtrace        (fub_axi_rtrace),
    .fub_axi_ridunq        (fub_axi_ridunq),
```

```
    .fub_axi_rpoison       (fub_axi_rpoison),
    .fub_axi_rchunkv       (fub_axi_rchunkv),
    .fub_axi_rchunknum     (fub_axi_rchunknum),
    .fub_axi_rchunkstrb    (fub_axi_rchunkstrb),
    .fub_axi_rtag          (fub_axi_rtag),
    .m_axi_arid            (m_axi_arid),
    .m_axi_araddr          (m_axi_araddr),
    .m_axi_arlen           (m_axi_arlen),
    .m_axi_arsize          (m_axi_arsize),
    .m_axi_arburst         (m_axi_arburst),
    .m_axi_arlock          (m_axi_arlock),
    .m_axi_arcache         (m_axi_arcache),
    .m_axi_arprot          (m_axi_arprot),
    .m_axi_arqos           (m_axi_arqos),
    .m_axi_aruser          (m_axi_aruser),
    .m_axi_arvalid         (m_axi_arvalid),
    .m_axi_awakeup         (m_axi_awakeup),
    .m_axi_arready         (m_axi_arready),
    .m_axi_arnsaid         (m_axi_arnsaid),
    .m_axi_artrace         (m_axi_artrace),
    .m_axi_armpam          (m_axi_armpam),
    .m_axi_armecid         (m_axi_armecid),
    .m_axi_aridunq        (m_axi_aridunq),
    .m_axi_archunken       (m_axi_archunken),
    .m_axi_artagop         (m_axi_artagop),
    .m_axi_rid             (m_axi_rid),
    .m_axi_rdata           (m_axi_rdata),
    .m_axi_rresp           (m_axi_rresp),
    .m_axi_rlast           (m_axi_rlast),
    .m_axi_ruser           (m_axi_ruser),
    .m_axi_rvalid          (m_axi_rvalid),
    .m_axi_rready          (m_axi_rready),
    .m_axi_rtrace          (m_axi_rtrace),
    .m_axi_ridunq          (m_axi_ridunq),
    .m_axi_rpoison         (m_axi_rpoison),
    .m_axi_rchunkv         (m_axi_rchunkv),
    .m_axi_rchunknum       (m_axi_rchunknum),
    .m_axi_rchunkstrb      (m_axi_rchunkstrb),
    .m_axi_rtag            (m_axi_rtag),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
```

```
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .debug_block_ready     (debug_block_ready),
    .cfg_conflict_error    (cfg_conflict_error),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

6.7 Design Notes

6.7.1 Configuration Best Practices

CRITICAL: Never enable COMPL + PERF simultaneously!

High packet volume can overwhelm monitor bus and cause backpressure.

Recommended Configurations:

Use Case	Enabled Packet Types	Notes
Functional debug	ERROR + COMPL + TIMEOUT	Catches bugs, monitors correctness
Performance tuning	ERROR + PERF	Latency, throughput metrics
Protocol compliance	ERROR + TIMEOUT	Minimal overhead
Deep debug	ERROR + DEBUG	Internal state visibility

6.7.2 Transaction Table Sizing

`MAX_TRANSACTIONS` must accommodate maximum outstanding reads:

AXI Configuration	Recommended MAX_TRANSACTIONS
Single outstanding	4
Moderate pipelining	16
Deep pipelining	32-64
High-performance	128+

6.7.3 Filtering Strategy

Start broad, then narrow: 1. Enable all packet types initially 2. Identify high-traffic events 3. Mask low-value events with Level 3 filters 4. Separate configurations for different test phases

6.7.4 Monitor Bus Backpressure

The monitor respects `monbus_ready` backpressure: - Packets buffered internally when `monbus_ready = 0` - A full internal FIFO RETRIES: the event stays unmarked in the transaction table and re-presents every cycle until space frees – nothing is dropped (upstream throttling via block_ready is the relief valve, exactly as the backpressure section describes) - **Always add downstream FIFO** for latency absorption

---

6.8 Related Modules

- **AXI5 Master Read** - Base module without monitoring
- **AXI5 Master Read CG** - With clock gating only
- **AXI5 Master Read Monitor CG** - Monitor + clock gating
- **AXI Monitor Filtered** - Monitor core specification
- **Monitor Package Spec** - Packet format details
- **AXI Monitor Configuration Guide** - Complete configuration reference

6.9 Testing

`val/amba/test_axi5_master_rd_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_rd_mon.py -v
```

6.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- The monitor’s own techniques (transaction table, timeouts, packet filtering) are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects” (“AXI bus protocol overview”) – textbook overview of AXI responses and ID ordering (secondary source).
- Indexed in `rtl/amba/references/README.md` (protocol specifications are citation only)

6.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

7 AXI5 Master Read with Monitor and Clock Gating

Module: `axi5_master_rd_mon_cg.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

7.1 Overview

The AXI5 Master Read with Monitor and Clock Gating module combines `axi5_master_rd_mon` (AXI5 master with integrated monitoring) with intelligent clock gating for power optimization. This is the everything variant: comprehensive transaction monitoring, error detection, and automatic power management in one instantiation.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

7.1.1 Key Features

- Carries the full AXI5 signal set unmodified – transport, not semantics; see Scope above
- **All AXI5 extensions:** NSAID, TRACE, MPAM, MECID, IDUNQ, CHUNKING, MTE, POISON, WAKEUP
- **Integrated AXI monitor** with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)
- **Error detection:** SLVERR, DECERR, orphaned read data (protocol-violation events are write-monitor only)
- **Timeout monitoring:** Stuck transactions, stalled channels
- **Performance metrics:** Latency, throughput, outstanding transactions
- **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
- **Automatic clock gating** based on activity detection
- **Configurable idle threshold** before clock gating activates
- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Dual status outputs:** Monitor status + clock gating status
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

7.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axi5_master_rd_monlite_cg`: the same gating logic around `axi5_master_rd_monlite` instead of `axi5_master_rd_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID

Parameter	Type	Default	Description
			indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	10	Monitor agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection

Parameter	Type	Default	Description
			(forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
ACLK_MHZ	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. The `USE_WDATA_ORDER_Q` pairing rule applies to WRITE monitors only – `axi_monitor_trans_mgr` guards on `(NUM_BANKS > 1) && !IS_READ && ! USE_WDATA_ORDER_Q`, so a read monitor may bank freely.
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves `cfg_addr_filter_enable` low filters nothing and looks broken.
ID_FILTER_ENABLE	bit	0	Synthesises the ID report filter (see `cfg_id_*` for the runtime override).
ID_MATCH_BASE	int	0	First ID this instance owns.
ID_MATCH_COUNT	int	0	How many IDs; `0` means

Parameter	Type	Default	Description
			ALL, so a zeroed register block does not silently filter everything away.
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

7.2.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the ID_MATCH_* parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

7.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression

AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS
CHUNK_STRB_WIDTH	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

7.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

7.3.3 FUB AXI5 Interface (Slave Side - Input)

Same as `axi5_master_rd_mon` - see AXI5 Master Read Monitor.

7.3.4 Master AXI5 Interface (Output Side)

Same as `axi5_master_rd_mon` - see AXI5 Master Read Monitor.

7.3.5 Monitor Configuration

Same as `axi5_master_rd_mon` - see AXI5 Master Read Monitor.

7.3.6 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	`monitor_packet_t` (128-bit format)
monbus_ti	64	Output	`monbus_timestamp_t`

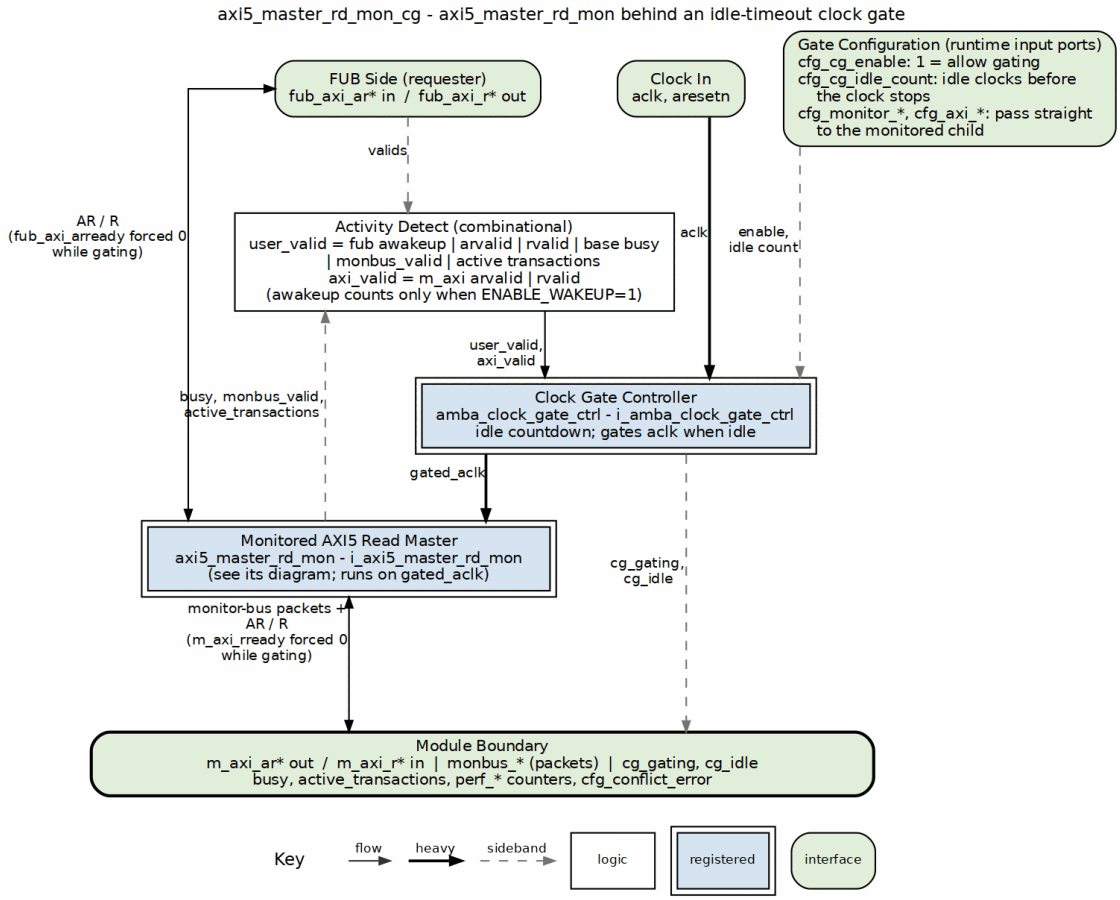
Port	Width	Direction	Description
mestamp			paired atomically with `monbus_packet`
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

7.3.7 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding transactions
error_count	16	Output	Cumulative error count (live – driven from the reporter's lifetime counter)
transaction_count	32	Output	Total transaction count (live – reporter lifetime counter, zero-extended)
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock gating active (1=gated, 0=running)
cg_idle	1	Output	Interface idle (1=no activity)

7.4 Functional Description

7.4.1 Architecture



axi5_master_rd_mon_cg block diagram

The base `axi5_master_rd_mon` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `fub_axi` valids, the child's busy, and `fub_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `m_axi` valids. The monitored child's `monbus_valid` and active-transaction count also count as activity, so the clock never stops under a pending packet.

7.4.2 Combined Monitoring and Power Management

This module provides the best of both worlds:

Monitoring (from axi5_master_rd_mon): - Real-time transaction monitoring - Error and timeout detection - Performance metrics collection - Configurable packet filtering - MonBus output for system integration

Power Management (from axi5_master_rd_cg): - Automatic clock gating during idle periods - Configurable idle threshold - Activity-based wake-up - Status outputs for power monitoring

7.4.3 Clock Gating with Monitor Active

The clock gating logic considers monitor activity:

```
user_valid = (ENABLE_WAKEUP && fub_axi_awakeup) || fub_axi_arvalid ||
fub_axi_rvalid || int_busy ||
             w_monbus_valid || (|active_transactions);  // peer VALID,
never peer READY
axi_valid = m_axi_arvalid || m_axi_rvalid;

// Clock remains active if:
// - User interface active (AR or R channels)
// - AXI interface active
// - CORE busy (transactions in flight in the datapath)
// - A monitor packet is pending on the monitor bus (w_monbus_valid)
// - The monitor CAM still holds entries (|active_transactions) --
covers
//   the reporter's retire -> FIFO -> output emission window
```

Monitor-bus delivery is exactly-once across gating, at any idle count. A packet pending on the monitor bus is a wake term: the block stays awake (or re-wakes within a cycle) until the consumer accepts it, and the external `monbus_valid` is masked with `!cg_gating` so a consumer can never sample a valid that the reporter's stopped clock could not retire. The monitor CAM's occupancy (`active_transactions`) is a wake term too: a tracked entry stays in the CAM until its packet is marked into the reporter FIFO, and the registered count lags one cycle past that, meeting `monbus_valid`'s assertion – so the clock cannot stop inside the reporter's few-cycle emission window and strand a trailing packet, even at `cfg_cg_idle_count = 0`. `val/amba/test_mon_cg_gating.py` phase 6 asserts exactly-once delivery of exactly the packets generated.

7.4.4 Ready Signal Control During Gating

When clock gating is active, ready signals are forced low to prevent new transactions:

```
fub_axi_arready = cg_gating ? 1'b0 : int_arready;
m_axi_rready = cg_gating ? 1'b0 : int_rready;
```

This ensures protocol compliance during power management.

7.4.5 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_master_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module —

see Performance Monitoring in axi5_master_rd_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. **WARNING:** an open measurement window is NOT a wake term – the window FSM and all counters run on the gated clock, so if the bus idles past `cfg_cg_idle_count` mid-window, `window_cycles` and the four buckets FREEZE while wall-clock time passes. For exact wall-clock windows hold `cfg_cg_enable` low around the measurement, or use the base module.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see Parameters), straight through to the base monitor.

7.5 Timing Characteristics

7.5.1 Clock Gating with Monitor Activity

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- AXI transaction sequence
- Monitor packet generation
- Idle period detection
- cg_gating activation
- Monitor quiescent before clock stops

7.5.2 Wake-up with Immediate Monitoring

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK resuming
- ARVALID assertion (wake trigger)
- cg_gating deactivation
- AXI transaction proceeds
- Monitor packets generated immediately

7.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_master_rd_mon_cg #(
    .SKID_DEPTH_AR         (2),
    .SKID_DEPTH_R          (4),
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (32),
    .AXI_USER_WIDTH        (1),
    .AXI_NSAID_WIDTH       (4),
    .AXI_MPAM_WIDTH        (11),
    .AXI_MECID_WIDTH       (16),
    .AXI_TAG_WIDTH         (4)
) u_axi5_master_rd_mon_cg (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cam_clear             (cam_clear),
    .cfg_cg_enable         (cfg_cg_enable),
    .cfg_cg_idle_count     (cfg_cg_idle_count),
    .fub_axi_arid          (fub_axi_arid),
    .fub_axi_araddr        (fub_axi_araddr),
    .fub_axi_arlen         (fub_axi_arlen),
    .fub_axi_arsize        (fub_axi_arsize),
    .fub_axi_arburst       (fub_axi_arburst),
    .fub_axi_arlock        (fub_axi_arlock),
    .fub_axi_arcache       (fub_axi_arcache),
    .fub_axi_arprot        (fub_axi_arprot),
    .fub_axi_arqos         (fub_axi_arqos),
    .fub_axi_aruser        (fub_axi_aruser),
    .fub_axi_arvalid       (fub_axi_arvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_arready       (fub_axi_arready),
    .fub_axi_arnsaid       (fub_axi_arnsaid),
    .fub_axi_artrace       (fub_axi_artrace),
    .fub_axi_armpam        (fub_axi_armpam),
    .fub_axi_armecid       (fub_axi_armecid),
    .fub_axi_aridunq      (fub_axi_aridunq),
    .fub_axi_archunken     (fub_axi_archunken),
    .fub_axi_artagop       (fub_axi_artagop),
    .fub_axi_rid           (fub_axi_rid),
    .fub_axi_rdata         (fub_axi_rdata),
    .fub_axi_rresp         (fub_axi_rresp),
    .fub_axi_rlast         (fub_axi_rlast),
    .fub_axi_ruser         (fub_axi_ruser),
    .fub_axi_rvalid        (fub_axi_rvalid),
    .fub_axi_rready        (fub_axi_rready),
```

```
    .fub_axi_rtrace        (fub_axi_rtrace),
    .fub_axi_ridunq        (fub_axi_ridunq),
    .fub_axi_rpoison       (fub_axi_rpoison),
    .fub_axi_rchunkv       (fub_axi_rchunkv),
    .fub_axi_rchunknum     (fub_axi_rchunknum),
    .fub_axi_rchunkstrb    (fub_axi_rchunkstrb),
    .fub_axi_rtag          (fub_axi_rtag),
    .m_axi_arid            (m_axi_arid),
    .m_axi_araddr          (m_axi_araddr),
    .m_axi_arlen           (m_axi_arlen),
    .m_axi_arsize          (m_axi_arsize),
    .m_axi_arburst         (m_axi_arburst),
    .m_axi_arlock          (m_axi_arlock),
    .m_axi_arcache         (m_axi_arcache),
    .m_axi_arprot          (m_axi_arprot),
    .m_axi_arqos           (m_axi_arqos),
    .m_axi_aruser          (m_axi_aruser),
    .m_axi_arvalid         (m_axi_arvalid),
    .m_axi_awakeup         (m_axi_awakeup),
    .m_axi_arready         (m_axi_arready),
    .m_axi_arnsaid         (m_axi_arnsaid),
    .m_axi_artrace         (m_axi_artrace),
    .m_axi_armpam          (m_axi_armpam),
    .m_axi_armecid         (m_axi_armecid),
    .m_axi_aridunq        (m_axi_aridunq),
    .m_axi_archunken       (m_axi_archunken),
    .m_axi_artagop         (m_axi_artagop),
    .m_axi_rid             (m_axi_rid),
    .m_axi_rdata           (m_axi_rdata),
    .m_axi_rresp           (m_axi_rresp),
    .m_axi_rlast           (m_axi_rlast),
    .m_axi_ruser           (m_axi_ruser),
    .m_axi_rvalid          (m_axi_rvalid),
    .m_axi_rready          (m_axi_rready),
    .m_axi_rtrace          (m_axi_rtrace),
    .m_axi_ridunq          (m_axi_ridunq),
    .m_axi_rpoison         (m_axi_rpoison),
    .m_axi_rchunkv         (m_axi_rchunkv),
    .m_axi_rchunknum       (m_axi_rchunknum),
    .m_axi_rchunkstrb      (m_axi_rchunkstrb),
    .m_axi_rtag            (m_axi_rtag),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
```

```
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .cfg_conflict_error    (cfg_conflict_error),
    .cg_gating             (cg_gating),
    .cg_idle               (cg_idle),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
```

```
    .perf_burst_count      (perf_burst_count)
);
```

7.7 Design Notes

7.7.1 When to Use This Module

Ideal for: - Battery-powered or power-sensitive systems - Bursty traffic patterns with significant idle periods - Systems requiring comprehensive debug visibility - Production systems needing runtime monitoring + power optimization

Consider alternatives if: - Continuous, high-throughput traffic (clock gating ineffective) - Ultra-low latency requirements (avoid gating overhead) - Area-constrained designs (monitor + CG adds ~10% area)

7.7.2 Configuration Strategy Matrix

Scenario	Monitor Config	Clock Gating Config	Expected Benefit
Development/Debug	ERROR + COMPL + TIMEOUT	Disabled	Full visibility, no wake latency
Performance Tuning	ERROR + PERF	Disabled	Metrics without power interference
Power Testing	ERROR only	Aggressive (count=0)	Maximum power savings
Production	ERROR + TIMEOUT	Balanced (count=3)	Error detection + power savings

7.7.3 Monitor + Clock Gating Interaction

Key Considerations:

1. **Monitor packets generated before gating:**
   - Monitor processes all events before idle state
   - MonBus packets FREEZE when the clock stops (see the WARNING above – nothing flushes them)
   - No events lost during gating transition
2. **Wake latency includes monitor initialization:**
   - 1 wake-up register stage; first usable gated-clock edge 2 cycles after activity
   - Monitor ready immediately (no reset needed)
   - First transaction monitored correctly
3. **Power savings account for monitor overhead:**

- Monitor adds ~10% dynamic power when active
- Clock gating saves ~80-90% when idle (first-order estimate – no power analysis has been run; same disclaimer as the sibling _cg page)
- Net savings: 70-80% during idle periods

7.7.4 Verification Recommendations

1. **Test monitor + CG independently first:**

```
// Phase 1: Monitor only (CG disabled)
.cfg_cg_enable (1'b0)
// Verify all monitor features

// Phase 2: CG only (monitor minimal)
.cfg_monitor_enable (1'b0)
.cfg_error_enable   (1'b0)
.cfg_cg_enable      (1'b1)
// Verify clock gating behavior

// Phase 3: Combined
// Both enabled, verify interaction
```

2. **Check monitor packet integrity across gating:**
   - No packets lost during gating transitions
   - No duplicate packets
   - Timestamps/latencies accurate
3. **Measure actual power savings:**
   - Instrument with power counters
   - Compare vs. always-on baseline
   - Verify efficiency in target traffic patterns

7.8 Related Modules

- **AXI5 Master Read** - Base module
- **AXI5 Master Read CG** - Clock gating only
- **AXI5 Master Read Monitor** - Monitor only
- **AXI5 Master Write Monitor CG** - Write variant
- **AXI Monitor Configuration Guide** - Monitor setup
- **AMBA Clock Gate Controller** - Clock gating details

7.9 Testing

`val/amba/test_axi5_master_rd_mon_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_rd_mon_cg.py -v
```

7.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- The monitor’s own techniques are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

7.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

8 AXI5 Master Write

Module: `axi5_master_wr.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

8.1 Overview

The AXI5 Master Write module is the master-side AW/W/B channel transport block. It carries a full AXI5 write signal set — `AWATOP` and the other AXI5 sideband extensions included — between a FUB (Functional Unit Block) interface and an external AXI5 master interface, with a configurable SKID buffer on each of the AW, W, and B channels.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. `AWATOP` is carried through unmodified but no atomic read-modify-write is performed, no MTE tag checking or `BTAGMATCH` generation is performed, and no outstanding-transaction tracking is done. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

8.1.1 Key Features

- Carries the full AXI5 signal set listed below, unmodified, across the SKID buffers
- **AWATOP:** Atomic transaction operation type (Compare/Swap, Fetch/Add, etc.)
- **AWNSAID:** Non-secure access identifier for security domains
- **AWTRACE:** Trace signal for debug and performance monitoring
- **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **AWMECID:** Memory Encryption Context ID for secure memory
- **AWIDUNQ / BIDUNQ:** Unique ID indicator on the request and its write response (IHI 0022L §A5.2)
- **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **WPOISON:** Write data poison indicator for error injection
- **WTAG:** Write data memory tags (MTE)
- **WTAGUPDATE:** Tag update mask for selective tag writing
- **BTRACE:** Response trace signal
- **BTAGMATCH[1:0]:** Tag match result, Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)
- Configurable SKID buffer depths for AW, W, and B channels
- Busy signal for power management and clock gating
- AWREGION not implemented (see *Design Notes*)

8.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_W	int	4	W channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_B	int	2	B channel SKID buffer depth (legal range 2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory

Parameter	Type	Default	Description
			partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low

8.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
SW	AXI_WSTRB_WIDTH	Write strobe width, one bit per data byte
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `wtag` field
AWSize	Sum of the enabled AW fields	AW SKID buffer payload width
WSize	Sum of the enabled W fields	W SKID buffer payload width
BSize	Sum of the enabled B fields	B SKID buffer payload width

8.3 Ports

8.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

8.3.2 FUB AXI5 Interface (Slave Side - Input)

8.3.2.1 AW Channel

Port	Width	Direction	Description
fub_axi_awid	IW	Input	Write address ID
fub_axi_awaddr	AW	Input	Write address
fub_axi_awlen	8	Input	Burst length
fub_axi_awsize	3	Input	Burst size
fub_axi_awburst	2	Input	Burst type
fub_axi_awlock	1	Input	Lock type
fub_axi_awcache	4	Input	Cache attributes
fub_axi_awprot	3	Input	Protection attributes
fub_axi_awqos	4	Input	Quality of Service
fub_axi_awuser	UW	Input	User-defined signal
fub_axi_awvalid	1	Input	Write address valid
fub_axi_aw	1	Input	Wake-up request,

Port	Width	Direction	Description
akeup			AWAKEUP (IHI 0022L §A14.1.1; ENABLE_WAKEUP)
fub_axi_awready	1	Output	Write address ready

8.3.2.2 AXI5 AW Extensions

Port	Width	Direction	Description
fub_axi_awatop	AXI_ATOP_WIDTH	Input	Atomic operation type
fub_axi_awnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_awtrace	1	Input	Trace signal
fub_axi_awmpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_awmecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_awidunq	1	Input	Unique ID indicator, AWIDUNQ (IHI 0022L §A5.2; ENABLE_UNIQUE)
fub_axi_awtagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

8.3.2.3 W Channel

Port	Width	Direction	Description
fub_axi_wdata	DW	Input	Write data
fub_axi_wstrb	SW	Input	Write strobes
fub_axi_wlast	1	Input	Last transfer in burst
fub_axi_wuser	UW	Input	User-defined signal
fub_axi_wvalid	1	Input	Write data valid
fub_axi_wread	1	Output	Write data

Port	Width	Direction	Description
y			ready

8.3.2.4 AXI5 W Extensions

Port	Width	Direction	Description
fub_axi_wpoison	1	Input	Data poison indicator
fub_axi_wtag	TW	Input	Write data tags
fub_axi_wtagupdate	NUM_TAGS	Input	Tag update mask

8.3.2.5 B Channel

Port	Width	Direction	Description
fub_axi_bid	IW	Output	Response ID
fub_axi_bresp	2	Output	Write response
fub_axi_buser	UW	Output	User-defined signal
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

8.3.2.6 AXI5 B Extensions

Port	Width	Direction	Description
fub_axi_btrace	1	Output	Response trace
fub_axi_bidunq	1	Output	Unique ID indicator on the write response, BIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_btagmatch	2	Output	Tag match result, IHI 0022L Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)

8.3.3 Master AXI5 Interface (Output Side)

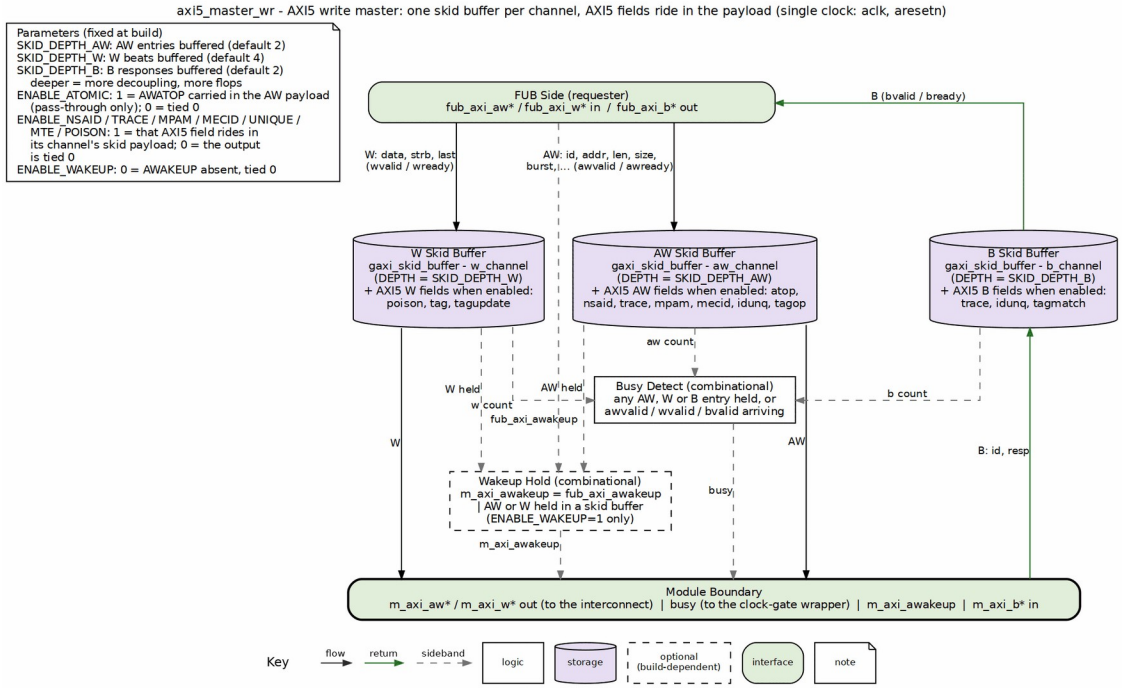
Same port list as FUB interface but with `m_axi_*` prefix and reversed directions.

8.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator for clock gating

8.4 Functional Description

8.4.1 Architecture



axi5_master_wr block diagram

The requester (FUB) side is at the top and the module boundary is the bar. W takes its own lane down the left, AW goes down the center, and B comes back up the right lane, each through its own skid buffer. The AXI5 fields each channel carries (NSAID, trace, MPAM, MECID, unique-ID, MTE tags, poison, and AWATOP for atomics) ride in that channel's skid payload when their `ENABLE_*` parameter is set; a disabled field's output is tied to 0. The busy detect feeds the clock-gate wrapper, and the wakeup hold keeps `m_axi_awakeup` asserted while an accepted request is still waiting in a skid buffer, not only while `fub_axi_awakeup` is high (IHI 0022L A14.1.1).

8.4.2 AXI5 Enhancements Over AXI4

Atomic Operations: - **AWATOP:** Supports atomic Compare/Swap, Fetch/Add, Fetch/And, etc. - Enables lock-free synchronization primitives

Security and Isolation: - **AWNSAID:** Identifies security domain for non-secure accesses - **AWMECID:** Provides encryption context for secure memory regions

Performance and Monitoring: - **AWTRACE:** Enables performance trace and debug capabilities - **AWMPAM:** Supports memory bandwidth partitioning and QoS

Advanced Features: - **AWIDUNQ / BIDUNQ:** Unique ID indicator on the request and its write response (IHI 0022L §A5.2) - **AWTAGOP/WTAG:** Memory Tagging Extension for security - **WTAGUPDATE:** Selective tag update mask

Data Integrity: - **WPOISON:** Indicates corrupted or test data - **BTRACE / BIDUNQ / BTAGMATCH[1:0]:** Response trace, unique-ID indicator and the 2-bit tag match result

Not implemented: - **AWREGION:** No port on this module. AxREGION is not deprecated by AXI5; it remains a valid optional signal and is simply omitted here. Decode or route by address instead, or use `axi4_master_wr`

8.4.3 Atomic Transactions

`AWATOP` is 6 bits wide (`AXI_ATOP_WIDTH = 6`). The module transports it unmodified; it does not perform the atomic operation. Executing the read-modify-write and returning the original data on the R channel is the downstream endpoint's responsibility.

AWATOP[5:4] - transaction class:

AWATOP[5:4]	Class	Description
2'b00	NonAtomic	Ordinary write; AWATOP[3:0] must be zero
2'b01	AtomicStore	Operation applied at the endpoint; no read data returned
2'b10	AtomicLoad	Operation applied at the endpoint; original data returned on R
2'b11	AtomicSwap / AtomicCompare	Selected by AWATOP[3:0] (see below)

AWATOP[3] - endianness (AtomicStore and AtomicLoad only):

AWATOP[3]	Meaning
1'b0	Little-endian operand
1'b1	Big-endian operand

AWATOP[2:0] - arithmetic/logical opcode (AtomicStore and AtomicLoad only):

AWATOP[2:0]	Operation
3'b000	ADD
3'b001	CLR (AND NOT)

AWATOP[2:0]	Operation
3’b010	EOR (XOR)
3’b011	SET (OR)
3’b100	SMAX (signed maximum)
3’b101	SMIN (signed minimum)
3’b110	UMAX (unsigned maximum)
3’b111	UMIN (unsigned minimum)

Full-field encodings when AWATOP[5:4] is 2’b11:

AWATOP	Transaction
6’b110000	AtomicSwap
6’b110001	AtomicCompare
All other 2’b11 values	Reserved

Set `ENABLE_ATOMIC = 0` to drop `AWATOP` from the AW SKID payload when atomics are not used.

8.5 Timing Characteristics

8.5.1 Basic Write Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- AWID, AWADDR, AWLEN, AWSIZE
- AWVALID, AWREADY
- AWATOP, AWNSAID, AWTRACE (AXI5 extensions)
- WDATA, WSTRB, WLAST
- WVALID, WREADY
- WPOISON, WTAG (AXI5 extensions)
- BID, BRESP
- BVALID, BREADY
- BTRACE, BIDUNQ, BTAGMATCH (AXI5 extensions)

8.5.2 Atomic Operation

Timing diagram pending. The signals and sequence this scenario exercises:

- AWATOP encoding for atomic operation
- Write data for atomic operand
- Response with atomic result

8.5.3 Memory Tagging Extension (MTE)

Timing diagram pending. The signals and sequence this scenario exercises:

- AWTAGOP encoding
- AWTAG delivery with address
- WTAG delivery with data
- WTAGUPDATE mask
- BTAGMATCH response

8.6 Usage Examples

```
axi5_master_wr #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    // Enable AXI5 features
    .ENABLE_ATOMIC      (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_master_wr (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // FUB interface (slave side)
    .fub_axi_awid       (fub_awid),
    .fub_axi_awaddr     (fub_awaddr),
    .fub_axi_awlen      (fub_awlen),
    .fub_axi_awsize     (fub_awsize),
    .fub_axi_awburst    (fub_awburst),
    .fub_axi_awlock     (fub_awlock),
    .fub_axi_awcache    (fub_awcache),
    .fub_axi_awprot     (fub_awprot),
    .fub_axi_awqos      (fub_awqos),
    .fub_axi_awuser     (fub_awuser),
    .fub_axi_awvalid    (fub_awvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_awready    (fub_awready),

    // AXI5 AW extensions
    .fub_axi_awatop     (fub_awatop),
    .fub_axi_awnsaid    (fub_awnsaid),
    .fub_axi_awtrace    (fub_awtrace),
    .fub_axi_awmpam     (fub_awmpam),
    .fub_axi_awmecid    (fub_awmecid),
    .fub_axi_awidunq   (fub_awidunq),
    .fub_axi_awtagop    (fub_awtagop),

    // W channel
```

```
    .fub_axi_wdata      (fub_wdata),
    .fub_axi_wstrb      (fub_wstrb),
    .fub_axi_wlast      (fub_wlast),
    .fub_axi_wuser      (fub_wuser),
    .fub_axi_wvalid     (fub_wvalid),
    .fub_axi_wready     (fub_wready),

    // AXI5 W extensions
    .fub_axi_wpoison    (fub_wpoison),
    .fub_axi_wtag       (fub_wtag),
    .fub_axi_wtagupdate (fub_wtagupdate),

    // B channel
    .fub_axi_bid        (fub_bid),
    .fub_axi_bresp      (fub_bresp),
    .fub_axi_buser      (fub_buser),
    .fub_axi_bvalid     (fub_bvalid),
    .fub_axi_bready     (fub_bready),

    // AXI5 B extensions
    .fub_axi_btrace     (fub_btrace),
    .fub_axi_bidunq     (fub_bidunq),
    .fub_axi_btagmatch  (fub_btagmatch),

    // Master interface (output side)
    .m_axi_awid         (m_axi_awid),
    .m_axi_awaddr       (m_axi_awaddr),
    // Every remaining m_axi_* port mirrors the fub_axi_* list above,
    // same names and widths, opposite directions. All must be
connected.

    // Status
    .busy               (master_wr_busy)
);
```

8.7 Design Notes

8.7.1 Atomic Operation Support

When ENABLE_ATOMIC=1: - **AWATOP[5:4]** selects atomic type (Swap/Compare/Fetch-op) - **AWATOP[3:0]** selects operation for Fetch-op - Write data provides operand value - Response may include atomic result (implementation-dependent)

Common atomic operations: - **Atomic Swap:** Exchange memory value with register - **Compare and Swap:** Update memory if value matches - **Fetch and Add:** Read-modify-write atomically

8.7.2 Memory Tagging Extension (MTE)

When ENABLE_MTE=1: - **AWTAGOP:** Specifies tag operation for address tags - **WTAG:** Provides data-level memory tags (1 tag per 16 bytes) - **WTAGUPDATE:** Mask for selective tag updates - **BTAGMATCH[1:0]:** Tag match result, Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)

8.7.3 Poison Support

When ENABLE_POISON=1: - **WPOISON:** Master flags data as poisoned - Use cases: Error injection, cache pollution testing, security

8.7.4 Feature Enable Strategy

Disable unused features to reduce area:

```
.ENABLE_ATOMIC   (0),  // No atomic operations
.ENABLE_NSAID    (0),  // No security domains
.ENABLE_TRACE    (0),  // No trace capability
.ENABLE_MPAM     (0),  // No memory partitioning
.ENABLE_MECID    (0),  // No encryption
.ENABLE_UNIQUE   (0),  // No unique-ID indicator
.ENABLE_MTE      (0),  // No memory tagging
.ENABLE_POISON   (0)   // No poison indication
```

8.8 Related Modules

- **AXI5 Master Read** - Master read interface
- **AXI5 Slave Write** - Slave write interface
- **AXI5 Master Write CG** - Clock-gated variant
- **AXI5 Master Write Monitor** - With integrated monitoring
- **AXI4 Master Write** - AXI4 version for comparison

8.9 Testing

`val/amba/test_axi5_master_wr.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_wr.py -v
```

8.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest — §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A6.4 “Atomic transactions” (AWATOP), §A16.1 “Data protection using Poison” (WPOISON), §A12.2 “Memory Tagging Extension (MTE)” (AWTAGOP, WTAG, WTAGUPDATE, BTAGMATCH), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (AWMECID). The ports follow the specification’s names and widths (GitHub #102): BTAGMATCH is 2 bits with the Table A12.14 encodings, there is no BTAG or AWTAG, the Unique ID indicator is AWIDUNQ with BIDUNQ on the response, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h — the same AMBA 5 features as first numbered: ch. C1 “AMBA AXI5”, §E1.1 “Atomic transactions”, §E1.15 “Memory tagging”, §E2.1 “Poison”, §E1.13 “Unique ID indicator”. MECID is not in Issue H.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 — relay stations: a channel can be pipelined by any number of back-pressured stages without changing its behavior, which is why a skid buffer per channel is transparent to both ends.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC ’06)*, July 2006, pp. 657-662. DOI: 10.1145/1146909.1147077 — the elastic buffer with valid/stop back-pressure, the structure a skid buffer implements.
- Chipressian. *Crack the Hardware Interview: from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (forward and backward register slices) — textbook treatment of the valid/ready slice (secondary source).

- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

8.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

9 AXI5 Master Write with Clock Gating

Module: `axi5_master_wr_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

9.1 Overview

The AXI5 Master Write with Clock Gating module wraps the standard `axi5_master_wr` core with intelligent clock gating for power optimization. When no AXI write activity is detected for a configurable number of idle cycles, the clock gates off automatically.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. `AWATOP` is carried through unmodified but no atomic read-modify-write is performed, no MTE tag checking or `BTAGMATCH` generation is performed, and no outstanding-transaction tracking is done. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

9.1.1 Key Features

- Carries the same AXI5 signal set as `axi5_master_wr`, which it wraps
- **AWATOP:** Atomic operation support (compare-and-swap, atomic operations)
- **AWNSAID:** Non-secure access identifier for security domains
- **AWTRACE:** Trace signal for debug and performance monitoring
- **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **AWMECID:** Memory Encryption Context ID for secure memory
- **AWIDUNQ / BIDUNQ:** Unique ID indicator on the request and its write response (IHI 0022L §A5.2)
- **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **WPOISON:** Write data poison indicator for corrupted data detection
- **WTAG/WTAGUPDATE:** Memory tags and tag update control (MTE)
- **BTRACE / BIDUNQ / BTAGMATCH[1:0]:** Response trace, unique-ID indicator and the 2-bit tag match result
- **Automatic clock gating** based on activity detection
- **Configurable idle count** before clock gating activates
- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Status outputs** for clock gating state monitoring
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

---

9.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context

Parameter	Type	Default	Description
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

9.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
SW	AXI_WSTRB_WIDTH	Write strobe width, one bit per data byte
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `wtag` field
AWSize	Sum of the enabled AW fields	AW SKID buffer payload width
WSize	Sum of the enabled W fields	W SKID buffer payload width
BSize	Sum of the enabled B fields	B SKID buffer payload width

9.3 Ports

9.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

9.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

9.3.3 FUB AXI5 Interface (Slave Side - Input)

9.3.3.1 AW Channel

Port	Width	Direction	Description
fub_axi_awid	IW	Input	Write address ID
fub_axi_awaddr	AW	Input	Write address
fub_axi_awlen	8	Input	Burst length
fub_axi_awsize	3	Input	Burst size
fub_axi_awburst	2	Input	Burst type
fub_axi_awlock	1	Input	Lock type
fub_axi_awcache	4	Input	Cache attributes
fub_axi_aw	3	Input	Protection attributes

Port	Width	Direction	Description
prot			
fub_axi_awqos	4	Input	Quality of Service
fub_axi_awuser	UW	Input	User-defined signal
fub_axi_awvalid	1	Input	Write address valid
fub_axi_awakeup	1	Input	Wake-up request, AWAKEUP (IHI 0022L §A14.1.1; `ENABLE_WAKEUP`)
fub_axi_awready	1	Output	Write address ready (forced 0 when gating)

9.3.3.2 AXI5 AW Extensions

Port	Width	Direction	Description
fub_axi_awatop	AXI_ATOP_WIDTH	Input	Atomic operation
fub_axi_awnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID
fub_axi_awtrace	1	Input	Trace signal
fub_axi_awmpam	AXI_MPAM_WIDTH	Input	Memory partitioning/monitoring
fub_axi_awmecid	AXI_MECID_WIDTH	Input	Memory encryption context ID
fub_axi_awidunq	1	Input	Unique ID indicator, AWIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_awtagop	AXI_TAGOP_WIDTH	Input	Tag operation (MTE)

9.3.3.3 W Channel

Port	Width	Direction	Description
fub_axi_w	DW	Input	Write data

Port	Width	Direction	Description
data			
fub_axi_wstrb	SW	Input	Write strobes
fub_axi_wlast	1	Input	Last transfer in burst
fub_axi_wuser	UW	Input	User-defined signal
fub_axi_wvalid	1	Input	Write data valid
fub_axi_wready	1	Output	Write data ready (forced 0 when gating)

9.3.3.4 AXI5 W Extensions

Port	Width	Direction	Description
fub_axi_wpoison	1	Input	Write data poison indicator
fub_axi_wtag	TW	Input	Memory tags on write data
fub_axi_wtagupdate	NUM_TAGS	Input	Tag update control (per 16B)

9.3.3.5 B Channel

Port	Width	Direction	Description
fub_axi_bid	IW	Output	Write response ID
fub_axi_bresp	2	Output	Write response
fub_axi_buser	UW	Output	User-defined signal
fub_axi_bvalid	1	Output	Write response valid
fub_axi_bready	1	Input	Write response ready

9.3.3.6 AXI5 B Extensions

Port	Width	Direction	Description
fub_axi_btrace	1	Output	Trace signal
fub_axi_bidunq	1	Output	Unique ID indicator on the write response, BIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
fub_axi_btagmatch	2	Output	Tag match result, IHI 0022L Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)

9.3.4 Master AXI5 Interface (Output Side)

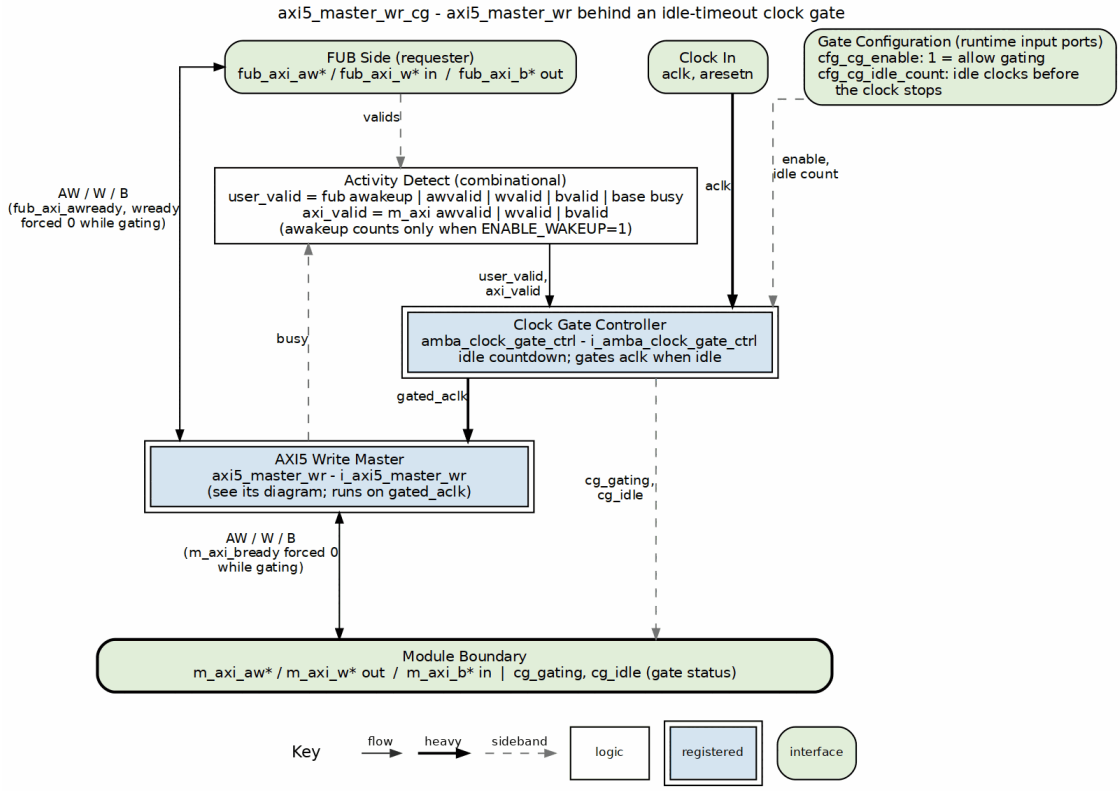
Same port list as FUB interface but with `m_axi_*` prefix and reversed directions.

9.3.5 Clock Gating Status Outputs

Port	Width	Direction	Description
cg_gating	1	Output	Clock gating active (1=clock gated, 0=clock running)
cg_idle	1	Output	Interface idle (1=no activity detected)

9.4 Functional Description

9.4.1 Architecture



axi5_master_wr_cg block diagram

The base `axi5_master_wr` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `fub_axi` valids, the child's busy, and `fub_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `m_axi` valids.

9.4.2 Clock Gating Operation

The clock gating controller monitors activity on both user and AXI sides for all three write channels:

User-side activity detection:

```
user_valid = (ENABLE_WAKEUP && fub_axi_awakeup) || fub_axi_awvalid ||
fub_axi_wvalid || fub_axi_bvalid || int_busy  // peer VALID (post-fix)
```

AXI-side activity detection:

```
axi_valid = m_axi_awvalid || m_axi_wvalid || m_axi_bvalid
```

Ready signal control during gating:

```
fub_axi_awready = cg_gating ? 1'b0 : int_awready;
fub_axi_wready = cg_gating ? 1'b0 : int_wready;
m_axi_bready = cg_gating ? 1'b0 : int_bready;
```

All three write channels must be idle before clock gating activates, ensuring no partial writes.

9.4.3 Ready Deassertion and Wake Behavior

While `cg_gating` is asserted, the ready outputs listed above are forced low.

This is legal AXI. A READY output may be deasserted for any number of cycles, and neither side may assume READY asserts within a bounded time, so holding it low is ordinary backpressure rather than a protocol violation. It cannot deadlock either: the same VALID being held off also feeds the activity detector, so gating releases and READY returns to the core's value.

Two consequences are worth designing around:

- **Wake latency adds to first-transfer latency.** Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. The AXI5 `_cg` wrappers drive the activity terms combinationally, so there is 1 register stage and the first usable gated-clock edge arrives 2 cycles after activity asserts. Size `cfg_cg_idle_count` so that cost is amortized over the traffic burst that follows.
- **READY is a combinational function of `cg_gating`.** The path from the idle counter to the `fub_axi_awready` and `fub_axi_wready` outputs may need attention during timing closure, particularly at high frequency.

9.4.4 Integration Caveat: `fub_axi_bready` in the Activity Term

The activity detector uses the corresponding VALID (`fub_axi_bvalid`), never the peer's READY. (Historically this term was `fub_axi_bready`, which meant a peer parking its ready high – a common and entirely correct style – pinned the block permanently awake and defeated gating; that defect was fixed family-wide, matching the rule the mon_cg wrappers always carried.)

An always-ready consumer no longer prevents gating.

9.4.5 AXI5 Write-Specific Features

Atomic transactions (AWATOP): When `ENABLE_ATOMIC=1`, `AWATOP` is packed into the AW SKID payload and carried through unmodified, so the following can be signalled end to end: -

AtomicStore - AtomicLoad - AtomicSwap - AtomicCompare - The ADD/CLR/EOR/SET/SMAX/SMIN/UMAX/UMIN opcodes of AtomicStore and AtomicLoad

The module does not execute the atomic operation. See AXI5 Master Write for the full `AWATOP` encoding.

Memory Tagging on Write (MTE): When `ENABLE_MTE=1`: - **AWTAGOP:** Specifies tag operation for write address - **WTAG:** Memory tags sent with each data beat - **WTAGUPDATE:** Controls which tags to update (per 16 bytes) - **BTAGMATCH[1:0]:** Tag match result, Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)

Write Poison (WPOISON): When `ENABLE_POISON=1`: - Indicates corrupted or invalid write data - Propagated to memory/cache - May trigger error response (SLVERR)

9.4.6 Idle Counter Configuration

Same as read variant - see AXI5 Master Read CG.

9.5 Timing Characteristics

9.5.1 Clock Gating with Write Burst

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- AW channel: AWVALID, AWREADY
- W channel: WDATA, WLAST, WVALID, WREADY
- B channel: BVALID, BREADY
- Idle counter after BREADY
- cg_gating activation after idle count

9.5.2 Atomic Operation with Clock Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- AWATOP encoding
- Atomic write sequence
- Clock gating after completion

9.6 Usage Examples

```
axi5_master_wr_cg #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .AXI_USER_WIDTH     (4),
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    // Enable AXI5 features
    .ENABLE_ATOMIC      (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1),
    // Clock gating configuration
    .CG_IDLE_COUNT_WIDTH (4)
) u_axi5_master_wr_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Clock gating configuration
    .cfg_cg_enable      (power_save_enable),
    .cfg_cg_idle_count  (4'd3),  // Gate after 4 idle cycles

    // FUB interface (slave side)
    .fub_axi_awid       (fub_awid),
    .fub_axi_awaddr     (fub_awaddr),
    .fub_axi_awlen      (fub_awlen),
    .fub_axi_awsize     (fub_awsize),
    .fub_axi_awburst    (fub_awburst),
    .fub_axi_awlock     (fub_awlock),
    .fub_axi_awcache    (fub_awcache),
    .fub_axi_awprot     (fub_awprot),
    .fub_axi_awqos      (fub_awqos),
    .fub_axi_awuser     (fub_awuser),
    .fub_axi_awvalid    (fub_awvalid),
    .fub_axi_awakeup    (fub_awakeup),
    .fub_axi_awready    (fub_awready),

    // AXI5 AW extensions
    .fub_axi_awatop     (fub_awatop),
    .fub_axi_awnsaid    (fub_awnsaid),
    .fub_axi_awtrace    (fub_awtrace),
```

```
    .fub_axi_awmpam     (fub_awmpam),
    .fub_axi_awmecid    (fub_awmecid),
    .fub_axi_awidunq   (fub_awidunq),
    .fub_axi_awtagop    (fub_awtagop),

    // W channel
    .fub_axi_wdata      (fub_wdata),
    .fub_axi_wstrb      (fub_wstrb),
    .fub_axi_wlast      (fub_wlast),
    .fub_axi_wuser      (fub_wuser),
    .fub_axi_wvalid     (fub_wvalid),
    .fub_axi_wready     (fub_wready),

    // AXI5 W extensions
    .fub_axi_wpoison    (fub_wpoison),
    .fub_axi_wtag       (fub_wtag),
    .fub_axi_wtagupdate (fub_wtagupdate),

    // B channel
    .fub_axi_bid        (fub_bid),
    .fub_axi_bresp      (fub_bresp),
    .fub_axi_buser      (fub_buser),
    .fub_axi_bvalid     (fub_bvalid),
    .fub_axi_bready     (fub_bready),

    // AXI5 B extensions
    .fub_axi_btrace     (fub_btrace),
    .fub_axi_bidunq     (fub_bidunq),
    .fub_axi_btagmatch  (fub_btagmatch),

    // Master interface (output side)
    .m_axi_awid         (m_axi_awid),
    .m_axi_awaddr       (m_axi_awaddr),
    // Every remaining m_axi_* port mirrors the fub_axi_* list above,
    // same names and widths, opposite directions. All must be
connected.

    // Clock gating status
    .cg_gating          (clock_gated),
    .cg_idle            (interface_idle)
);

// Power monitoring
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        write_count <= '0;
```

```
        power_cycles_saved <= '0;
    end else begin
        if (fub_awvalid && fub_awready)
            write_count <= write_count + 1;

        if (clock_gated)
            power_cycles_saved <= power_cycles_saved + 1;
    end
end
```

9.7 Design Notes

9.7.1 Write vs. Read Clock Gating

Aspect	Read (AR/R)	Write (AW/W/B)
Channels	2 (AR, R)	3 (AW, W, B)
Activity detection	2 valids	3 valids
Idle condition	Both channels idle	All 3 channels idle
Typical idle %	Higher (read-mostly traffic)	Lower (write completion delays)
Relative gating opportunity	Higher	Lower

9.7.2 Write-Specific Gating Considerations

Write data buffering: - Clock may gate after AW but before W (if W delayed) - SKID buffers ensure no data loss - Gate activation waits for W channel idle

Response latency: - B channel responses may delay gating - Typical: B arrives many cycles after W - Idle counter starts after BREADY

Atomic operations: - Longer latency than normal writes - Atomic read-modify-write sequence - Idle detection after atomic completion

---

9.8 Related Modules

- **AXI5 Master Write** - Non-gated base module
- **AXI5 Master Read CG** - Read with clock gating
- **AXI5 Master Write Monitor CG** - With monitoring + clock gating
- **AMBA Clock Gate Controller** - Clock gating controller spec

9.9 Testing

`val/amba/test_axi5_master_wr_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_wr_cg.py -v
```

9.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A6.4 “Atomic transactions” (AWATOP), §A16.1 “Data protection using Poison” (WPOISON), §A12.2 “Memory Tagging Extension (MTE)” (AWTAGOP, WTAG, WTAGUPDATE, BTAGMATCH), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (AWMECID). The ports follow the specification’s names and widths (GitHub #102): BTAGMATCH is 2 bits with the Table A12.14 encodings, there is no BTAG or AWTAG, the Unique ID indicator is AWIDUNQ with BIDUNQ on the response, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).

- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

9.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

10 AXI5 Master Write with Monitor

Module: `axi5_master_wr_mon.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

10.1 Overview

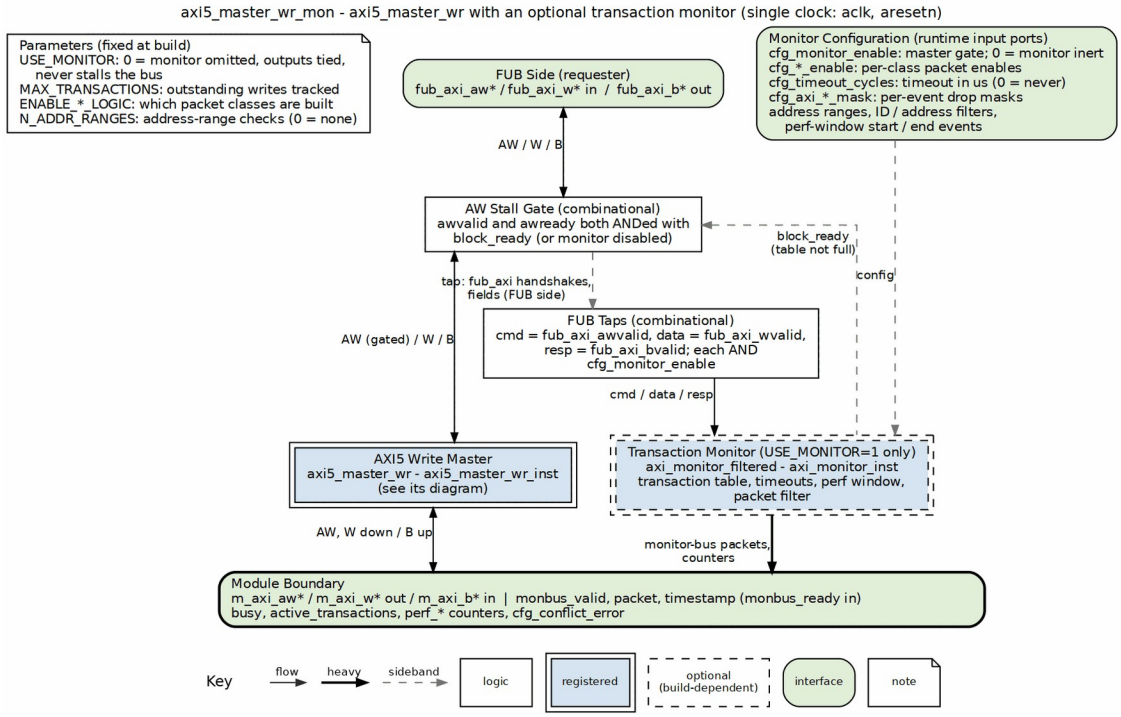
The AXI5 Master Write with Monitor module wraps the standard `axi5_master_wr` core with an integrated `axi_monitor_filtered`, so every write transaction crossing the core is also visible to the monitor in real time. You get write transaction monitoring, error detection, and configurable packet filtering in one block — no external monitor to wire up.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

10.1.1 Key Features

- Carries the full AXI5 signal set unmodified (wraps `axi5_master_wr`) – transport, not semantics; see Scope above
- **AWATOP:** Atomic operation support (compare-and-swap, atomic operations)
- **AWNSAID:** Non-secure access identifier for security domains
- **AWTRACE:** Trace signal for debug and performance monitoring
- **AWMPAM:** Memory Partitioning and Monitoring (PartID + PMG)
- **AWMECID:** Memory Encryption Context ID for secure memory
- **AWIDUNQ / BIDUNQ:** Unique ID indicator on the request and its write response (IHI 0022L §A5.2)
- **AWTAGOP:** Memory tag operation (MTE - Memory Tagging Extension)
- **WPOISON:** Write data poison indicator for corrupted data detection
- **WTAG/WTAGUPDATE:** Memory tags and tag update control (MTE)
- **BTRACE / BIDUNQ / BTAGMATCH[1:0]:** Response trace, unique-ID indicator and the 2-bit tag match result
- **Integrated AXI monitor** with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)
- **Error detection:** Protocol violations, SLVERR, DECERR
- **Timeout monitoring:** Stuck transactions, stalled channels
- **Performance metrics:** Latency, throughput, outstanding transactions
- **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
- **Configuration validation:** Detects filter conflicts

10.1.2 Module Architecture



axi5_master_wr_mon block diagram

The base `axi5_master_wr` is one box (see its own diagram). Traffic runs down the left through the AW stall gate, which ANDs `fub_axi_awvalid` and its ready with the `block_ready` the monitor raises while its transaction table has room. The taps watch the FUB-side (`fub_axi`) handshake – the same point the stall gate acts on, so a falling `block_ready` stops new events with no skid lag – and feed `axi_monitor_filtered`, which is built only when `USE_MONITOR=1`; its monitor-bus packets are the heavy edge out. With `cfg_monitor_enable` low the monitor is inert and never stalls the bus.

10.2 Parameters

A lite-monitor version of this wrapper exists as `axi5_master_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Transport sizing first, then the monitor knobs. The defaults are sane; `MAX_TRANSACTIONS` is the one to think about (see the backpressure section).

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_WSTRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations

Parameter	Type	Default	Description
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	11	Monitor agent identifier (default 11 for write)
MAX_TRANSACTIONS	int	16	Transaction table size
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when `cfg_threshold_enable=1`. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. >**1 on a WRITE**

Parameter	Type	Default	Description
			monitor requires `USE_WDATA_ORDER_Q=1` – `axi_monitor_trans_mgr` fails elaboration otherwise
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
ACLK_MHZ	int	100	Clock frequency in MHz – keeps the 1 us tick exact off-100MHz
CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 =

Parameter	Type	Default	Description
			checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the `axi_monitor_timeout` instance.
ENABLE_COMPL_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six `ENABLE_*_LOGIC` parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's `axi_monitor_filtered` instance the perf/debug master switches are fixed (`ENABLE_PERF_PACKETS = 1`, `ENABLE_DEBUG_MODULE = 0`). The former `CAM_PIPELINE` / `TRANS_CAM_PIPELINE` parameters were removed — the transaction CAM is now always pipelined.

10.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS

10.3 Ports

10.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

10.3.2 FUB AXI5 Interface (Slave Side)

Same as `axi5_master_wr` - see AXI5 Master Write for complete port listing.

10.3.3 Master AXI5 Interface (Output Side)

Same as `axi5_master_wr` - see AXI5 Master Write for complete port listing.

10.3.4 Monitor Configuration

Same as `axi5_master_rd_mon` - see AXI5 Master Read Monitor for complete configuration port listing. This includes the `cam_clear` control input (1, Input) - synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4]).

The configuration set also includes the detection-cone enables `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` (each 1, Input) which turn on the completion, threshold, and debug reporter sub-blocks. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4) and `cfg_debug_mask` (16) inputs are tied to `4'h0` / `16'h0`; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`). None are wrapper ports.

10.3.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	`monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with

Port	Width	Direction	Description
			monbus_packet
debug_block_ready	1	Output	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)
i_mon_time	64	Input	Free-running counter from monbus_group_core, sampled at packet emission

10.3.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding write transactions
error_count	16	Output	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
transaction_count	32	Output	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when ENABLE_PERF_LOGIC=0 or USE_MONITOR=0)
cfg_conflict_error	1	Output	Configuration conflict detected

10.3.7 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
`cfg_addr_filter_enable`	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
`cfg_addr_filter_low`	`ADDR_WIDTH`	Window base, inclusive
`cfg_addr_filter_high`	`ADDR_WIDTH`	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (`N_ADDR_RANGES`), which re-evaluates per command.

Runtime ID filter — overrides the `ID_MATCH_BASE`/`ID_MATCH_COUNT` elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
`cfg_id_filter_enable`	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
`cfg_id_match_base`	`ID_WIDTH`	First ID owned
`cfg_id_match_count`	`ID_WIDTH+1`	How many; `0` means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: `[base, base + count)`.

10.4 Functional Description

10.4.1 Monitor Bus Packet Format

Same 128-bit standardized format (with 64-bit side-band timestamp) as read monitor - see AXI5 Master Read Monitor.

10.4.2 Write-Specific Monitoring Events

Write Transaction Tracking: The monitor correlates three channels: 1. **AW:** Address and control (AWID, AWADDR, AWLEN, etc.) 2. **W:** Write data beats (WDATA, WSTRB, WLAST) 3. **B:** Write response (BID, BRESP)

Event Timeline:

```
1. AW handshake → Transaction started
2. W beats → Data transfer monitored
3. WLAST → Write data complete
4. B response → Transaction complete
```

Error Detection Events:

Protocol Errors (implemented set): - Response-before-data ordering (EVT_PROTOCOL) - Orphaned responses (orphaned WRITE DATA cannot be detected on an AXI write monitor – with IS_AXI=1 data never allocates a table entry, so an unmatched W burst is invisible)

(Handshake-stability, WLAST, ID-width, burst-length, alignment and strobe checks are NOT implemented – the monitor taps only cmd/data/resp handshakes, IDs and response codes; none of those signals reach it.)

Response Errors: - SLVERR (slave error response) - DECERR (decode error response) - Orphaned write response (no matching AW) – a B beat whose ID matches no live entry

There is no BID-versus-AWID comparison. A slave returning the BID of a *different* outstanding write is silently mis-attributed with no error; a BID matching nothing is the orphan case above.

Timeout Errors – per phase, measured as DURATION not stall: - AW phase outstanding too long – `EVT_CMD_TIMEOUT` - W phase outstanding too long – `EVT_DATA_TIMEOUT` - B phase outstanding too long – `EVT_RESP_TIMEOUT`

Each timer is zeroed only while its phase is **not pending** (`if (!w_data_pending[idx]) r_data_timer[idx] <= '0;`) – a beat handshake does NOT reset it. These are phase-duration limits, not stall detectors: a long multi-beat write burst making steady progress still trips `EVT_DATA_TIMEOUT` once the W phase as a whole exceeds `cfg_timeout_cycles`.

`axi_monitor_timeout` runs three per-phase timers (addr, data, resp) and there is no whole-transaction timer. A write can therefore take arbitrarily long AW-to-B without any timeout firing, provided each individual phase completes inside its own threshold – the gaps BETWEEN phases are not timed by anything.

Data Integrity: NOT monitored – WPOISON, MTE tags and ATOP status never reach the monitor (the AXI5 extension signals pass through the transport core untapped). The bullets this section once carried were a wishlist, not the implementation.

Threshold Violations: - Outstanding transaction count > threshold - Transaction latency > cfg_latency_threshold

10.4.3 Atomic Operation Monitoring

NOT implemented: `ENABLE_ATOMIC` shapes the TRANSPORT (AWATOP carried on the AW channel); the monitor does not tap AWATOP or BTAGMATCH and emits no atomic-specific events. BRESP errors on atomic transactions surface as ordinary SLVERR/DECERR events.

10.4.4 Filtering Hierarchy (two active levels, one reserved)

Same as read monitor - see AXI5 Master Read Monitor.

10.4.5 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `fub_axi_awready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `fub_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, `ap_disabled_never_stalls`; this module has no `ifdef FORMAL` block of its own, so here the guarantee rests on the gate expression above, not a proof).
- The monitor taps the FUB-side handshake (every monitor taps the FUB side, for timing), which on a master is also where block_ready gates: the watchpoint and the gate coincide, so new events stop the cycle block_ready goes low, with no skid-depth lag.

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `ocelus_common_pkg`), and `block_ready` re-asserts at occupancy `< MAX_TRANSACTIONS - (reserve - 1)` – a threshold strictly ABOVE the

command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

10.4.6 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper does not expose `ADDR_RANGE_IS_ERROR`; all ranges default to the DEBUG (match) flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

10.4.7 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

10.4.7.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the

cfg_perf_enable edge) and can also be fired directly by the cfg_start_trigger / cfg_end_trigger pulses (from an engine or CSR write). cfg_window_force_close is a software override that closes the window immediately. While the window is open:

- window_active is high.
- window_cycles free-runs, counting every clock elapsed inside the window.

10.4.7.2 Utilization buckets (W data channel)

Every cycle inside the window is classified by the **W** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
perf_prod_cycles	32	wvalid && wready	productive beat transferred
perf_bp_cycles	32	wvalid && !wready	back-pressure (data offered, sink not ready)
perf_starv_cycles	32	!wvalid && wready	starvation (sink ready, no data)
perf_idle_cycles	32	!wvalid && !wready	idle

The four buckets sum to window_cycles - 1 (the start cycle seeds window_cycles to 1 while the buckets reset to 0); utilization = perf_prod_cycles / window_cycles is off by one count – negligible for long windows, use window_cycles - 1 for exactness.

10.4.7.3 Throughput counters

Output	Width	Meaning
perf_beat_count	32	W data beats transferred (= perf_prod_cycles, 1 beat/cycle)
perf_byte_count	64	bytes transferred = beats x (1 << latched AWSIZE), using the AWSIZE captured at the most recent AW address phase
perf_burst_count	32	AW address-phase handshakes

Average burst length is `perf_beat_count / perf_burst_count`. (Transaction completion is still tracked on the B channel; the throughput counters measure the W data phase.)

10.4.7.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window **start** event source select
cfg_end_event_sel	3	Input	Window **end** event source select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	`wvalid && wready` cycles
perf_bp_cycles	32	Output	`wvalid && !wready` cycles (back-pressure)
perf_starv_cycles	32	Output	`!wvalid && wready` cycles (starvation)
perf_idle_cycles	32	Output	`!wvalid && !wready` cycles
perf_beat_count	32	Output	W data beats transferred

Port	Width	Direction	Description
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AW address-phase handshakes

When `USE_MONITOR = 0` all perfmon outputs are tied to 0. With `USE_MONITOR = 1` and `ENABLE_PERF_LOGIC = 0`, only error_count / transaction_count read 0 – the window and bucket/throughput outputs remain LIVE.

10.5 Timing Characteristics

10.5.1 Monitored Write Burst with Error

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- AW channel: AWID, AWADDR, AWLEN, AWVALID, AWREADY
- W channel: WDATA, WSTRB, WLAST, WVALID, WREADY
- B channel: BID, BRESP (SLVERR), BVALID, BREADY
- Monitor bus: monbus_valid, monbus_packet showing ERROR packet

10.5.2 Atomic Operation Monitoring

Timing diagram pending. The signals and sequence this scenario exercises:

- AWATOP encoding (atomic type)
- AW/W channels
- Monitor packets for atomic sequence

10.5.3 Write Timeout Detection

Timing diagram pending. The signals and sequence this scenario exercises:

- AW channel: AWVALID asserted, AWREADY stuck low
- Timeout counter incrementing
- cfg_timeout_cycles threshold
- Monitor bus: TIMEOUT packet generated

10.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_master_wr_mon #(
    .SKID_DEPTH_AW         (2),
    .SKID_DEPTH_W          (4),
    .SKID_DEPTH_B          (2),
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (32),
    .AXI_USER_WIDTH        (1),
    .AXI_ATOP_WIDTH        (6),
    .AXI_NSAID_WIDTH       (4),
    .AXI_MPAM_WIDTH        (11)
) u_axi5_master_wr_mon (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cam_clear             (cam_clear),
    .fub_axi_awid          (fub_axi_awid),
    .fub_axi_awaddr        (fub_axi_awaddr),
    .fub_axi_awlen         (fub_axi_awlen),
    .fub_axi_awsize        (fub_axi_awsize),
    .fub_axi_awburst       (fub_axi_awburst),
    .fub_axi_awlock        (fub_axi_awlock),
    .fub_axi_awcache       (fub_axi_awcache),
    .fub_axi_awprot        (fub_axi_awprot),
    .fub_axi_awqos         (fub_axi_awqos),
    .fub_axi_awuser        (fub_axi_awuser),
    .fub_axi_awvalid       (fub_axi_awvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_awready       (fub_axi_awready),
    .fub_axi_awatop        (fub_axi_awatop),
    .fub_axi_awnsaid       (fub_axi_awnsaid),
    .fub_axi_awtrace       (fub_axi_awtrace),
    .fub_axi_awmpam        (fub_axi_awmpam),
    .fub_axi_awmecid       (fub_axi_awmecid),
    .fub_axi_awidunq      (fub_axi_awidunq),
    .fub_axi_awtagop       (fub_axi_awtagop),
    .fub_axi_wdata         (fub_axi_wdata),
    .fub_axi_wstrb         (fub_axi_wstrb),
    .fub_axi_wlast         (fub_axi_wlast),
    .fub_axi_wuser         (fub_axi_wuser),
    .fub_axi_wvalid        (fub_axi_wvalid),
    .fub_axi_wready        (fub_axi_wready),
    .fub_axi_wpoison       (fub_axi_wpoison),
    .fub_axi_wtag          (fub_axi_wtag),
    .fub_axi_wtagupdate    (fub_axi_wtagupdate),
```

```
    .fub_axi_bid           (fub_axi_bid),
    .fub_axi_bresp         (fub_axi_bresp),
    .fub_axi_buser         (fub_axi_buser),
    .fub_axi_bvalid        (fub_axi_bvalid),
    .fub_axi_bready        (fub_axi_bready),
    .fub_axi_btrace        (fub_axi_btrace),
    .fub_axi_bidunq        (fub_axi_bidunq),
    .fub_axi_btagmatch     (fub_axi_btagmatch),
    .m_axi_awid            (m_axi_awid),
    .m_axi_awaddr          (m_axi_awaddr),
    .m_axi_awlen           (m_axi_awlen),
    .m_axi_awsize          (m_axi_awsize),
    .m_axi_awburst         (m_axi_awburst),
    .m_axi_awlock          (m_axi_awlock),
    .m_axi_awcache         (m_axi_awcache),
    .m_axi_awprot          (m_axi_awprot),
    .m_axi_awqos           (m_axi_awqos),
    .m_axi_awuser          (m_axi_awuser),
    .m_axi_awvalid         (m_axi_awvalid),
    .m_axi_awakeup         (m_axi_awakeup),
    .m_axi_awready         (m_axi_awready),
    .m_axi_awatop          (m_axi_awatop),
    .m_axi_awnsaid         (m_axi_awnsaid),
    .m_axi_awtrace         (m_axi_awtrace),
    .m_axi_awmpam          (m_axi_awmpam),
    .m_axi_awmecid         (m_axi_awmecid),
    .m_axi_awidunq        (m_axi_awidunq),
    .m_axi_awtagop         (m_axi_awtagop),
    .m_axi_wdata           (m_axi_wdata),
    .m_axi_wstrb           (m_axi_wstrb),
    .m_axi_wlast           (m_axi_wlast),
    .m_axi_wuser           (m_axi_wuser),
    .m_axi_wvalid          (m_axi_wvalid),
    .m_axi_wready          (m_axi_wready),
    .m_axi_wpoison         (m_axi_wpoison),
    .m_axi_wtag            (m_axi_wtag),
    .m_axi_wtagupdate      (m_axi_wtagupdate),
    .m_axi_bid             (m_axi_bid),
    .m_axi_bresp           (m_axi_bresp),
    .m_axi_buser           (m_axi_buser),
    .m_axi_bvalid          (m_axi_bvalid),
    .m_axi_bready          (m_axi_bready),
    .m_axi_btrace          (m_axi_btrace),
    .m_axi_bidunq          (m_axi_bidunq),
    .m_axi_btagmatch       (m_axi_btagmatch),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
```

```
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .debug_block_ready     (debug_block_ready),
    .cfg_conflict_error    (cfg_conflict_error),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
```

```
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

10.7 Design Notes

10.7.1 W-Channel Wiring into the Monitor Core

The wrapper feeds the shared `axi_monitor_base` data channel from the W channel with `data_id = AWID` (the W channel itself carries no WID in AXI4/AXI5) and `data_resp = 2'b00` (write data carries no response; the response arrives on B). This matches the AXI4 write-monitor convention. Before commit `95c9490a` the AXI5 write monitors wired `data_id = BID` and `data_resp = BRESP` onto W beats (cross-channel), which mis-keyed write-data attribution; that defect is fixed.

10.7.2 Write vs. Read Monitoring Differences

Aspect	Read Monitor	Write Monitor
Channels monitored	2 (AR, R)	3 (AW, W, B)
Transaction ID	ARID → RID	AWID → BID
Data direction	Slave → Master	Master → Slave
Typical latency	Lower	Higher (write buffers)
Common errors	SLVERR on read data	SLVERR on write response
Atomic ops	Load atomics	Store/RMW atomics

10.7.3 Write Transaction Table Management

The transaction table tracks: - **AW phase:** Allocate entry, store address/control - **W phase:** Count beats; completion is WLAST OR the expected count – WLAST is not VERIFIED against AWLEN (no mismatch event) - **B phase:** Match BID to AWID, record response, deallocate entry

Outstanding write tracking:

```
Active transactions = (AW issued) - (B received)
```

10.7.4 Write Latency Measurement

Start: AW handshake (AWVALID && AWREADY) **End:** B handshake (BVALID && BREADY) **Latency:** Clock cycles from start to end

Typical write latencies: - Single-beat write: 5-20 cycles - Burst write: 10-100 cycles - Atomic operation: 20-200 cycles (includes read phase)

10.7.5 Configuration Best Practices

Same as read monitor - see AXI5 Master Read Monitor.

Write-specific tip: Writes often have higher latency variance than reads due to write buffering and posted writes. Set `cfg_latency_threshold` higher for write monitors.

10.8 Related Modules

- **AXI5 Master Write** - Base module without monitoring
- **AXI5 Master Write CG** - With clock gating only
- **AXI5 Master Write Monitor CG** - Monitor + clock gating
- **AXI5 Master Read Monitor** - Read variant
- **AXI Monitor Filtered** - Monitor core specification
- **Monitor Package Spec** - Packet format details
- **AXI Monitor Configuration Guide** - Complete configuration reference

10.9 Testing

`val/amba/test_axi5_master_wr_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_master_wr_mon.py -v
```

10.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- The monitor’s own techniques (transaction table, timeouts, packet filtering) are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects” (“AXI bus protocol overview”) – textbook overview of AXI responses and ID ordering (secondary source).
- Indexed in `rtl/amba/references/README.md` (protocol specifications are citation only)

10.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

11 AXI5 Master Write with Monitor and Clock Gating

Module: `axi5_master_wr_mon_cg.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

11.1 Overview

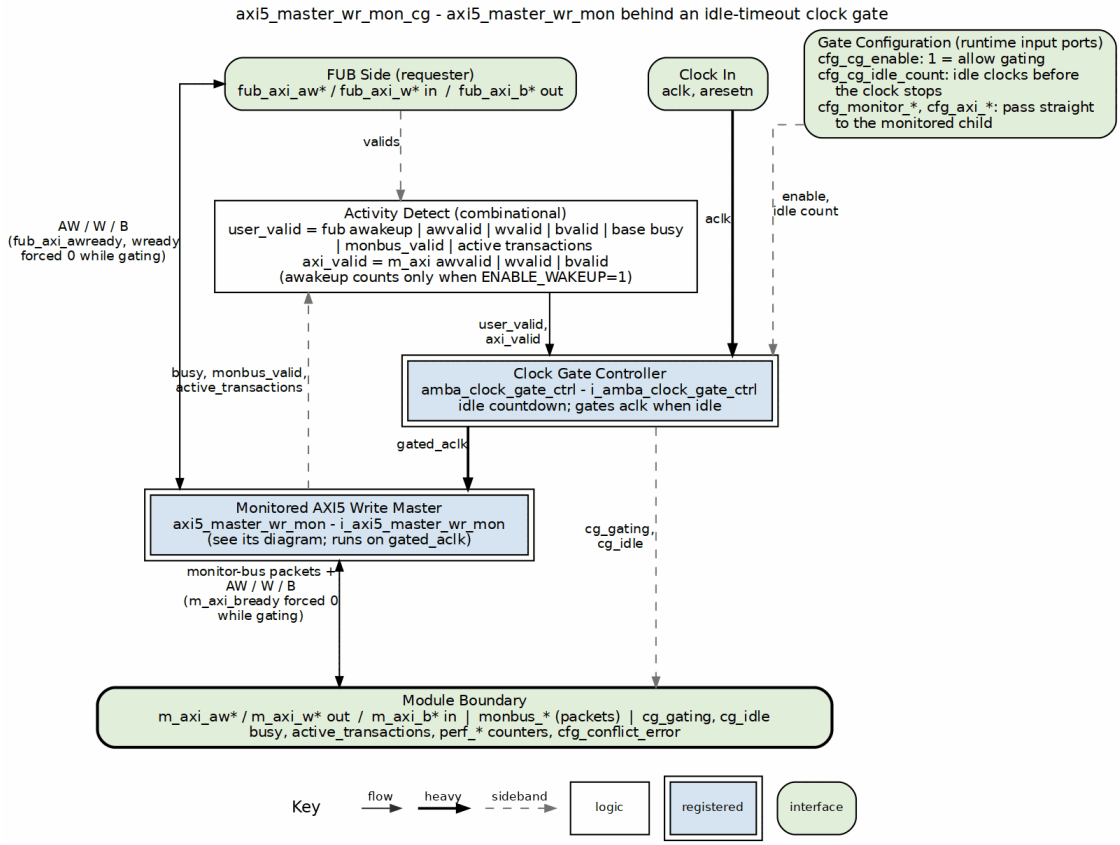
Take the monitored write master and add power management. This module combines `axi5_master_wr_mon` (AXI5 write master with integrated monitoring) with activity-driven clock gating, so the write path keeps its full transaction monitoring and error detection while the clock shuts itself off during idle stretches.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

11.1.1 Key Features

- Carries the full AXI5 signal set unmodified – transport, not semantics; see Scope above
- **All AXI5 write extensions:** ATOP, NSAID, TRACE, MPAM, MECID, IDUNQ, MTE, POISON, WAKEUP
- **Integrated AXI monitor** with 2-Level Filtering: packet-type drop masks + per-event masks (err_select is reserved – no routing)
- **Error detection:** Protocol violations, SLVERR, DECERR
- **Timeout monitoring:** Stuck transactions, stalled channels
- **Performance metrics:** Latency, throughput, outstanding transactions
- **MonBus output:** Standardized 128-bit monitor packet format paired with 64-bit side-band timestamp
- **Automatic clock gating** based on write activity detection
- **Configurable idle threshold** before clock gating activates
- **Power savings** during idle periods
- **Transparent operation** - no protocol changes
- **Dual status outputs:** Monitor status + clock gating status
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

11.1.2 Module Architecture



axi5_master_wr_mon_cg block diagram

The base `axi5_master_wr_mon` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `fub_axi` valids, the child's busy, and `fub_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `m_axi` valids. The monitored child's `monbus_valid` and active-transaction count also count as activity, so the clock never stops under a pending packet.

11.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axi5_master_wr_monlite_cg`: the same gating logic around `axi5_master_wr_monlite` instead of `axi5_master_wr_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width (PartID + PMG)
AXI_MECID_WIDTH	int	16	Memory encryption context ID width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	Enable memory encryption context
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitor unit identifier
AGENT_ID	int	11	Monitor agent identifier (default 11 for write)
MAX_TRANSACTIONS	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage in monitor
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base

Parameter	Type	Default	Description
			module)
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was

Parameter	Type	Default	Description
			unreachable through this wrapper until 2026-09-01.
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. >**1 on this WRITE monitor requires USE_WDATA_ORDER_Q=1** – axi_monitor_trans_mgr fails elaboration otherwise (the WID-less fallback double-counts one W beat across banks).
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
ID_FILTER_ENABLE	bit	0	Synthesises the ID report filter (see cfg_id_* for the runtime override).

Parameter	Type	Default	Description
ID_MATCH_BASE	int	0	First ID this instance owns.
ID_MATCH_COUNT	int	0	How many IDs; 0 means ALL, so a zeroed register block does not silently filter everything away.
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max 2^N-1 cycles)

11.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS

11.3 Ports

11.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

11.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

11.3.3 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Clock gating enable (1=enable, 0=always

Port	Width	Direction	Description
			active)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating activates

11.3.4 FUB AXI5 Interface (Slave Side)

Same as `axi5_master_wr_mon` - see AXI5 Master Write Monitor.

11.3.5 Master AXI5 Interface (Output Side)

Same as `axi5_master_wr_mon` - see AXI5 Master Write Monitor.

11.3.6 Monitor Configuration

Same as `axi5_master_wr_mon` - see AXI5 Master Write Monitor.

11.3.7 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready (backpressure)
monbus_packet	128	Output	`monitor_packet_t` (128-bit format)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

11.3.8 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Core busy indicator
active_transactions	8	Output	Number of outstanding write transactions
error_coun	16	Output	Cumulative error count

Port	Width	Direction	Description
t			(live – reporter lifetime counter)
transaction_count	32	Output	Total transaction count (live – reporter lifetime counter, zero-extended)
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock gating active (1=gated, 0=running)
cg_idle	1	Output	Interface idle (1=no activity)

11.4 Functional Description

11.4.1 Combined Write Monitoring and Power Management

This module stacks both jobs on the write path:

Monitoring (from axi5_master_wr_mon): - Real-time write transaction monitoring (AW/W/B) - Error and timeout detection - Performance metrics collection - Atomic operation tracking - Configurable packet filtering - MonBus output for system integration

Power Management (from axi5_master_wr_cg): - Automatic clock gating during write idle periods - Configurable idle threshold - Activity-based wake-up - Status outputs for power monitoring

11.4.2 Clock Gating with Write Monitor Active

The clock gating logic considers all three write channels plus monitor activity:

```
user_valid = (ENABLE_WAKEUP && fub_axi_awakeup) || fub_axi_awvalid ||
fub_axi_wvalid || fub_axi_bvalid || int_busy ||
             w_monbus_valid || (|active_transactions);  // peer VALID,
never peer READY
axi_valid = m_axi_awvalid || m_axi_wvalid || m_axi_bvalid;

// Clock remains active if:
// - AW channel active (address phase)
// - W channel active (data phase)
// - B channel active (response phase)
// - Internal busy (transactions in flight)
// - A monitor packet is pending on the monitor bus (w_monbus_valid)
// - The monitor CAM still holds entries (|active_transactions) --
covers
//   the reporter's retire -> FIFO -> output emission window
```

The monitor continues operating during clock gating transitions, ensuring no write events are lost. A packet pending on the monitor bus holds the block awake until the consumer accepts it, and the external `monbus_valid` is masked with `!cg_gating`, so monitor-bus delivery is exactly-once across gating at any idle count (asserted by `val/amba/test_mon_cg_gating.py` phase 6). The monitor CAM's occupancy (`active_transactions`) is also a wake term: a tracked entry stays in the CAM until its packet is marked into the reporter FIFO, so the clock cannot stop inside the reporter's few-cycle emission window and strand a trailing packet, even at `cfg_cg_idle_count = 0`.

11.4.3 Ready Signal Control During Gating

When clock gating is active, all write channel ready signals are forced low:

```
fub_axi_awready = cg_gating ? 1'b0 : int_awready;
fub_axi_wready = cg_gating ? 1'b0 : int_wready;
m_axi_bready = cg_gating ? 1'b0 : int_bready;
```

This ensures protocol compliance during power management across all write phases.

11.4.4 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_master_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi5_master_wr_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AW (write address) handshakes. **WARNING:** an open measurement window is NOT a wake term – the window FSM and all counters run on the gated clock, so if the bus idles past `cfg_cg_idle_count` mid-window the counters FREEZE while wall-clock time passes, and trigger pulses arriving while gated are DROPPED. For exact wall-clock windows hold `cfg_cg_enable` low around the measurement, or use the base module.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see Parameters), straight through to the base monitor.

11.5 Timing Characteristics

11.5.1 Clock Gating with Write Burst and Monitoring

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK (gated clock)
- AW channel: AWVALID, AWREADY
- W channel: WDATA, WLAST, WVALID, WREADY
- B channel: BVALID, BREADY
- Monitor packets generated
- Idle period detection after B phase
- cg_gating activation
- Monitor quiescent before clock stops

11.5.2 Atomic Operation with Monitor + Clock Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- AWATOP encoding
- Atomic write sequence
- Monitor packet for atomic operation
- B response with BTAGMATCH
- Clock gating after atomic completion

11.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_master_wr_mon_cg #(
    .SKID_DEPTH_AW          (2),
    .SKID_DEPTH_W           (4),
    .SKID_DEPTH_B           (2),
    .AXI_ID_WIDTH           (8),
    .AXI_ADDR_WIDTH         (32),
    .AXI_DATA_WIDTH         (32),
    .AXI_USER_WIDTH         (1),
    .AXI_ATOP_WIDTH         (6),
    .AXI_NSAID_WIDTH        (4),
    .AXI_MPAM_WIDTH         (11)
) u_axi5_master_wr_mon_cg (
    .aclk                   (aclk),
    .aresetn                (aresetn),
    .cam_clear              (cam_clear),
    .cfg_cg_enable          (cfg_cg_enable),
    .cfg_cg_idle_count      (cfg_cg_idle_count),
    .fub_axi_awid           (fub_axi_awid),
    .fub_axi_awaddr         (fub_axi_awaddr),
    .fub_axi_awlen          (fub_axi_awlen),
    .fub_axi_awsize         (fub_axi_awsize),
    .fub_axi_awburst        (fub_axi_awburst),
    .fub_axi_awlock         (fub_axi_awlock),
    .fub_axi_awcache        (fub_axi_awcache),
    .fub_axi_awprot         (fub_axi_awprot),
    .fub_axi_awqos          (fub_axi_awqos),
    .fub_axi_awuser         (fub_axi_awuser),
    .fub_axi_awvalid        (fub_axi_awvalid),
    .fub_axi_awakeup        (fub_axi_awakeup),
    .fub_axi_awready        (fub_axi_awready),
    .fub_axi_awatop         (fub_axi_awatop),
    .fub_axi_awnsaid        (fub_axi_awnsaid),
    .fub_axi_awtrace        (fub_axi_awtrace),
    .fub_axi_awmpam         (fub_axi_awmpam),
    .fub_axi_awmecid        (fub_axi_awmecid),
    .fub_axi_awidunq       (fub_axi_awidunq),
    .fub_axi_awtagop        (fub_axi_awtagop),
    .fub_axi_wdata          (fub_axi_wdata),
    .fub_axi_wstrb          (fub_axi_wstrb),
    .fub_axi_wlast          (fub_axi_wlast),
    .fub_axi_wuser          (fub_axi_wuser),
    .fub_axi_wvalid         (fub_axi_wvalid),
    .fub_axi_wready         (fub_axi_wready),
    .fub_axi_wpoison        (fub_axi_wpoison),
```

```
    .fub_axi_wtag          (fub_axi_wtag),
    .fub_axi_wtagupdate    (fub_axi_wtagupdate),
    .fub_axi_bid           (fub_axi_bid),
    .fub_axi_bresp         (fub_axi_bresp),
    .fub_axi_buser         (fub_axi_buser),
    .fub_axi_bvalid        (fub_axi_bvalid),
    .fub_axi_bready        (fub_axi_bready),
    .fub_axi_btrace        (fub_axi_btrace),
    .fub_axi_bidunq        (fub_axi_bidunq),
    .fub_axi_btagmatch     (fub_axi_btagmatch),
    .m_axi_awid            (m_axi_awid),
    .m_axi_awaddr          (m_axi_awaddr),
    .m_axi_awlen           (m_axi_awlen),
    .m_axi_awsize          (m_axi_awsize),
    .m_axi_awburst         (m_axi_awburst),
    .m_axi_awlock          (m_axi_awlock),
    .m_axi_awcache         (m_axi_awcache),
    .m_axi_awprot          (m_axi_awprot),
    .m_axi_awqos           (m_axi_awqos),
    .m_axi_awuser          (m_axi_awuser),
    .m_axi_awvalid         (m_axi_awvalid),
    .m_axi_awakeup         (m_axi_awakeup),
    .m_axi_awready         (m_axi_awready),
    .m_axi_awatop          (m_axi_awatop),
    .m_axi_awnsaid         (m_axi_awnsaid),
    .m_axi_awtrace         (m_axi_awtrace),
    .m_axi_awmpam          (m_axi_awmpam),
    .m_axi_awmecid         (m_axi_awmecid),
    .m_axi_awidunq        (m_axi_awidunq),
    .m_axi_awtagop         (m_axi_awtagop),
    .m_axi_wdata           (m_axi_wdata),
    .m_axi_wstrb           (m_axi_wstrb),
    .m_axi_wlast           (m_axi_wlast),
    .m_axi_wuser           (m_axi_wuser),
    .m_axi_wvalid          (m_axi_wvalid),
    .m_axi_wready          (m_axi_wready),
    .m_axi_wpoison         (m_axi_wpoison),
    .m_axi_wtag            (m_axi_wtag),
    .m_axi_wtagupdate      (m_axi_wtagupdate),
    .m_axi_bid             (m_axi_bid),
    .m_axi_bresp           (m_axi_bresp),
    .m_axi_buser           (m_axi_buser),
    .m_axi_bvalid          (m_axi_bvalid),
    .m_axi_bready          (m_axi_bready),
    .m_axi_btrace          (m_axi_btrace),
    .m_axi_bidunq          (m_axi_bidunq),
    .m_axi_btagmatch       (m_axi_btagmatch),
    .cfg_monitor_enable    (cfg_monitor_enable),
```

```
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .cfg_conflict_error    (cfg_conflict_error),
    .cg_gating             (cg_gating),
    .cg_idle               (cg_idle),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
```

```
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

11.7 Design Notes

11.7.1 When to Use This Module

Ideal for: - Battery-powered systems with write-intensive workloads - Systems with bursty write traffic and significant idle periods - Designs requiring comprehensive write path visibility - Production systems needing runtime monitoring + power optimization - Write-heavy cache implementations - DMA engines with intermittent activity

Consider alternatives if: - Continuous streaming writes (clock gating ineffective) - Ultra-low write latency requirements (avoid gating overhead) - Area-constrained designs (monitor + CG adds ~12% area) (first-order estimate – no power/area analysis has been run) - Simple write-through caches (simpler monitoring sufficient)

11.7.2 Write-Specific Configuration Strategy

Write Path Characteristics:

Aspect	Typical Value	Impact on Configuration
Write latency	10-100 cycles	Set higher cfg_latency_threshold
Burst length	4-16 beats	Longer monitoring windows
Response delay	5-50 cycles	Longer idle count before gating
Atomic operations	20-200 cycles	Extended timeout thresholds

Recommended Configuration:

```
// For write path (vs. read path adjustments)
.cfg_cg_idle_count      (4'd5),    // Longer than read (B latency)
.cfg_timeout_cycles     (16'd15),   // 15 microseconds per phase (full
16-bit range)
.cfg_latency_threshold  (32'd800)   // Higher variance than reads
```

11.7.3 Monitor + Clock Gating Interaction for Writes

Key Write-Specific Considerations:

1. **Three-channel coordination:**
   - Monitor tracks AW, W, B phases independently

- All three must be idle before gating
- Longer idle detection time than read (2 channels)

2. **Write buffering impact:**
   - Posted writes may delay B responses
   - Clock gating may activate between W and B
   - Monitor ensures B responses still tracked after wake
3. **Atomic operation handling:**
   - Atomic ops have read+write phases
   - Longer latency prevents premature gating
   - Monitor tracks full atomic sequence
4. **Power savings accounting:**
   - Writes typically 60-70% traffic vs. reads (system dependent)
   - Write-specific gating saves 15-30% total power
   - Combined with read path CG: 40-60% total savings

11.7.4 Performance Analysis with Write Monitoring

The monitor provides valuable write path metrics:

```
// Extract write performance from monitor packets
always_comb begin
    write_bandwidth = (write_count * data_width) / elapsed_cycles;
    write_efficiency = (active_write_cycles) / total_cycles;
    avg_outstanding = total_outstanding_samples / sample_count;
end

// Compare with/without clock gating
$display("Write Perf: BW=%d GB/s, Efficiency=%d%%, Power saved=%d%%",
    write_bandwidth, write_efficiency,
    (power_cycles_saved * 100) / total_cycles);
```

11.8 Related Modules

- **AXI5 Master Write** - Base module
- **AXI5 Master Write CG** - Clock gating only
- **AXI5 Master Write Monitor** - Monitor only
- **AXI5 Master Read Monitor CG** - Read variant
- **AXI Monitor Configuration Guide** - Monitor setup
- **AMBA Clock Gate Controller** - Clock gating details

11.9 Testing

11.9.1 Verification Recommendations for Write Path

1. **Test write burst + gating:**

```
// Phase 1: Short bursts with gating between
for (int i = 0; i < 100; i++) begin
    write_burst(addr, len=4);
    wait_idle_cycles(10);  // Allow gating
    check_monitor_packets();
end
```

2. **Test atomic operations:**

```
// Verify atomic ops don't get interrupted by gating
atomic_compare_swap(addr, old_val, new_val);
check_no_premature_gating();
verify_atomic_completion_monitored();
```

3. **Test write errors with gating:**

```
// Trigger SLVERR, verify monitored correctly
write_to_protected_address();
wait_error_packet();
check_gating_after_error_handled();
```

11.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- The monitor’s own techniques are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

11.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

12 AXI5 Slave Read

Module: `axi5_slave_rd.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

12.1 Overview

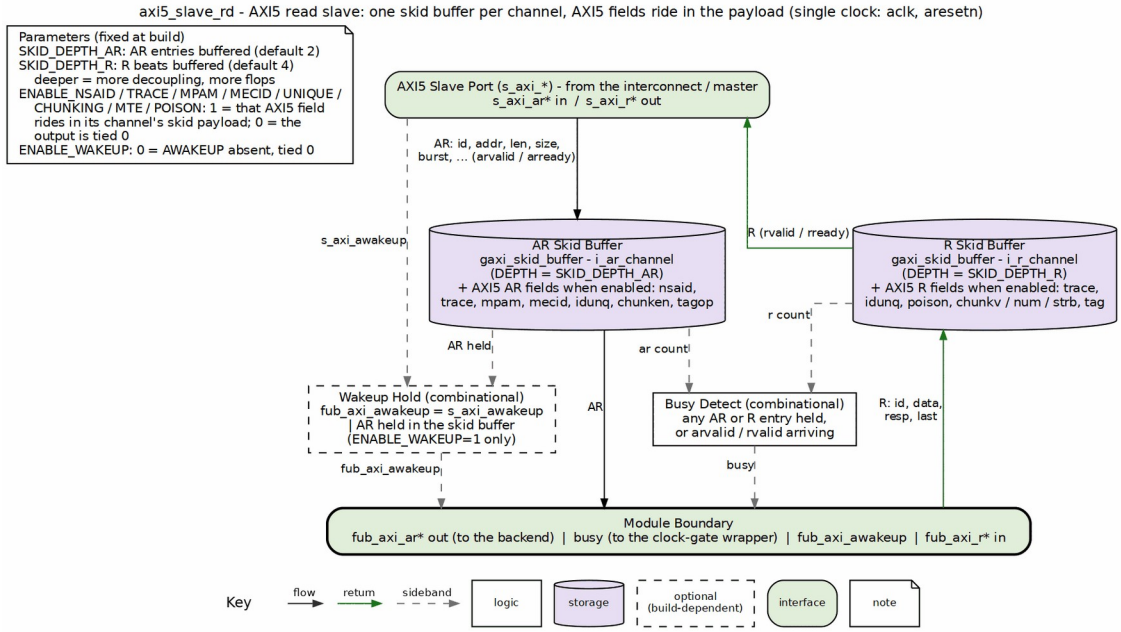
The AXI5 Slave Read module is the slave-side AR/R channel transport block. It receives read requests from an external AXI5 master and forwards them, with the full AXI5 sideband signal set intact, to backend memory or logic on the FUB (Functional Unit Block) interface.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no outstanding-transaction tracking. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

12.1.1 Key Features

- Carries the full AXI5 signal set listed below, unmodified, across the SKID buffers
- **ARNSAID:** Non-secure access identifier reception
- **ARTRACE:** Trace signal support
- **ARMPAM:** Memory Partitioning and Monitoring reception
- **ARMECID:** Memory Encryption Context ID reception
- **ARIDUNQ / RIDUNQ:** Unique ID indicator on the request and on every read data beat (IHI 0022L §A5.2)
- **ARCHUNKEN:** Chunking enable reception
- **ARTAGOP:** Memory tag operation reception (MTE)
- **RTRACE/RPOISON:** Response extensions
- **RCHUNKV/RCHUNKNUM/RCHUNKSTRB:** Chunking support
- **RTAG:** Memory tags returned with read data (MTE)
- SKID buffering for AR and R channels
- Busy signal for power management

12.1.2 Module Architecture



axi5_slave_rd block diagram

The slave port (`s_axi_*`) is at the top and the module boundary is the bar. AR goes down the left lane through its skid buffer and R comes back up the right lane through its own. The AXI5 fields each channel carries (NSAID, trace, MPAM, MECID, unique-ID, MTE tags, poison, chunking) ride in that channel's skid payload when their `ENABLE_*` parameter is set; a disabled field's output is tied to 0. The busy detect feeds the clock-gate wrapper, and the wakeup hold keeps `fub_axi_awakeup` asserted while an accepted request is still waiting in a skid buffer, not only while `s_axi_awakeup` is high (IHI 0022L A14.1.1).

12.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_R	int	4	R channel SKID buffer depth (legal range 2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)

Parameter	Type	Default	Description
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low

12.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `rtag` field
CHUNK_STRB_WIDTH	max(AXI_DATA_WIDTH / 128, 1)	Width of `rchunkstrb`, one bit per 128-bit granule
ARSize	Sum of the enabled AR fields	AR SKID buffer payload width
RSize	Sum of the enabled R fields	R SKID buffer payload width

Note on AXI_WSTRB_WIDTH: this module declares `AXI_WSTRB_WIDTH` (default `AXI_DATA_WIDTH/8`) and its short alias `SW`, but neither is used. A read module has no W channel and therefore no write strobes. The parameter is retained only so the read and write module parameter lists line up; overriding it has no effect.

12.3 Ports

12.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

12.3.2 Slave AXI5 Interface (External Master Input)

12.3.2.1 AR Channel (Input)

Port	Width	Direction	Description
s_axi_arid	IW	Input	Read address ID
s_axi_araddr	AW	Input	Read address
s_axi_arlen	8	Input	Burst length
s_axi_arsize	3	Input	Burst size
s_axi_arburst	2	Input	Burst type
s_axi_arlock	1	Input	Lock type
s_axi_arcache	4	Input	Cache attributes
s_axi_arprot	3	Input	Protection attributes
s_axi_arqos	4	Input	Quality of Service
s_axi_aruser	UW	Input	User-defined signal
s_axi_arvalid	1	Input	Read address valid
s_axi_awakeup	1	Input	Wake-up request, AWAKEUP (IHI 0022L §A14.1.1; `ENABLE_WAKEUP`)

Port	Width	Direction	Description
s_axi_arready	1	Output	Read address ready
s_axi_arnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID (AXI5)
s_axi_artrace	1	Input	Trace signal (AXI5)
s_axi_armpam	AXI_MPAM_WIDTH	Input	Memory partitioning (AXI5)
s_axi_armecid	AXI_MECID_WIDTH	Input	Memory encryption context (AXI5)
s_axi_ariduniq	1	Input	Unique ID indicator, ARIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
s_axi_archunken	1	Input	Chunking enable (AXI5)
s_axi_artagop	AXI_TAGOP_WIDTH	Input	Tag operation (AXI5 MTE)

12.3.2.2 R Channel (Output)

Port	Width	Direction	Description
s_axi_rid	IW	Output	Read data ID
s_axi_rdata	DW	Output	Read data
s_axi_rresp	2	Output	Read response
s_axi_rlast	1	Output	Last transfer in burst
s_axi_ruser	UW	Output	User-defined signal
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready
s_axi_rtrace	1	Output	Trace signal (AXI5)
s_axi_riduniq	1	Output	Unique ID indicator on every read data beat,

Port	Width	Direction	Description
			RIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
s_axi_rpoison	1	Output	Data poison indicator (AXI5)
s_axi_rchunkv	1	Output	Chunk valid (AXI5)
s_axi_rchunknum	AXI_CHUNKNUM_WIDTH	Output	Chunk number (AXI5)
s_axi_rchunkstrb	CHUNK_STRB_WIDTH	Output	Chunk strobe (AXI5)
s_axi_rtag	TW	Output	Memory tags (AXI5 MTE)

12.3.3 FUB Interface (Backend Memory/Logic Output)

Same signals as slave interface but with `fub_axi_*` prefix and reversed directions for AR (output) and R (input).

12.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator

12.4 Functional Description

This slave module acts as an adapter between: - **External AXI5 master** (s_axi_* interface) - **Backend memory or functional unit block** (fub_axi_* interface)

Signal Flow: 1. AR channel: External master → SKID → Backend 2. R channel: Backend → SKID → External master

SKID Buffering: Provides elasticity and breaks combinational paths for timing closure.

12.5 Timing Characteristics

12.5.1 Basic Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- S_AXI_ARID, S_AXI_ARADDR, S_AXI_ARVALID, S_AXI_ARREADY
- FUB_AXI_ARVALID, FUB_AXI_ARREADY (request forwarded to the backend)
- FUB_AXI_RDATA, FUB_AXI_RVALID, FUB_AXI_RREADY
- S_AXI_RDATA, S_AXI_RLAST, S_AXI_RVALID, S_AXI_RREADY

12.6 Usage Examples

```
axi5_slave_rd #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_rd (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Slave interface (from external master)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    .s_axi_arlen        (s_axi_arlen),
    // Every remaining s_axi_ar* port mirrors the fub_axi_ar* list.
All must be connected.
    .s_axi_arnsaid      (s_axi_arnsaid),
    .s_axi_artrace      (s_axi_artrace),
    // ... (connect AXI5 extensions)

    .s_axi_rid          (s_axi_rid),
    .s_axi_rdata        (s_axi_rdata),
    // Every remaining s_axi_r* port mirrors the fub_axi_r* list. All
must be connected.

    // FUB interface (to backend)
    .fub_axi_arid       (mem_arid),
    .fub_axi_araddr     (mem_araddr),
    // ... (connect to memory controller)

    .busy               (slave_rd_busy)
);
```

12.7 Design Notes

- Use when you need AXI5 slave functionality with SKID buffering
- Backend FUB interface can connect to memory controller, SRAM, or functional units
- SKID buffers improve timing by breaking long combinational paths
- Feature enables allow area optimization by disabling unused AXI5 extensions

12.8 Related Modules

- **AXI5 Slave Write** - Slave write interface
- **AXI5 Master Read** - Master read interface
- **AXI5 Slave Read CG** - Clock-gated variant
- **AXI5 Slave Read Monitor** - With integrated monitoring

12.9 Testing

`val/amba/test_axi5_slave_rd.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_rd.py -v
```

12.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A5.6.2 “Read data chunking” (ARCHUNKEN, RCHUNKV/NUM/STRB), §A16.1 “Data protection using Poison” (RPOISON), §A12.2 “Memory Tagging Extension (MTE)” (ARTAGOP, RTAG), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (ARMECID). The ports follow the specification’s names (GitHub #102): there is no RTAGMATCH, the Unique ID indicator is ARIDUNQ with RIDUNQ on every read data beat, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h – the same AMBA 5 features as first numbered: ch. C1 “AMBA AXI5”, §E1.11 “Read data chunking”, §E1.15 “Memory tagging”, §E2.1 “Poison”, §E1.13 “Unique ID indicator”. MECID is not in Issue H.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: a channel can be pipelined by any number of back-pressured stages without changing its behavior, which is why a skid buffer per channel is transparent to both ends.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC ’06)*, July 2006, pp. 657-662. DOI: 10.1145/1146909.1147077 – the elastic buffer with valid/stop back-pressure, the structure a skid buffer implements.
- Chipressian. *Crack the Hardware Interview: from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (forward and backward register slices) – textbook treatment of the valid/ready slice (secondary source).
- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

12.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

13 AXI5 Slave Read with Clock Gating

Module: `axi5_slave_rd_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

13.1 Overview

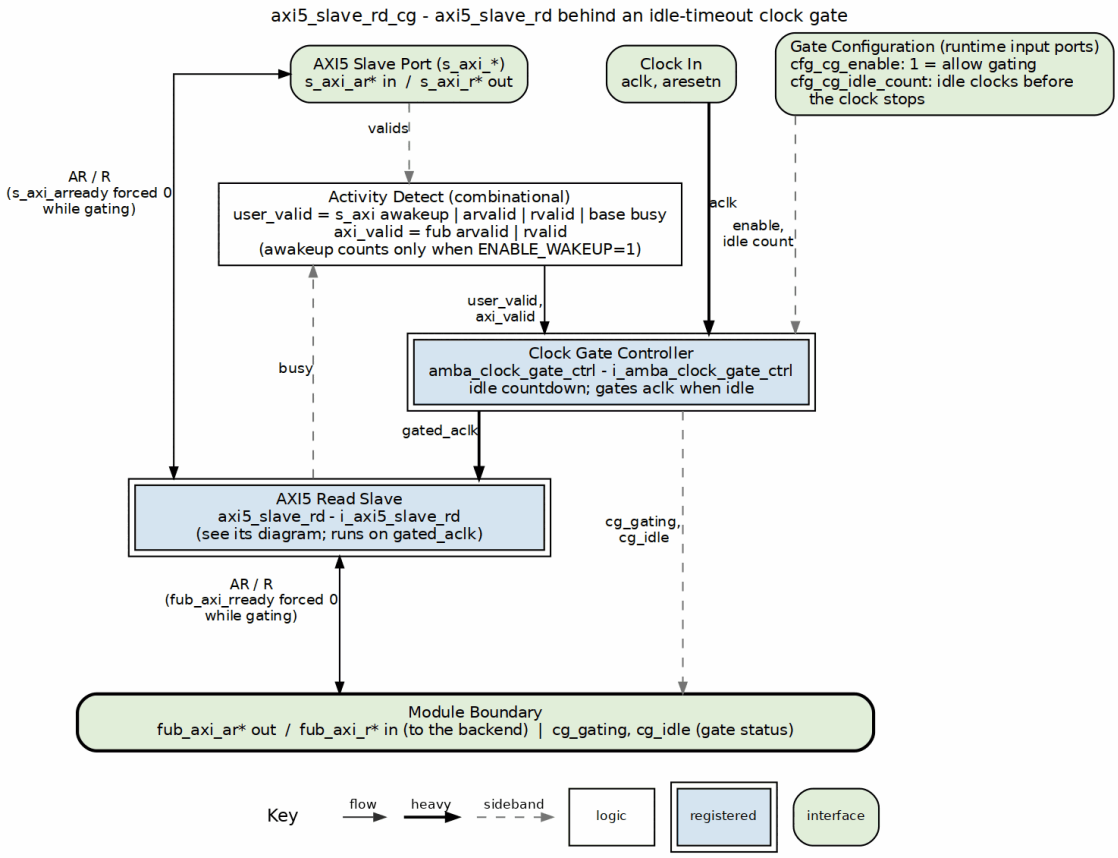
The AXI5 Slave Read with Clock Gating module wraps the standard `axi5_slave_rd` module with integrated clock gating for power optimization. When the module sits idle, the internal clock gates itself off.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no outstanding-transaction tracking. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

13.1.1 Key Features

- Carries the full AXI5 read signal set listed below, unmodified, across the SKID buffers
- **Integrated clock gating** for dynamic power reduction
- Configurable idle count before gating
- All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, IDUNQ, CHUNKING, MTE, POISON, WAKEUP)
- Transparent gating - no protocol changes
- Gating status outputs for system monitoring
- SKID buffering for AR and R channels
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

13.1.2 Module Architecture



axi5_slave_rd_cg block diagram

The base `axi5_slave_rd` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `s_axi` valids, the child's busy, and `s_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `fub_axi` valids.

13.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging

Parameter	Type	Default	Description
			Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
CG_IDLE_COUNT_WIDTH	int	4	Clock gating idle counter width

13.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `rtag` field
CHUNK_STRB_WIDTH	max(AXI_DATA_WIDTH / 128, 1)	Width of `rchunkstrb`, one bit per 128-bit granule
ARSize	Sum of the enabled AR fields	AR SKID buffer payload width
RSize	Sum of the enabled R fields	R SKID buffer payload width

Note on AXI_WSTRB_WIDTH: this module declares `AXI_WSTRB_WIDTH` (default `AXI_DATA_WIDTH/8`) and its short alias `SW`, but neither is used. A read module has no W channel and therefore no write strobes. The parameter is retained only so the read and write module parameter lists line up; overriding it has no effect.

13.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AW	AXI_ADDR_WIDTH

Derived parameter	Default expression
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
UW	AXI_USER_WIDTH

13.3 Ports

13.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

13.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

13.3.3 Slave AXI5 Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

13.3.4 FUB Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

13.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

13.4 Functional Description

13.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (AWAKEUP, arvalid, rvalid, or internal busy – peer VALID, post-fix) - **axi_valid:** Asserted when FUB interface has activity (arvalid, rvalid)

Gating State Machine:

```
stateDiagram-v2
    [*] --> ACTIVE

    ACTIVE --> COUNTING : !user_valid && !axi_valid
    COUNTING --> ACTIVE : user_valid || axi_valid
    COUNTING --> GATED : count == cfg_cg_idle_count
    GATED --> ACTIVE : user_valid || axi_valid

    state ACTIVE {
        note right of ACTIVE : Clock enabled<br/>cg_gating = 0
    }
    state COUNTING {
        note right of COUNTING : Idle countdown<br/>cg_gating = 0
    }
    state GATED {
        note right of GATED : Clock gated<br/>cg_gating = 1
    }
```

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals forced to 0 when gated (prevents new transactions) - Gating only occurs after configured idle period - Any activity immediately ungates the clock

13.4.2 Ready Deassertion and Wake Behavior

While `cg_gating` is asserted, the ready outputs listed above are forced low.

This is legal AXI. A READY output may be deasserted for any number of cycles, and neither side may assume READY asserts within a bounded time, so holding it low is ordinary backpressure rather than a protocol violation. It cannot deadlock either: the same VALID being held off also feeds the activity detector, so gating releases and READY returns to the core's value.

Two consequences are worth designing around:

- **Wake latency adds to first-transfer latency.** Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG

enable, which is combinational. The AXI5 `_cg` wrappers drive the activity terms combinationally, so there is 1 register stage and the first usable gated-clock edge arrives 2 cycles after activity asserts. Size `cfg_cg_idle_count` so that cost is amortized over the traffic burst that follows.

- **READY is a combinational function of `cg_gating`.** The path from the idle counter to the `s_axi_arready` output may need attention during timing closure, particularly at high frequency.

13.4.3 Integration Caveat: `s_axi_rready` in the Activity Term

The activity detector uses the corresponding VALID (`s_axi_rvalid`), never the peer's READY. (Historically this term was `s_axi_rready`, which meant a peer parking its ready high – a common and entirely correct style – pinned the block permanently awake and defeated gating; that defect was fixed family-wide, matching the rule the mon_cg wrappers always carried.)

An always-ready consumer no longer prevents gating.

13.4.4 Clock Gating Configuration

13.4.4.1 Idle Count Selection

cfg_cg_idle_count	Idle Cycles	Use Case
0	1	Aggressive power saving, frequent gating
1-3	2-4	Balanced, reduces gate churn
4-7	5-8	Conservative, for bursty traffic
8-15	9-16	Minimal gating (the 4-bit field maxes at 16 cycles – the count is LITERAL, gating at count+1, not a power of two)

Recommendations: - **Streaming workloads:** Higher count (4-8) to avoid frequent gate/ungate - **Sporadic transactions:** Lower count (0-2) for maximum power savings - **Real-time systems:** Disable gating or use high count to ensure deterministic latency

---

13.5 Timing Characteristics

13.5.1 Clock Gating Sequence

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK
- user_valid, axi_valid activity terms
- Idle counter progression
- cg_idle assertion
- cg_gating activation

13.6 Usage Examples

```
axi5_slave_rd_cg #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .CG_IDLE_COUNT_WIDTH(4),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_CHUNKING    (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_rd_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Clock gating config
    .cfg_cg_enable      (1'b1),          // Enable gating
    .cfg_cg_idle_count  (4'd3),          // Gate after 4 idle cycles
(count+1; a LITERAL count, not a power of two)

    // Slave interface (from external master)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    // Every remaining s_axi_ar*/s_axi_r* port mirrors the fub_axi_*
list,
    // same names and widths, opposite directions. All must be
connected.

    // FUB interface (to backend)
    .fub_axi_arid       (mem_arid),
    .fub_axi_araddr     (mem_araddr),
    // ... (connect to memory controller)

    // Clock gating status
    .cg_gating          (slave_rd_gating),
    .cg_idle            (slave_rd_idle)
);

// Optional: Monitor power savings
always @(posedge axi_clk) begin
    if (slave_rd_gating)
```

```
        $display("AXI5 Slave RD clock gated - saving power");
end
```

13.7 Design Notes

13.7.1 Power Savings Estimation

Worked example, assuming the module is active 60% of the time: - **Without gating:** 100% of this module's dynamic power - **With gating (idle_count=2):** roughly 50%, allowing for wake/sleep transitions - **With gating (idle_count=0):** roughly 60%; more aggressive gating, but more transition overhead

All power figures on this page are first-order estimates derived from duty cycle, not measured results. No power analysis (gate-level or otherwise) has been run against these modules. Actual savings depend on the gated logic's share of total design power, the technology library's clock-gate cell, and leakage, which clock gating does not reduce at all. Treat these numbers as a sizing aid and characterize your own instance before quoting a savings figure.

13.7.2 Clock Gating Overhead

- **Area:** +5 flops, counted from the RTL – `r_wakeup` in `amba_clock_gate_ctrl` plus `r_idle_counter` at `IDLE_CNTR_WIDTH` (default 4), scaling as 1 + `CG_IDLE_COUNT_WIDTH`. Against a base module of a few hundred flops that is low single-digit percent. The ICG itself is a latch/BUFGCE, not fabric flops; the LUT delta is not measured
- **Timing:** Clock gating adds minimal delay (typically <50ps)
- **Power:** Overhead from gate control logic usually <1% of savings

13.7.3 When to Use Clock Gating

Good candidates: - Low-duty-cycle interfaces (sporadic transactions) - Systems with strict power budgets - Battery-operated devices

Avoid when: - Interface is continuously active - Gate/ungate transitions exceed power savings - Deterministic latency required (use high idle count instead)

13.7.4 Integration with Power Management

```
// System power controller
assign axi_power_save_mode = slave_rd_gating &&
                             slave_wr_gating &&
                             master_rd_gating;

// Optional: Track gating efficiency
always @(posedge axi_clk or negedge aresetn) begin
    if (!aresetn) begin
        gating_cycles <= 0;
        total_cycles <= 0;
    end else begin
        total_cycles <= total_cycles + 1;
```

```
        if (slave_rd_gating)
            gating_cycles <= gating_cycles + 1;
    end
end

assign gating_efficiency = (gating_cycles * 100) / total_cycles;
```

13.8 Related Modules

- **AXI5 Slave Read** - Non-clock-gated version
- **AXI5 Slave Write CG** - Clock-gated write variant
- **AXI5 Slave Read Monitor CG** - With monitoring
- **AMBA Clock Gate Control** - Clock gating controller

13.9 Testing

`val/amba/test_axi5_slave_rd_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_rd_cg.py -v
```

13.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A5.6.2 “Read data chunking” (ARCHUNKEN, RCHUNKV/NUM/STRB), §A16.1 “Data protection using Poison” (RPOISON), §A12.2 “Memory Tagging Extension (MTE)” (ARTAGOP, RTAG), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (ARMECID). The ports follow the specification’s names (GitHub #102): there is no RTAGMATCH, the Unique ID indicator is ARIDUNQ with RIDUNQ on every read data beat, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

13.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

14 AXI5 Slave Read Monitor

Module: `axi5_slave_rd_mon.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

14.1 Overview

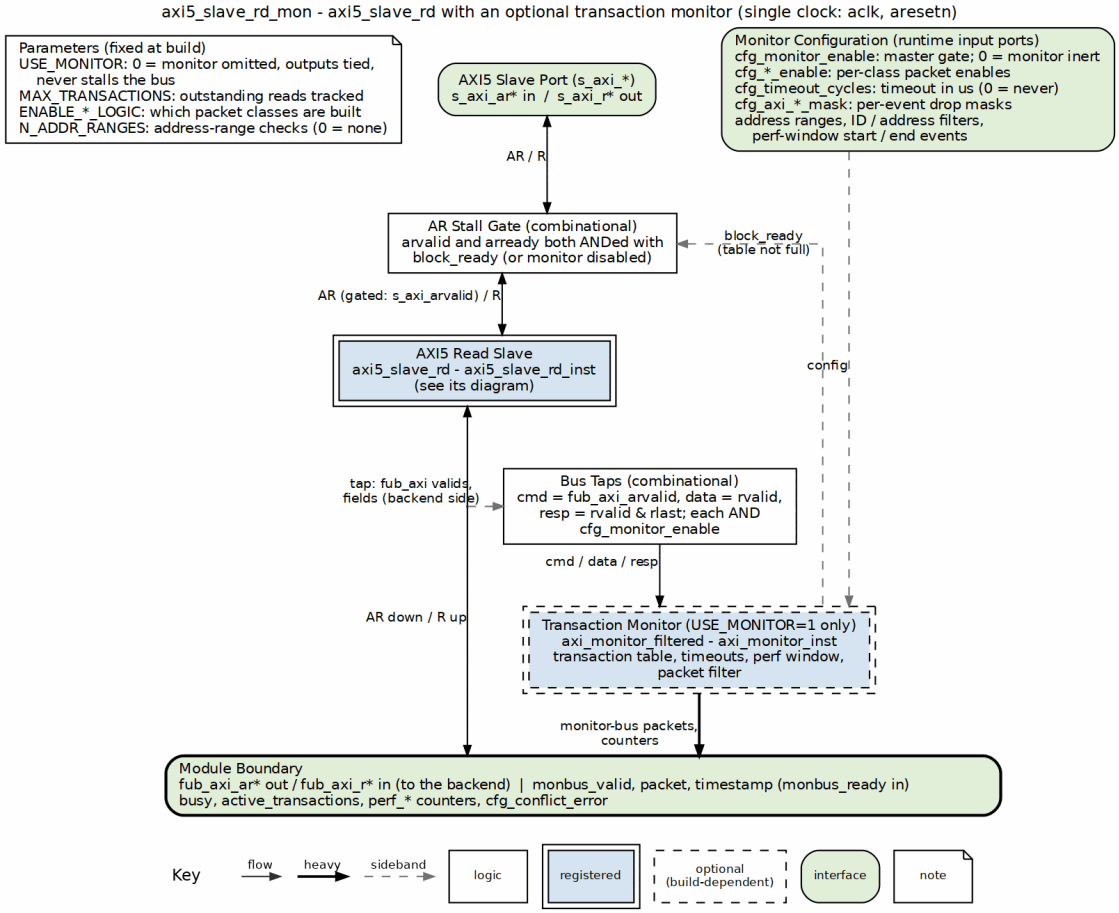
The AXI5 Slave Read Monitor module combines the `axi5_slave_rd` interface with integrated transaction monitoring. You get real-time visibility into slave read operations, with configurable packet filtering and error detection built in rather than bolted on.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

14.1.1 Key Features

- Full AMBA AXI5 slave read protocol compliance
- **Integrated filtered monitoring** - no external monitor needed
- All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, IDUNQ, CHUNKING, MTE, POISON, WAKEUP)
- Transaction tracking with configurable table size
- Error detection (SLVERR, timeout, orphan transactions)
- Performance metrics (latency, throughput)
- Configurable packet filtering to reduce bandwidth
- 128-bit monitor bus packet output paired with 64-bit side-band timestamp
- Active transaction count tracking

14.1.2 Module Architecture



axi5_slave_rd_mon block diagram

The base `axi5_slave_rd` is one box (see its own diagram). Traffic runs down the left through the AR stall gate, which ANDs `s_axi_arvalid` and its ready with the `block_ready` the monitor raises while its transaction table has room. The bus taps watch the `fub_axi` (backend) side below the base, unlike the AXI4 slave monitors, which tap `s_axi` and feed `axi_monitor_filtered`, which is built only when `USE_MONITOR=1`; its monitor-bus packets are the heavy edge out. With `cfg_monitor_enable` low the monitor is inert and never stalls the bus.

14.2 Parameters

A lite-monitor version of this wrapper exists as `axi5_slave_rd_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID

Parameter	Type	Default	Description
			indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	12	Agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. **>1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
ACLK_MHZ	int	100	Clock frequency in MHz – keeps the 1 us tick

Parameter	Type	Default	Description
			exact off-100MHz
CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when cfg_threshold_enable=1. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared allowlist checker: a debug-range hit -> AddrMatch, an error-

Parameter	Type	Default	Description
			allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the `axi_monitor_timeout` instance.
ENABLE_COMPL_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six `ENABLE_*_LOGIC` parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's `axi_monitor_filtered` instance the perf/debug master switches are fixed (`ENABLE_PERF_PACKETS = 1`, `ENABLE_DEBUG_MODULE = 0`). The former `CAM_PIPELINE` / `TRANS_CAM_PIPELINE` parameters were removed — the transaction CAM is now always pipelined.

14.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS
CHUNK_STRB_WIDTH	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1

14.3 Ports

14.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

14.3.2 Slave AXI5 Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

14.3.3 FUB Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

14.3.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Master runtime gate. 0 = monitor inert: no allocation, transaction CAM held clear, and the upstream ready is never stalled (a disabled monitor cannot block the datapath). 1 = normal operation
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packets
cfg_timeout_enable	1	Input	Enable timeout packets
cfg_perf_e	1	Input	Enable performance

Port	Width	Direction	Description
nable			packets (see Performance Monitoring)
cfg_compl_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Unified coarse timeout control, a MICROSECOND count passed through at FULL 16-bit width: 1..65535 us per phase; 0 = 16'hFFFF (~65 ms, effectively never). One value drives all three phase counts (addr/data/resp). The old 4-bit squash that saturated >= 16 at 15 us is retired
cfg_latency_threshold	32	Input	High latency threshold (cycles)
cfg_freq_sel	4	Input	`counter_freq_invariant` LUT index scaling the 1 us timer tick
cfg_axi_pkt_mask	16	Input	Packet type filter mask
cfg_axi_err_select	16	Input	Error selection mask
cfg_axi_error_mask	16	Input	Error event filter
cfg_axi_timeout_mask	16	Input	Timeout event filter

Port	Width	Direction	Description
cfg_axi_compl_mask	16	Input	Completion event filter
cfg_axi_thresh_mask	16	Input	Threshold event filter
cfg_axi_perf_mask	16	Input	Performance event filter
cfg_axi_addr_mask	16	Input	Address event filter
cfg_axi_debug_mask	16	Input	Debug event filter

Detection-cone enables: `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4) and `cfg_debug_mask` (16) inputs are tied to `4'h0` / `16'h0`; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`). None are wrapper ports.

14.3.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	`monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_block_ready	1	Output	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)

Port	Width	Direction	Description
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

14.3.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
transaction_count	32	Output	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
cfg_conflict_error	1	Output	Configuration conflict detected

14.3.7 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
cfg_addr_filter_enable	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
cfg_addr_filter_low	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	ADDR_WIDTH	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (N_ADDR_RANGES), which re-evaluates per command.

Runtime ID filter — overrides the ID_MATCH_BASE/ID_MATCH_COUNT elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
cfg_id_filter_enable	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
cfg_id_match_base	ID_WIDTH	First ID owned
cfg_id_match_count	ID_WIDTH+1	How many; 0 means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: [base, base + count).

14.4 Functional Description

14.4.1 Monitor Packet Format

The 128-bit `monbus_packet` (paired with the 64-bit `monbus_timestamp` side-band signal) follows the standardized AMBA monitor bus format:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, DECERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished)
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits — AXI ID or channel index)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits — full address, latency, etc.)
```

14.4.2 Event Types

14.4.2.1 Error Packets (Type=0)

Event Code	Description	Event Data
0x1	SLVERR response	Zero-extended 32-bit ADDRESS (the reporter cones carry address only)
0x2	DECERR response	Zero-extended 32-bit ADDRESS
0x3	Orphan data/response	Zero-extended 32-bit ADDRESS
0x4	Protocol violation	Zero-extended 32-bit ADDRESS – **write monitors only**, see note

(Event code 0x4 is part of the shared monitor encoding but cannot be emitted by a read monitor: `axi_monitor_trans_mgr` sets `EVT_PROTOCOL` only inside its `if (!IS_READ && resp_valid && resp_ready)` block, which a read instance never enters.)

(Event codes 0x5 “poison” and 0x6 “tag mismatch” were documented here but are NOT implemented – no poison/tag signal reaches the monitor.)

14.4.2.2 Completion Packets (Type=1)

Event Code	Description	Event Data
0x0	Read completion	Zero-extended 32-bit ADDRESS

14.4.2.3 Timeout Packets (Type=3)

Event Code	Description	Event Data
0x1	AR channel timeout	Zero-extended 32-bit ADDRESS
0x2	R channel timeout	Zero-extended 32-bit ADDRESS

14.4.2.4 Performance Packets (Type=4)

Event Code	Description	Event Data
(only two exist)	AXI_PERF_COMPLETED_COUNT	64’(completed-packet lifetime count)
	AXI_PERF_ERROR_COUNT	64’(error-packet lifetime count)

(High-latency / bandwidth / outstanding packets were never implemented – window outputs carry that data, not packets.)

14.4.3 Monitor Backpressure (block_ready)

`block_ready` is exported as the `debug_block_ready` output port – the wrapper deliberately makes the gating contract observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor’s transaction-table occupancy reaches its blocking threshold (a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it). The wrapper ANDs it into the upstream-facing `s_axi_aready` so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `s_axi_aready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property,

ap_disabled_never_stalls; this module has no ifdef FORMAL block of its own, so here the guarantee rests on the gate expression above, not a proof).

- **For axi5 slave variants**: the monitor watches the FUB-side handshake, so there is a SKID_DEPTH_AR cycle lag between block_ready going low and new events ceasing. MAX_TRANSACTIONS should be sized to cover this margin.

Recovery is guaranteed by the **saturation-recovery contract**: command-originated table entries are capped at MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS) (reserve = 4 for tables of 16 or more, 0 below; the function lives in ocelus_common_pkg), and block_ready re-asserts at occupancy < MAX_TRANSACTIONS - (reserve - 1) – a threshold strictly ABOVE the command cap – so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size MAX_TRANSACTIONS to cover NUM_CHANNELS x per-channel outstanding plus margin – the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's --unroll-count raised (default 64) in sim builds.

14.4.4 Address-Range Checker

With N_ADDR_RANGES > 0 the wrapper instantiates the shared allowlist checker (axi_monitor_addr_check). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a PktTypeAddrMatch (4'h8) packet with event code AXI_ADDR_RANGE_MATCH (8'h01), gated by cfg_debug_enable.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a PktTypeError (4'h0) packet with event code AXI_ERR_ADDR_RANGE (8'h0D), gated by cfg_error_enable.

This wrapper does not expose ADDR_RANGE_IS_ERROR; all ranges default to the DEBUG (match) flavor.

Config inputs (active only when N_ADDR_RANGES > 0): - cfg_addr_check_enable — master on/off for the checker. - cfg_addr_range_enable[N-1:0] — per-range enable bit. - cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0] — inclusive bounds.

Event encoding: event_data[63:60] = range_index (matching DEBUG range, or 4'hF sentinel on an ERROR miss); event_data[59:0] = full address. **Filtering:** AddrMatch is dropped by cfg_axi_addr_mask[1], the ADDR_RANGE error by cfg_axi_error_mask[13]. See the checker page for coalescing + formal properties.

14.4.5 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window's totals.

14.4.5.1 The measurement window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles` free-runs, counting every clock elapsed inside the window.

14.4.5.2 Utilization buckets (R data channel)

Every cycle inside the window is classified by the **R** data channel's valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	`rvalid && rready`	productive beat transferred
`perf_bp_cycles`	32	`rvalid && !rready`	back-pressure (data offered, sink not ready)
`perf_starv_cycles`	32	`!rvalid && rready`	starvation (sink ready, no data)
`perf_idle_cycles`	32	`!rvalid && !rready`	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); utilization = `perf_prod_cycles / window_cycles` is off by one count – negligible for long windows, use `window_cycles - 1` for exactness.

14.4.5.3 Throughput counters

Output	Width	Meaning
`perf_beat_count`	32	R data beats transferred (= `perf_prod_cycles`, 1

		beat/cycle)
`perf_byte_count`	64	bytes transferred = beats x (1 << latched ARSIZE), using the ARSIZE captured at the most recent AR address phase
`perf_burst_count`	32	AR address-phase handshakes

Average burst length is `perf_beat_count / perf_burst_count`.

14.4.5.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window **start** event source select
cfg_end_event_sel	3	Input	Window **end** event source select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_trigger	1	Input	Pulse: close the measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	`rvalid && rready` cycles
perf_bp_cy	32	Output	`rvalid && !rready` cycles

Port	Width	Direction	Description
cles			(back-pressure)
perf_starv_cycles	32	Output	`!rvalid && rready` cycles (starvation)
perf_idle_cycles	32	Output	`!rvalid && !rready` cycles
perf_beat_count	32	Output	R data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AR address-phase handshakes

When `USE_MONITOR = 0` all perfmon outputs are tied to 0. With `USE_MONITOR = 1` and `ENABLE_PERF_LOGIC = 0`, only error_count / transaction_count read 0 – the window and bucket/throughput outputs remain LIVE.

14.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

14.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_slave_rd_mon #(
    .SKID_DEPTH_AR         (2),
    .SKID_DEPTH_R          (4),
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (32),
    .AXI_USER_WIDTH        (1),
    .AXI_NSAID_WIDTH       (4),
    .AXI_MPAM_WIDTH        (11),
    .AXI_MECID_WIDTH       (16),
    .AXI_TAG_WIDTH         (4)
) u_axi5_slave_rd_mon (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cam_clear             (cam_clear),
    .s_axi_arid            (s_axi_arid),
    .s_axi_araddr          (s_axi_araddr),
    .s_axi_arlen           (s_axi_arlen),
    .s_axi_arsize          (s_axi_arsize),
    .s_axi_arburst         (s_axi_arburst),
    .s_axi_arlock          (s_axi_arlock),
    .s_axi_arcache         (s_axi_arcache),
    .s_axi_arprot          (s_axi_arprot),
    .s_axi_arqos           (s_axi_arqos),
    .s_axi_aruser          (s_axi_aruser),
    .s_axi_arvalid         (s_axi_arvalid),
    .s_axi_awakeup         (s_axi_awakeup),
    .s_axi_arready         (s_axi_arready),
    .s_axi_arnsaid         (s_axi_arnsaid),
    .s_axi_artrace         (s_axi_artrace),
    .s_axi_armpam          (s_axi_armpam),
    .s_axi_armecid         (s_axi_armecid),
    .s_axi_aridunq        (s_axi_aridunq),
    .s_axi_archunken       (s_axi_archunken),
    .s_axi_artagop         (s_axi_artagop),
    .s_axi_rid             (s_axi_rid),
    .s_axi_rdata           (s_axi_rdata),
    .s_axi_rresp           (s_axi_rresp),
    .s_axi_rlast           (s_axi_rlast),
    .s_axi_ruser           (s_axi_ruser),
    .s_axi_rvalid          (s_axi_rvalid),
    .s_axi_rready          (s_axi_rready),
    .s_axi_rtrace          (s_axi_rtrace),
    .s_axi_ridunq          (s_axi_ridunq),
```

```
    .s_axi_rpoison         (s_axi_rpoison),
    .s_axi_rchunkv         (s_axi_rchunkv),
    .s_axi_rchunknum       (s_axi_rchunknum),
    .s_axi_rchunkstrb      (s_axi_rchunkstrb),
    .s_axi_rtag            (s_axi_rtag),
    .fub_axi_arid          (fub_axi_arid),
    .fub_axi_araddr        (fub_axi_araddr),
    .fub_axi_arlen         (fub_axi_arlen),
    .fub_axi_arsize        (fub_axi_arsize),
    .fub_axi_arburst       (fub_axi_arburst),
    .fub_axi_arlock        (fub_axi_arlock),
    .fub_axi_arcache       (fub_axi_arcache),
    .fub_axi_arprot        (fub_axi_arprot),
    .fub_axi_arqos         (fub_axi_arqos),
    .fub_axi_aruser        (fub_axi_aruser),
    .fub_axi_arvalid       (fub_axi_arvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_arready       (fub_axi_arready),
    .fub_axi_arnsaid       (fub_axi_arnsaid),
    .fub_axi_artrace       (fub_axi_artrace),
    .fub_axi_armpam        (fub_axi_armpam),
    .fub_axi_armecid       (fub_axi_armecid),
    .fub_axi_aridunq      (fub_axi_aridunq),
    .fub_axi_archunken     (fub_axi_archunken),
    .fub_axi_artagop       (fub_axi_artagop),
    .fub_axi_rid           (fub_axi_rid),
    .fub_axi_rdata         (fub_axi_rdata),
    .fub_axi_rresp         (fub_axi_rresp),
    .fub_axi_rlast         (fub_axi_rlast),
    .fub_axi_ruser         (fub_axi_ruser),
    .fub_axi_rvalid        (fub_axi_rvalid),
    .fub_axi_rready        (fub_axi_rready),
    .fub_axi_rtrace        (fub_axi_rtrace),
    .fub_axi_ridunq        (fub_axi_ridunq),
    .fub_axi_rpoison       (fub_axi_rpoison),
    .fub_axi_rchunkv       (fub_axi_rchunkv),
    .fub_axi_rchunknum     (fub_axi_rchunknum),
    .fub_axi_rchunkstrb    (fub_axi_rchunkstrb),
    .fub_axi_rtag          (fub_axi_rtag),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
```

```
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .debug_block_ready     (debug_block_ready),
    .cfg_conflict_error    (cfg_conflict_error),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

14.7 Design Notes

- Monitor packets provide real-time transaction visibility without external logic
- Filtering reduces monitor bus bandwidth - critical for high-throughput systems
- Transaction table size (MAX_TRANSACTIONS) must accommodate peak outstanding transactions
- Performance packets can generate high traffic - use sparingly or with filtering
- UNIT_ID and AGENT_ID identify this monitor in multi-agent systems
- Error count WRAPS at 16'hFFFF (plain increment, no saturation – the perfmon BUCKET counters are the ones that saturate)

14.8 Related Modules

- **AXI5 Slave Read** - Non-monitored version
- **AXI5 Slave Read Monitor CG** - Clock-gated variant
- **AXI5 Slave Write Monitor** - Write monitor
- **AXI Monitor Filtered** - Monitor core
- **Monitor Package Spec** - Packet format details

14.9 Testing

`val/amba/test_axi5_slave_rd_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_rd_mon.py -v
```

14.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- The monitor’s own techniques (transaction table, timeouts, packet filtering) are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects” (“AXI bus protocol overview”) – textbook overview of AXI responses and ID ordering (secondary source).
- Indexed in `rtl/amba/references/README.md` (protocol specifications are citation only)

14.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

15 AXI5 Slave Read Monitor with Clock Gating

Module: `axi5_slave_rd_mon_cg.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

15.1 Overview

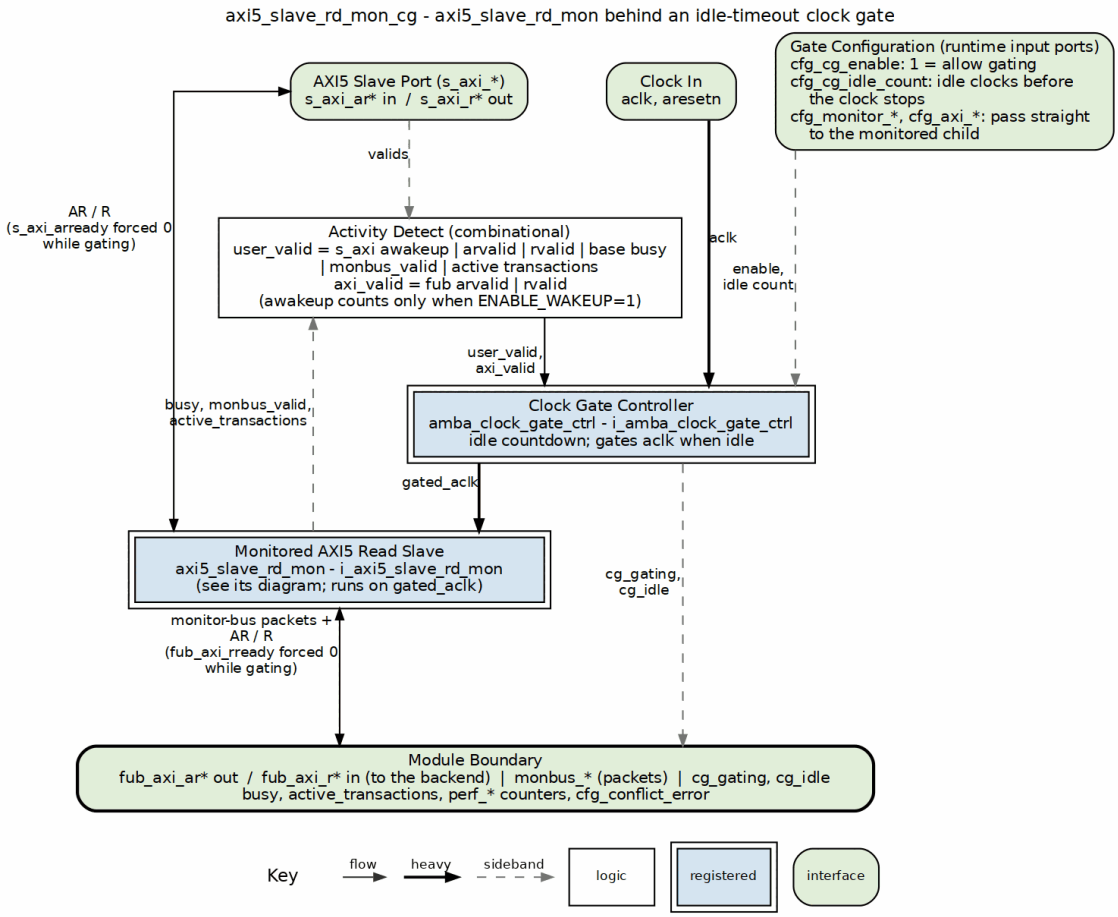
The AXI5 Slave Read Monitor with Clock Gating module puts both power management and observability on the slave read path: it wraps `axi5_slave_rd_mon` with clock gating logic, so transaction monitoring and automatic clock gating come in one block.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

15.1.1 Key Features

- Full AMBA AXI5 slave read protocol compliance
- **Integrated filtered monitoring** - real-time transaction visibility
- **Integrated clock gating** for dynamic power reduction
- All AXI5 extensions supported (NSAID, TRACE, MPAM, MECID, IDUNQ, CHUNKING, MTE, POISON, WAKEUP)
- Configurable idle count before gating
- Transaction tracking with error detection
- Performance metrics and filtering
- Transparent gating - no protocol changes
- Gating status outputs for system monitoring
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

15.1.2 Module Architecture



axi5_slave_rd_mon_cg block diagram

The base `axi5_slave_rd_mon` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `s_axi` valids, the child's busy, and `s_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `fub_axi` valids. The monitored child's `monbus_valid` and active-transaction count also count as activity, so the clock never stops under a pending packet.

15.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axi5_slave_rd_monlite_cg`: the same gating logic around `axi5_slave_rd_monlite` instead of `axi5_slave_rd_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel SKID buffer depth
SKID_DEPTH_R	int	4	R channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
AXI_CHUNKNUM_WIDTH	int	4	Chunk number width
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and

Parameter	Type	Default	Description
			BIDUNQ/RIDUNQ on the responses)
ENABLE_CHUNKING	bit	1	Enable data chunking
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	12	Agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection

Parameter	Type	Default	Description
			(forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.

Parameter	Type	Default	Description
CFI_MIN_FREQ_MHZ	int	ACLK_MHZ	Lowest frequency the tick LUT must cover (dynamic-frequency builds).
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. The USE_WDATA_ORDER_Q pairing rule applies to WRITE monitors only – axi_monitor_trans_mgr guards on (NUM_BANKS > 1) && !IS_READ && ! USE_WDATA_ORDER_Q, so a read monitor may bank freely.
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
ID_FILTER_ENABLE	bit	0	Synthesises the ID report filter (see cfg_id_* for the runtime override).
ID_MATCH_BASE	int	0	First ID this instance owns.
ID_MATCH_COUNT	int	0	How many IDs; 0 means ALL, so a zeroed register

Parameter	Type	Default	Description
			block does not silently filter everything away.
CG_IDLE_COUNT_WIDTH	int	4	Clock gating idle counter width

15.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
AW	AXI_ADDR_WIDTH
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS
CHUNK_STRB_WIDTH	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1

15.3 Ports

15.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

15.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

15.3.3 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating

Port	Width	Direction	Description
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

15.3.4 Slave AXI5 Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

15.3.5 FUB Interface

Same as `axi5_slave_rd` - see AXI5 Slave Read for complete port list.

15.3.6 Monitor Configuration

Same as `axi5_slave_rd_mon` - see AXI5 Slave Read Monitor for complete list.

15.3.7 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	`monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

15.3.8 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions

Port	Width	Direction	Description
error_count	16	Output	Total errors detected
transaction_count	32	Output	Total transactions completed
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

15.4 Functional Description

15.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (AWAKEUP, arvalid, rvalid, internal busy, a monitor packet pending on the monitor bus, or monitor CAM entries still live – peer VALID) - **axi_valid:** Asserted when FUB interface has activity (arvalid, rvalid)

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals forced to 0 when gated (prevents new transactions) - Gating only occurs after configured idle period - Any activity immediately ungates the clock - Monitor state IS a wake term: both a packet pending on the monitor bus (`w_monbus_valid`) and live monitor CAM entries (`|active_transactions`, which covers the reporter's few-cycle emission window) hold the block awake, and the external `monbus_valid` is masked with `!cg_gating` – so monitor-bus delivery is exactly-once across gating at any idle count (`val/amba/test_mon_cg_gating.py` phase 6)

15.4.2 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_slave_rd_mon`. The measurement-window state machine, the four R-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi5_slave_rd_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AR (read address) handshakes. **WARNING:** an open measurement window is NOT a wake term – the window FSM and all counters run on the gated clock, so if the bus idles past `cfg_cg_idle_count` mid-window the counters FREEZE while wall-clock time passes, and trigger pulses arriving while gated are DROPPED. For exact wall-clock windows hold `cfg_cg_enable` low around the measurement, or use the base module.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see Parameters), straight through to the base monitor.

15.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AR	2 entries
SKID_DEPTH_R	4 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

15.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_slave_rd_mon_cg #(
    .SKID_DEPTH_AR         (2),
    .SKID_DEPTH_R          (4),
    .AXI_ID_WIDTH          (8),
    .AXI_ADDR_WIDTH        (32),
    .AXI_DATA_WIDTH        (32),
    .AXI_USER_WIDTH        (1),
    .AXI_NSAID_WIDTH       (4),
    .AXI_MPAM_WIDTH        (11),
    .AXI_MECID_WIDTH       (16),
    .AXI_TAG_WIDTH         (4)
) u_axi5_slave_rd_mon_cg (
    .aclk                  (aclk),
    .aresetn               (aresetn),
    .cam_clear             (cam_clear),
    .cfg_cg_enable         (cfg_cg_enable),
    .cfg_cg_idle_count     (cfg_cg_idle_count),
    .s_axi_arid            (s_axi_arid),
    .s_axi_araddr          (s_axi_araddr),
    .s_axi_arlen           (s_axi_arlen),
    .s_axi_arsize          (s_axi_arsize),
    .s_axi_arburst         (s_axi_arburst),
    .s_axi_arlock          (s_axi_arlock),
    .s_axi_arcache         (s_axi_arcache),
    .s_axi_arprot          (s_axi_arprot),
    .s_axi_arqos           (s_axi_arqos),
    .s_axi_aruser          (s_axi_aruser),
    .s_axi_arvalid         (s_axi_arvalid),
    .s_axi_awakeup         (s_axi_awakeup),
    .s_axi_arready         (s_axi_arready),
    .s_axi_arnsaid         (s_axi_arnsaid),
    .s_axi_artrace         (s_axi_artrace),
    .s_axi_armpam          (s_axi_armpam),
    .s_axi_armecid         (s_axi_armecid),
    .s_axi_aridunq        (s_axi_aridunq),
    .s_axi_archunken       (s_axi_archunken),
    .s_axi_artagop         (s_axi_artagop),
    .s_axi_rid             (s_axi_rid),
    .s_axi_rdata           (s_axi_rdata),
    .s_axi_rresp           (s_axi_rresp),
    .s_axi_rlast           (s_axi_rlast),
    .s_axi_ruser           (s_axi_ruser),
    .s_axi_rvalid          (s_axi_rvalid),
    .s_axi_rready          (s_axi_rready),
```

```
    .s_axi_rtrace          (s_axi_rtrace),
    .s_axi_ridunq          (s_axi_ridunq),
    .s_axi_rpoison         (s_axi_rpoison),
    .s_axi_rchunkv         (s_axi_rchunkv),
    .s_axi_rchunknum       (s_axi_rchunknum),
    .s_axi_rchunkstrb      (s_axi_rchunkstrb),
    .s_axi_rtag            (s_axi_rtag),
    .fub_axi_arid          (fub_axi_arid),
    .fub_axi_araddr        (fub_axi_araddr),
    .fub_axi_arlen         (fub_axi_arlen),
    .fub_axi_arsize        (fub_axi_arsize),
    .fub_axi_arburst       (fub_axi_arburst),
    .fub_axi_arlock        (fub_axi_arlock),
    .fub_axi_arcache       (fub_axi_arcache),
    .fub_axi_arprot        (fub_axi_arprot),
    .fub_axi_arqos         (fub_axi_arqos),
    .fub_axi_aruser        (fub_axi_aruser),
    .fub_axi_arvalid       (fub_axi_arvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_arready       (fub_axi_arready),
    .fub_axi_arnsaid       (fub_axi_arnsaid),
    .fub_axi_artrace       (fub_axi_artrace),
    .fub_axi_armpam        (fub_axi_armpam),
    .fub_axi_armecid       (fub_axi_armecid),
    .fub_axi_aridunq      (fub_axi_aridunq),
    .fub_axi_archunken     (fub_axi_archunken),
    .fub_axi_artagop       (fub_axi_artagop),
    .fub_axi_rid           (fub_axi_rid),
    .fub_axi_rdata         (fub_axi_rdata),
    .fub_axi_rresp         (fub_axi_rresp),
    .fub_axi_rlast         (fub_axi_rlast),
    .fub_axi_ruser         (fub_axi_ruser),
    .fub_axi_rvalid        (fub_axi_rvalid),
    .fub_axi_rready        (fub_axi_rready),
    .fub_axi_rtrace        (fub_axi_rtrace),
    .fub_axi_ridunq        (fub_axi_ridunq),
    .fub_axi_rpoison       (fub_axi_rpoison),
    .fub_axi_rchunkv       (fub_axi_rchunkv),
    .fub_axi_rchunknum     (fub_axi_rchunknum),
    .fub_axi_rchunkstrb    (fub_axi_rchunkstrb),
    .fub_axi_rtag          (fub_axi_rtag),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
```

```
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .cfg_conflict_error    (cfg_conflict_error),
    .cg_gating             (cg_gating),
    .cg_idle               (cg_idle),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
```

```
    .perf_burst_count      (perf_burst_count)
);
```

15.7 Design Notes

15.7.1 Power Optimization

Power Savings Estimation: - Base slave logic: ~40% of total power *(first-order estimates – no power/area analysis has been run)* - Monitor logic: ~60% of total power - With gating at 50% duty cycle: ~50% dynamic power savings - Actual savings depend on traffic pattern and idle_count setting

15.7.2 Observability

Monitoring Capabilities: - Error detection (SLVERR, timeout, orphan) - Performance tracking (latency, throughput) - Transaction completion tracking - All monitoring continues when ungated

15.7.3 When to Use This Module

Ideal for: - Power-constrained systems needing visibility - Debug/validation builds with power budgets - Systems with sporadic transaction patterns - SoC integration requiring both monitoring and power optimization

Avoid when: - Interface is continuously active (minimal gating opportunities) - Monitoring overhead unacceptable (use non-monitored version) - Deterministic latency critical (use high idle count or disable gating)

15.7.4 Configuration Recommendations

Low-Power Debug Mode:

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd1),   // Aggressive gating
.cfg_monitor_enable (1'b1),   // Full monitoring
.cfg_error_enable   (1'b1),
.cfg_perf_enable    (1'b0)    // Reduce packet traffic
```

Performance Analysis Mode:

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd4),   // Conservative gating
.cfg_monitor_enable (1'b1),   // Master gate MUST stay 1 (0 disables
ALL monitoring); disable completions via cfg_compl_enable
.cfg_perf_enable    (1'b1)    // Enable performance metrics
```

15.7.5 Area and Timing Impact

- **Area:** not 5-8%. That figure was an unsourced estimate and is off by orders of magnitude. The monitor's transaction table alone is a counted floor of thousands of flops (see the resource table in the family README) against a few hundred for the unmonitored slave, so a monitored instance is a

multiple of the base module, not a few percent on top of it. Clock gating adds 5 flops. `MAX_TRANSACTIONS` is the knob and the cost is linear in it

- **Timing:** first-order estimate, not measured. The monitor is NOT automatically off the critical path – `axi_monitor_trans_mgr` is banked precisely because it was not (16 entries closed at WNS +1.018 ns where 40 entries came in at -25.183 ns), so size the transaction table for your clock
- **Power:** Net savings depends on activity factor (typically 30-50% at 50% duty cycle)

15.8 Related Modules

- **AXI5 Slave Read** - Non-monitored, non-gated version
- **AXI5 Slave Read CG** - Clock-gated without monitoring
- **AXI5 Slave Read Monitor** - Monitored without clock gating
- **AXI5 Slave Write Monitor CG** - Write variant
- **AMBA Clock Gate Control** - Clock gating controller

15.9 Testing

`val/amba/test_axi5_slave_rd_mon_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_rd_mon_cg.py -v
```

15.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- The monitor’s own techniques are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

15.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

16 AXI5 Slave Write

Module: `axi5_slave_wr.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

16.1 Overview

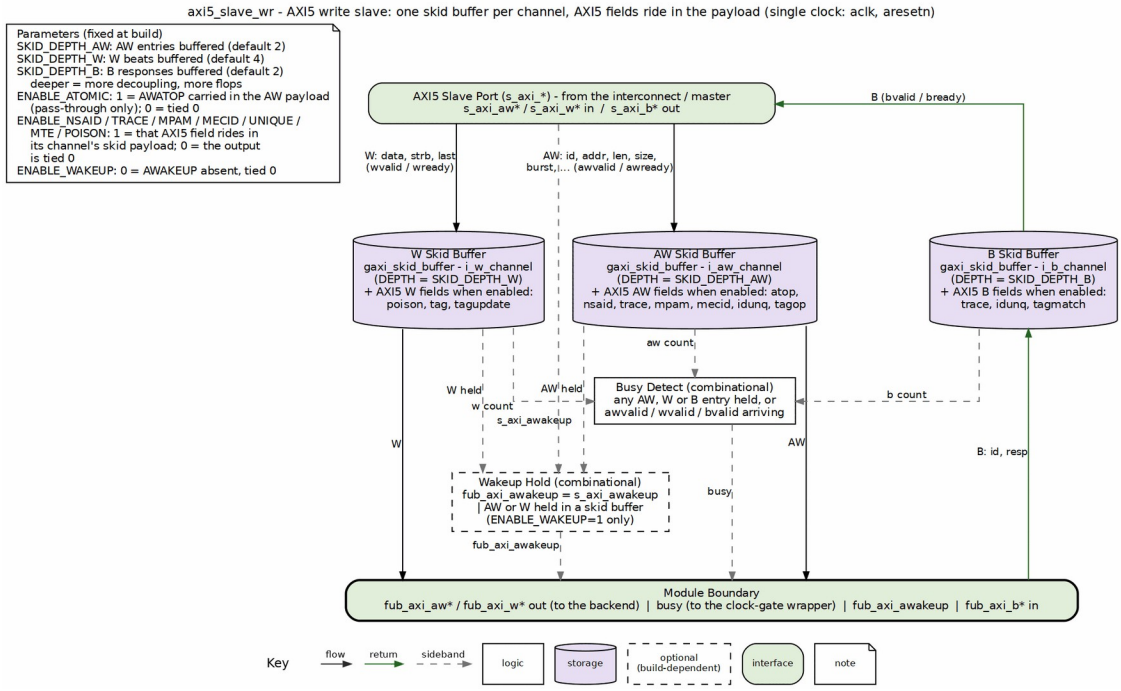
The AXI5 Slave Write module implements a complete AMBA AXI5 slave write interface with full AXI5 protocol support. It receives write requests from external masters and forwards them to backend memory/logic (FUB interface).

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. `AWATOP` is carried through unmodified but no atomic read-modify-write is performed, no MTE tag checking or `BTAGMATCH` generation is performed, and no outstanding-transaction tracking is done. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

16.1.1 Key Features

- Carries the full AXI5 signal set listed below, unmodified, across the SKID buffers
- **AWATOP:** Atomic transaction operation type
- **AWNSAID:** Non-secure access identifier reception
- **AWTRACE:** Trace signal support
- **AWMPAM:** Memory Partitioning and Monitoring reception
- **AWMECID:** Memory Encryption Context ID reception
- **AWIDUNQ / BIDUNQ:** Unique ID indicator on the request and its write response (IHI 0022L §A5.2)
- **AWTAGOP:** Memory tag operation reception (MTE)
- **WPOISON:** Write data poison indicator
- **WTAG/WTAGUPDATE:** Write data memory tags (MTE)
- **BTRACE / BIDUNQ / BTAGMATCH[1:0]:** Response trace, unique-ID indicator and the 2-bit tag match result
- SKID buffering for AW, W, and B channels
- Busy signal for power management
- AWREGION not implemented (see *Design Notes*)

16.1.2 Module Architecture



axi5_slave_wr block diagram

The slave port (`s_axi_*`) is at the top and the module boundary is the bar. W takes its own lane down the left, AW goes down the center, and B comes back up the right lane, each through its own skid buffer. The AXI5 fields each channel carries (NSAID, trace, MPAM, MECID, unique-ID, MTE tags, poison, and AWATOP for atomics) ride in that channel's skid payload when their `ENABLE_*` parameter is set; a disabled field's output is tied to 0. The busy detect feeds the clock-gate wrapper, and the wakeup hold keeps `fub_axi_awakeup` asserted while an accepted request is still waiting in a skid buffer, not only while `s_axi_awakeup` is high (IHI 0022L A14.1.1).

16.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_W	int	4	W channel SKID buffer depth (legal range 2..8 inclusive, any integer)
SKID_DEPTH_B	int	2	B channel SKID buffer depth (legal range 2..8 inclusive, any integer)
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption

Parameter	Type	Default	Description
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low

16.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
SW	AXI_WSTRB_WIDTH	Write strobe width, one bit per data byte
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `wtag` field
AWSize	Sum of the enabled AW fields	AW SKID buffer payload width
WSize	Sum of the enabled W fields	W SKID buffer payload width
BSize	Sum of the enabled B fields	B SKID buffer payload width

16.3 Ports

16.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

16.3.2 Slave AXI5 Interface (External Master Input)

16.3.2.1 AW Channel (Input)

Port	Width	Direction	Description
s_axi_awid	IW	Input	Write address ID
s_axi_awaddr	AW	Input	Write address
s_axi_awlen	8	Input	Burst length
s_axi_awsize	3	Input	Burst size
s_axi_awburst	2	Input	Burst type
s_axi_awlock	1	Input	Lock type
s_axi_awcache	4	Input	Cache attributes
s_axi_awprot	3	Input	Protection attributes
s_axi_awqos	4	Input	Quality of Service
s_axi_awuser	UW	Input	User-defined signal
s_axi_awvalid	1	Input	Write address valid
s_axi_awakeup	1	Input	Wake-up request, AWAKEUP (IHI 0022L

Port	Width	Direction	Description
			§A14.1.1; ENABLE_WAKEUP)
s_axi_awready	1	Output	Write address ready
s_axi_awatop	AXI_ATOP_WIDTH	Input	Atomic operation (AXI5)
s_axi_awnsaid	AXI_NSAID_WIDTH	Input	Non-secure access ID (AXI5)
s_axi_awtrace	1	Input	Trace signal (AXI5)
s_axi_awmpam	AXI_MPAM_WIDTH	Input	Memory partitioning (AXI5)
s_axi_awmecid	AXI_MECID_WIDTH	Input	Memory encryption context (AXI5)
s_axi_awidunq	1	Input	Unique ID indicator, AWIDUNQ (IHI 0022L §A5.2; ENABLE_UNIQUE)
s_axi_awtagop	AXI_TAGOP_WIDTH	Input	Tag operation (AXI5 MTE)

16.3.2.2 W Channel (Input)

Port	Width	Direction	Description
s_axi_wdata	DW	Input	Write data
s_axi_wstrb	SW	Input	Write strobes
s_axi_wlast	1	Input	Last transfer in burst
s_axi_wuser	UW	Input	User-defined signal
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready
s_axi_wpoison	1	Input	Data poison indicator

Port	Width	Direction	Description
			(AXI5)
s_axi_wtag	TW	Input	Write data memory tags (AXI5 MTE)
s_axi_wtagupdate	NUM_TAGS	Input	Tag update mask (AXI5 MTE)

16.3.2.3 B Channel (Output)

Port	Width	Direction	Description
s_axi_bid	IW	Output	Write response ID
s_axi_bresp	2	Output	Write response
s_axi_buser	UW	Output	User-defined signal
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready
s_axi_btrace	1	Output	Trace signal (AXI5)
s_axi_biduq	1	Output	Unique ID indicator on the write response, BIDUNQ (IHI 0022L §A5.2; `ENABLE_UNIQUE`)
s_axi_btagmatch	2	Output	Tag match result, IHI 0022L Table A12.14 (00 None, 01 Separate, 10 Fail, 11 Pass)

16.3.3 FUB Interface (Backend Memory/Logic Output)

Same signals as slave interface but with `fub_axi_*` prefix and reversed directions for AW/W (output) and B (input).

16.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator

16.4 Functional Description

This slave module acts as an adapter between: - **External AXI5 master** (s_axi_* interface) - **Backend memory or functional unit block** (fub_axi_* interface)

Signal Flow: 1. AW channel: External master → SKID → Backend 2. W channel: External master → SKID → Backend 3. B channel: Backend → SKID → External master

SKID Buffering: Provides elasticity and breaks combinational paths for timing closure.

Busy Signal: Asserted when any SKID buffer has data or handshakes are pending.

16.5 Timing Characteristics

16.5.1 Basic Write Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- S_AXI_AWID, S_AXI_AWADDR, S_AXI_AWVALID, S_AXI_AWREADY
- S_AXI_WDATA, S_AXI_WSTRB, S_AXI_WLAST, S_AXI_WVALID, S_AXI_WREADY
- FUB_AXI_AW* and FUB_AXI_W* (request forwarded to the backend)
- FUB_AXI_BRESP, FUB_AXI_BVALID, FUB_AXI_BREADY
- S_AXI_BID, S_AXI_BRESP, S_AXI_BVALID, S_AXI_BREADY

16.5.2 Write with Poison Indicator

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK
- S_AXI_WDATA, S_AXI_WVALID, S_AXI_WREADY
- S_AXI_WPOISON asserted on the poisoned beat
- FUB_AXI_WPOISON transported to the backend on the same beat

16.6 Usage Examples

```
axi5_slave_wr #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    .ENABLE_ATOMIC      (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_wr (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Slave interface (from external master)
    .s_axi_awid         (s_axi_awid),
    .s_axi_awaddr       (s_axi_awaddr),
    .s_axi_awlen        (s_axi_awlen),
    // Every remaining s_axi_aw* port mirrors the fub_axi_aw* list.
All must be connected.
    .s_axi_awatop       (s_axi_awatop),
    .s_axi_awnsaid      (s_axi_awnsaid),
    .s_axi_awtrace      (s_axi_awtrace),
    // ... (connect AXI5 extensions)

    .s_axi_wdata        (s_axi_wdata),
    .s_axi_wstrb        (s_axi_wstrb),
    // Every remaining s_axi_w* port mirrors the fub_axi_w* list. All
must be connected.

    .s_axi_bid          (s_axi_bid),
    .s_axi_bresp        (s_axi_bresp),
    // Every remaining s_axi_b* port mirrors the fub_axi_b* list. All
must be connected.

    // FUB interface (to backend)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    // ... (connect to memory controller)
```

```
    .busy                (slave_wr_busy)
);
```

16.7 Design Notes

- Use when you need AXI5 slave write functionality with SKID buffering
- Backend FUB interface can connect to memory controller, SRAM, or functional units
- SKID buffers improve timing by breaking long combinational paths
- W channel SKID depth typically larger than AW/B due to burst data
- Feature enables allow area optimization by disabling unused AXI5 extensions
- AWREGION has no port on this module. AxREGION is not deprecated by AXI5; it remains a valid optional signal and is simply omitted here. Decode or route by address instead, or use `axi4_slave_wr`

16.8 Related Modules

- **AXI5 Slave Read** - Slave read interface
- **AXI5 Master Write** - Master write interface
- **AXI5 Slave Write CG** - Clock-gated variant
- **AXI5 Slave Write Monitor** - With integrated monitoring

16.9 Testing

`val/amba/test_axi5_slave_wr.py` exercises this module. It collects 8 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_wr.py -v
```

16.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A6.4 “Atomic transactions” (AWATOP), §A16.1 “Data protection using Poison” (WPOISON), §A12.2 “Memory Tagging Extension (MTE)” (AWTAGOP, WTAG, WTAGUPDATE, BTAGMATCH), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (AWMECID). The ports follow the specification’s names and widths (GitHub #102): BTAGMATCH is 2 bits with the Table A12.14 encodings, there is no BTAG or AWTAG, the Unique ID indicator is AWIDUNQ with BIDUNQ on the response, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Arm. *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022, Issue H, Mar. 2020. developer.arm.com/documentation/ihi0022/h – the same AMBA 5 features as first numbered: ch. C1 “AMBA AXI5”, §E1.1 “Atomic transactions”, §E1.15 “Memory tagging”, §E2.1 “Poison”, §E1.13 “Unique ID indicator”. MECID is not in Issue H.
- Carloni, L.P., McMillan, K.L., Sangiovanni-Vincentelli, A.L. “Theory of Latency-Insensitive Design.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 20(9), Sept. 2001. DOI: 10.1109/43.945302 – relay stations: a channel can be pipelined by any number of back-pressured stages without changing its behavior, which is why a skid buffer per channel is transparent to both ends.
- Cortadella, J., Kishinevsky, M., Grundmann, B. “Synthesis of Synchronous Elastic Architectures.” *Proc. 43rd Design Automation Conference (DAC ’06)*, July 2006, pp. 657-662. DOI: 10.1145/1146909.1147077 – the elastic buffer with valid/stop back-pressure, the structure a skid buffer implements.
- Chipressian. *Crack the Hardware Interview: from RTL Designers’ Perspective*, Part 2 ch. 2 “Handshake Protocols” (forward and backward register slices) – textbook treatment of the valid/ready slice (secondary source).

- Indexed in `rtl/amba/references/README.md` (local PDF: `LatencyInsensitiveDesign__Carloni2001.pdf`, `ElasticBuffer__Cortadella2006.pdf`)

16.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

17 AXI5 Slave Write with Clock Gating

Module: `axi5_slave_wr_cg.sv` **Location:** `rtl/amba/axi5/` **Status:** Production Ready

17.1 Overview

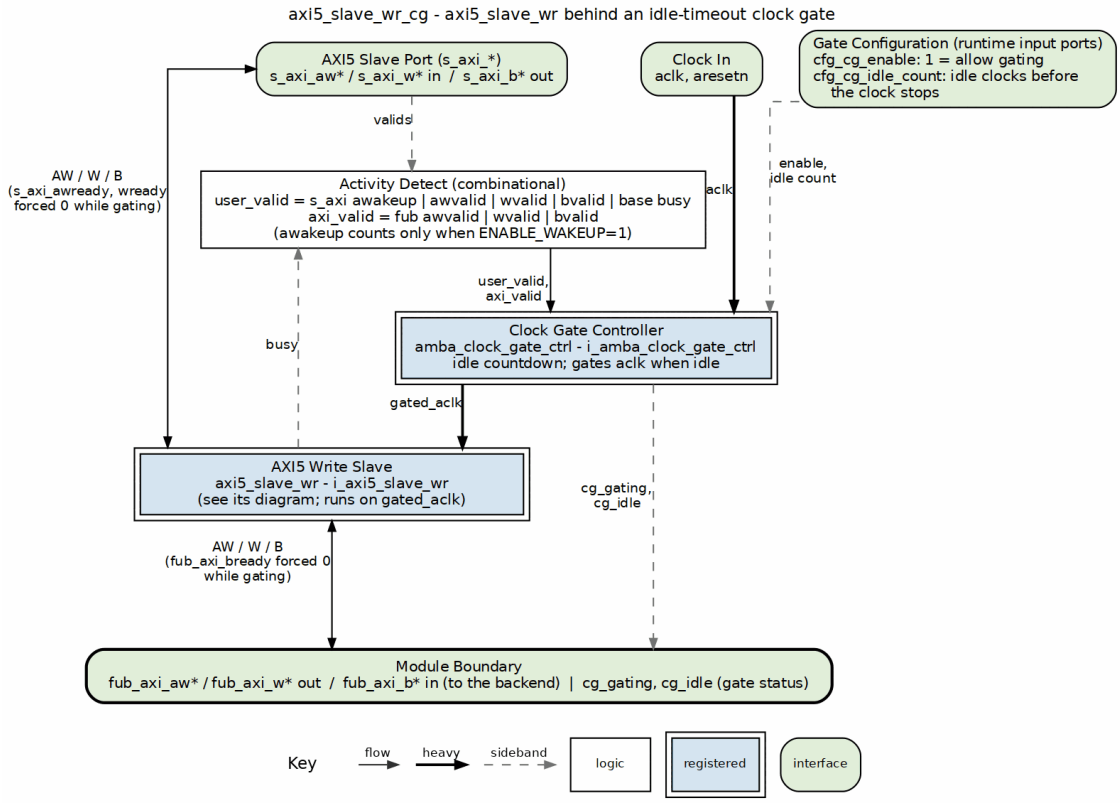
The AXI5 Slave Write with Clock Gating module wraps the standard `axi5_slave_wr` module with integrated clock gating for power optimization. When the module sits idle, the internal clock gates itself off.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. `AWATOP` is carried through unmodified but no atomic read-modify-write is performed, no MTE tag checking or `BTAGMATCH` generation is performed, and no outstanding-transaction tracking is done. Those behaviors belong to the endpoints on either side. See Scope of This Implementation in the AXI5 index for the full coverage statement.

17.1.1 Key Features

- Carries the full AXI5 write signal set listed below, unmodified, across the SKID buffers
- **Integrated clock gating** for dynamic power reduction
- Configurable idle count before gating
- All AXI5 extensions supported (ATOMIC, NSAID, TRACE, MPAM, MECID, IDUNQ, MTE, POISON, WAKEUP)
- Transparent gating - no protocol changes
- Gating status outputs for system monitoring
- SKID buffering for AW, W, and B channels
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

17.1.2 Module Architecture



axi5_slave_wr_cg block diagram

The base `axi5_slave_wr` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `s_axi` valids, the child's busy, and `s_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `fub_axi` valids.

17.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the

Parameter	Type	Default	Description
			responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
CG_IDLE_COUNT_WIDTH	int	4	Clock gating idle counter width

17.2.1 Derived Parameters

These are computed inside the module from the parameters above. Do not override them.

Parameter	Expression	Description
SW	AXI_WSTRB_WIDTH	Write strobe width, one bit per data byte
NUM_TAGS	max(AXI_DATA_WIDTH / 128, 1)	MTE tags carried per beat (one tag per 16 bytes)
TW	AXI_TAG_WIDTH * NUM_TAGS	Total width of the `wtag` field
AWSize	Sum of the enabled AW fields	AW SKID buffer payload width
WSize	Sum of the enabled W fields	W SKID buffer payload width
BSize	Sum of the enabled B fields	B SKID buffer payload width

17.2.2 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
`DW`	`AXI_DATA_WIDTH`

Derived parameter	Default expression
IW	AXI_ID_WIDTH
UW	AXI_USER_WIDTH

17.3 Ports

17.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset

17.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

17.3.3 Slave AXI5 Interface

Same as `axi5_slave_wr` - see AXI5 Slave Write for complete port list.

17.3.4 FUB Interface

Same as `axi5_slave_wr` - see AXI5 Slave Write for complete port list.

17.3.5 Clock Gating Status

Port	Width	Direction	Description
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

17.4 Functional Description

17.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (AWAKEUP, awvalid, wvalid, bvalid, or internal busy – peer VALID, post-fix) - **axi_valid:** Asserted when FUB interface has activity (awvalid, wvalid, bvalid)

Gating State Machine:

```
stateDiagram-v2
    [*] --> ACTIVE

    ACTIVE --> COUNTING : !user_valid && !axi_valid
    COUNTING --> ACTIVE : user_valid || axi_valid
    COUNTING --> GATED : count == cfg_cg_idle_count
    GATED --> ACTIVE : user_valid || axi_valid

    state ACTIVE {
        note right of ACTIVE : Clock enabled<br/>cg_gating = 0
    }
    state COUNTING {
        note right of COUNTING : Idle countdown<br/>cg_gating = 0
    }
    state GATED {
        note right of GATED : Clock gated<br/>cg_gating = 1
    }
```

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals (awready, wready) forced to 0 when gated (prevents new transactions) - bready forced to 0 when gated (prevents accepting responses) - Gating only occurs after configured idle period - Any activity immediately ungates the clock

17.4.2 Ready Deassertion and Wake Behavior

While `cg_gating` is asserted, the ready outputs listed above are forced low.

This is legal AXI. A READY output may be deasserted for any number of cycles, and neither side may assume READY asserts within a bounded time, so holding it low is ordinary backpressure rather than a protocol violation. It cannot deadlock either: the same VALID being held off also feeds the activity detector, so gating releases and READY returns to the core's value.

Two consequences are worth designing around:

- **Wake latency adds to first-transfer latency.** Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl`

combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. The AXI5 `_cg` wrappers drive the activity terms combinationally, so there is 1 register stage and the first usable gated-clock edge arrives 2 cycles after activity asserts. Size `cfg_cg_idle_count` so that cost is amortized over the traffic burst that follows.

- **READY is a combinational function of `cg_gating`.** The path from the idle counter to the `s_axi_awready` and `s_axi_wready` outputs may need attention during timing closure, particularly at high frequency.

17.4.3 Integration Caveat: `s_axi_bready` in the Activity Term

The activity detector uses the corresponding VALID (`s_axi_bvalid`), never the peer's READY. (Historically this term was `s_axi_bready`, which meant a peer parking its ready high – a common and entirely correct style – pinned the block permanently awake and defeated gating; that defect was fixed family-wide, matching the rule the mon_cg wrappers always carried.)

An always-ready consumer no longer prevents gating.

17.4.4 Clock Gating Configuration

17.4.4.1 Idle Count Selection

cfg_cg_idle_count	Idle Cycles	Use Case
0	1	Aggressive power saving, frequent gating
1-3	2-4	Balanced, reduces gate churn
4-7	5-8	Conservative, for bursty traffic
8-15	9-16	Minimal gating (the 4-bit field maxes at 16 cycles – the count is LITERAL, gating at count+1, not a power of two)

Recommendations: - **Burst writes:** Higher count (4-8) to avoid gating mid-burst - **Sporadic writes:** Lower count (0-2) for maximum power savings - **Real-time systems:** Disable gating or use high count to ensure deterministic latency

---

17.5 Timing Characteristics

17.5.1 Clock Gating During Write Burst

Timing diagram pending. The signals and sequence this scenario exercises:

- ACLK (ungated)
- GATED_ACLK
- AWVALID, WVALID, BVALID across a burst write
- user_valid, axi_valid activity terms
- cg_idle, cg_gating
- Gating engaging after the burst and its B response complete

17.6 Usage Examples

```
axi5_slave_wr_cg #(
    .AXI_ID_WIDTH       (8),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),
    .SKID_DEPTH_B       (2),
    .CG_IDLE_COUNT_WIDTH(4),
    .ENABLE_ATOMIC      (1),
    .ENABLE_NSAID       (1),
    .ENABLE_TRACE       (1),
    .ENABLE_MPAM        (1),
    .ENABLE_MECID       (1),
    .ENABLE_UNIQUE      (1),
    .ENABLE_MTE         (1),
    .ENABLE_POISON      (1)
) u_axi5_slave_wr_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // Clock gating config
    .cfg_cg_enable      (1'b1),          // Enable gating
    .cfg_cg_idle_count  (4'd3),          // Gate after 4 idle cycles
(count+1; a LITERAL count, not a power of two)

    // Slave interface (from external master)
    .s_axi_awid         (s_axi_awid),
    .s_axi_awaddr       (s_axi_awaddr),
    // Every remaining s_axi_aw*/s_axi_w*/s_axi_b* port mirrors the
fub_axi_* list,
    // same names and widths, opposite directions. All must be
connected.

    // FUB interface (to backend)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    // ... (connect to memory controller)

    // Clock gating status
    .cg_gating          (slave_wr_gating),
    .cg_idle            (slave_wr_idle)
);

// Optional: Monitor power savings
always @(posedge axi_clk) begin
```

```
    if (slave_wr_gating)
        $display("AXI5 Slave WR clock gated - saving power");
end

// Power management integration
assign system_low_power = slave_wr_gating &&
                          slave_rd_gating;
```

17.7 Design Notes

17.7.1 Power Savings Estimation

Worked example, assuming the module is active 60% of the time: - **Without gating:** 100% of this module's dynamic power - **With gating (idle_count=2):** roughly 50%, allowing for wake/sleep transitions - **With gating (idle_count=0):** roughly 60%; more aggressive gating, but more transition overhead

All power figures on this page are first-order estimates derived from duty cycle, not measured results. No power analysis (gate-level or otherwise) has been run against these modules. Actual savings depend on the gated logic's share of total design power, the technology library's clock-gate cell, and leakage, which clock gating does not reduce at all. Treat these numbers as a sizing aid and characterize your own instance before quoting a savings figure.

17.7.2 Write-Specific Considerations

Burst Handling: - W channel SKID depth (4) larger than AW/B (2) to accommodate burst data - Gating should not occur mid-burst (use adequate idle_count) - Burst latency unaffected if gating disabled during active bursts

Atomic Operations: - Atomic writes may have longer latencies - Consider higher idle_count when ENABLE_ATOMIC=1 - Gating safe between atomic transactions

17.7.3 Clock Gating Overhead

- **Area:** +5 flops, counted from the RTL – `r_wakeup` in `amba_clock_gate_ctrl` plus `r_idle_counter` at `IDLE_CNTR_WIDTH` (default 4), scaling as 1 + `CG_IDLE_COUNT_WIDTH`. Against a base module of a few hundred flops that is low single-digit percent. The ICG itself is a latch/BUFGCE, not fabric flops; the LUT delta is not measured
- **Timing:** Clock gating adds minimal delay (typically <50ps)
- **Power:** Overhead from gate control logic usually <1% of savings

17.7.4 When to Use Clock Gating

Good candidates: - Low-duty-cycle interfaces (sporadic transactions) - Systems with strict power budgets - Battery-operated devices - Write-infrequent applications (logging, configuration)

Avoid when: - Interface is continuously active (streaming writes) - Gate/ungate transitions exceed power savings - Deterministic latency required (use high idle count instead) - Write bursts dominate (minimal idle periods)

17.8 Related Modules

- **AXI5 Slave Write** - Non-clock-gated version
- **AXI5 Slave Read CG** - Clock-gated read variant
- **AXI5 Slave Write Monitor CG** - With monitoring
- **AMBA Clock Gate Control** - Clock gating controller

17.9 Testing

`val/amba/test_axi5_slave_wr_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_wr_cg.py -v
```

17.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A2.3.1 “Valid-Ready signals”, §A2.3.2 “Dependencies between channel handshake signals” and §A2.5 “Pipelining and register stages” (the transport this module buffers per channel), and the AXI5 signals it carries unchanged: §A6.4 “Atomic transactions” (AWATOP), §A16.1 “Data protection using Poison” (WPOISON), §A12.2 “Memory Tagging Extension (MTE)” (AWTAGOP, WTAG, WTAGUPDATE, BTAGMATCH), §A5.2 “Unique ID indicator”, §A12.3 “Trace signals”, §A10.1 “Non-secure Access Identifiers (NSAID)”, §A12.1 “Memory System Resource Partitioning and Monitoring (MPAM)” and §A4.6 “Memory Encryption Contexts” (AWMECID). The ports follow the specification’s names and widths (GitHub #102): BTAGMATCH is 2 bits with the Table A12.14 encodings, there is no BTAG or AWTAG, the Unique ID indicator is AWIDUNQ with BIDUNQ on the response, and AWAKEUP follows §A14.1.1 “AWAKEUP rules and recommendations”.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).

- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

17.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

18 AXI5 Slave Write Monitor

Module: `axi5_slave_wr_mon.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

18.1 Overview

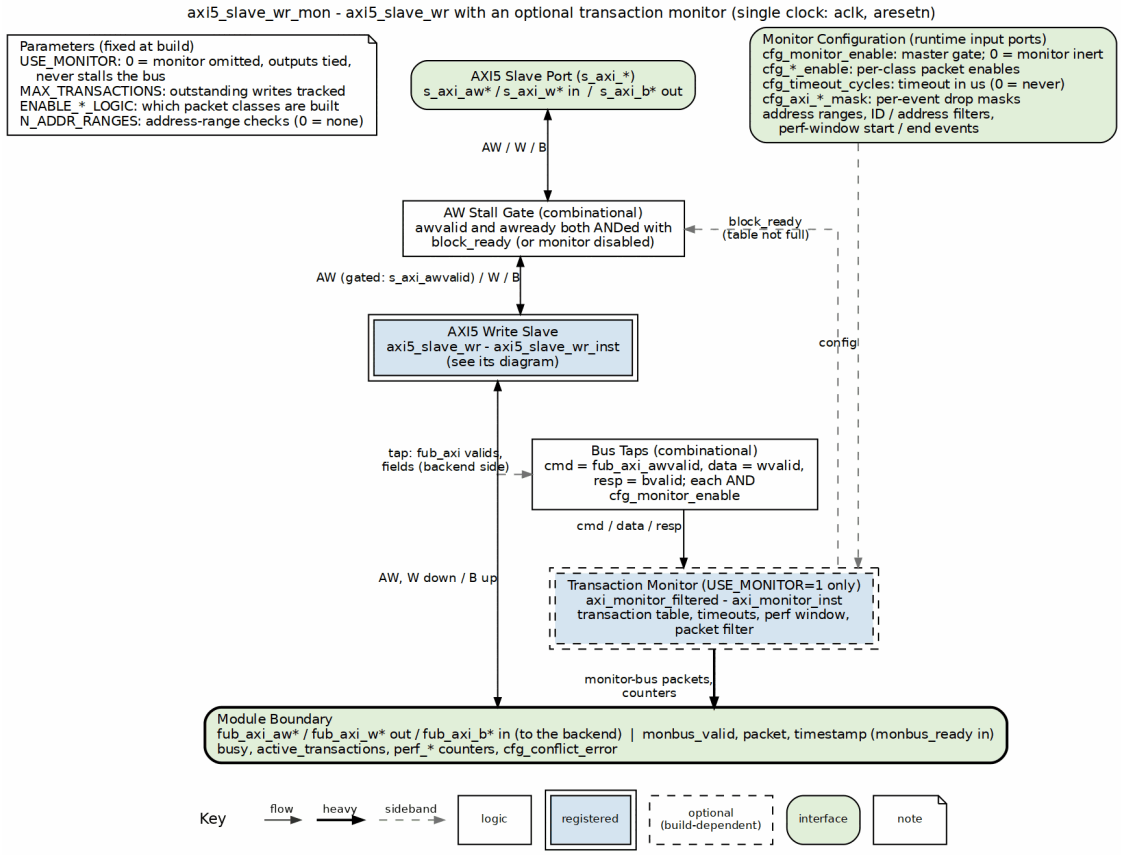
Most designs don't want a separate monitor bolted onto the bus — you want the slave to tell you what it's seeing. That's what this module does. It takes the `axi5_slave_wr` interface and builds the transaction monitor directly into the wrapper, giving you real-time visibility into slave write operations with configurable packet filtering and error detection. No external monitor block, no extra wiring beyond the monitor bus itself.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

18.1.1 Key Features

- Full AMBA AXI5 slave write protocol compliance
- **Integrated filtered monitoring** — no external monitor needed
- All AXI5 extensions supported (ATOMIC, NSAID, TRACE, MPAM, MECID, IDUNQ, MTE, POISON, WAKEUP)
- Transaction tracking with configurable table size
- Error detection (SLVERR, timeout, orphan transactions — poison is NOT observable by the monitor)
- Performance metrics (latency, throughput)
- Configurable packet filtering to reduce bandwidth
- 128-bit monitor bus packet output paired with 64-bit side-band timestamp
- Active transaction count tracking

18.1.2 Block Diagram



axi5_slave_wr_mon block diagram

The base `axi5_slave_wr` is one box (see its own diagram). Traffic runs down the left through the AW stall gate, which ANDs `s_axi_awvalid` and its ready with the `block_ready` the monitor raises while its transaction table has room. The bus taps watch the `fub_axi` (backend) side below the base, unlike the AXI4 slave monitors, which tap `s_axi` and feed `axi_monitor_filtered`, which is built only when `USE_MONITOR=1`; its monitor-bus packets are the heavy edge out. With `cfg_monitor_enable` low the monitor is inert and never stalls the bus.

18.2 Parameters

A lite-monitor version of this wrapper exists as `axi5_slave_wr_monlite`: the same core and taps with `axi_monitor_lite` instead of `axi_monitor_filtered`, about a fifth of the monitor gates, error/timeout/completion/threshold packets only. Selected in generated bridges by `mon_preset = "lite"`.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning

Parameter	Type	Default	Description
ENABLE_MECID	bit	1	Enable memory encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	13	Agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
ACLK_MHZ	int	100	Clock frequency in MHz — keeps the 1 us tick exact, including off-100MHz operation
CFI_MIN_FREQ_MHZ / CFI_MAX_FREQ_MHZ	int	= ACLK_MHZ	Freq-invariant counter LUT bounds (`cfg_freq_sel` indexes within them)
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue
NUM_BANKS	int	1	Banked transaction tables. **>1 on a WRITE monitor requires `USE_WDATA_ORDER_Q=1`** – `axi_monitor_trans_mgr` fails elaboration otherwise

Parameter	Type	Default	Description
ID_FILTER_ENABLE	bit	0	Synthesises the per-instance ID-slice filter
ID_MATCH_BASE	int	0	First ID this instance owns
ID_MATCH_COUNT	int	0	How many; `0` means ALL, so a zeroed register block does not silently filter everything away
ACTIVE_TRANS_THRESHOLD	int	MAX_TRANSACTIONS/2	Active-transaction count that trips a threshold packet when cfg_threshold_enable=1. Replaces the former hardwired 8/4; threshold packets now scale with the table sizing
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable. 0 = omit monitor and tie outputs to safe non-blocking defaults; 1 = full monitor functionality.
N_ADDR_RANGES	int	0	Number of address-range comparators. 0 = checker omitted (zero area). >0 = N independent [low, high] ranges; feeds the shared

Parameter	Type	Default	Description
			allowlist checker: a debug-range hit -> AddrMatch, an error-allowlist miss -> Error/ADDR_RANGE (see axi_monitor_addr_check.md).
ENABLE_ERROR_LOGIC	bit	1	Compile-in the error-detection cone (0 drops it for area).
ENABLE_TIMEOUT_LOGIC	bit	1	Compile-in the timeout cone and the `axi_monitor_timeout` instance.
ENABLE_COMPL_LOGIC	bit	1	Compile-in the completion cone.
ENABLE_THRESHOLD_LOGIC	bit	1	Compile-in the threshold cone.
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – the window FSM and bucket/beat/byte/burst counters are unconditional (always compiled, always live)
ENABLE_DEBUG_LOGIC	bit	0	Compile-in the debug cone (off by default).

Synthesis-cone note: the six `ENABLE_*_LOGIC` parameters gate each detection cone at synthesis via generate-if, so unused logic drops to zero area (classic cones default on, debug off). Inside the wrapper's `axi_monitor_filtered` instance the perf/debug master switches are fixed (`ENABLE_PERF_PACKETS = 1`, `ENABLE_DEBUG_MODULE = 0`). The former

CAM_PIPELINE / TRANS_CAM_PIPELINE parameters were removed — the transaction CAM is now always pipelined.

18.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS

18.3 Ports

18.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

18.3.2 Slave AXI5 Interface

Identical to `axi5_slave_wr` — see AXI5 Slave Write for the complete port list.

18.3.3 FUB Interface

Identical to `axi5_slave_wr` — see AXI5 Slave Write for the complete port list.

18.3.4 Monitor Configuration

Port	Width	Direction	Description
cfg_monitor_enable	1	Input	Master runtime gate. 0 = monitor inert: no allocation, transaction CAM held clear, and the upstream ready is never stalled (a disabled monitor cannot block the datapath). 1 = normal operation
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (driven from the harness clear control bit, e.g. CTRL[4])
cfg_error_enable	1	Input	Enable error packets
cfg_timeout_enable	1	Input	Enable timeout packets
cfg_perf_e	1	Input	Enable performance

Port	Width	Direction	Description
nable			packets (see Performance Monitoring)
cfg_compl_enable	1	Input	Enable transaction-completion packets
cfg_threshold_enable	1	Input	Enable threshold-crossed packets
cfg_debug_enable	1	Input	Enable debug/trace packets (gates the debug cone — the 6th reporter sub-block)
cfg_timeout_cycles	16	Input	Unified coarse timeout control, a MICROSECOND count passed through at FULL 16-bit width: 1..65535 us per phase; 0 = 16'hFFFF (~65 ms, effectively never). One value drives all three phase counts (addr/data/resp). The old 4-bit squash that saturated >= 16 at 15 us is retired
cfg_latency_threshold	32	Input	High latency threshold (cycles)
cfg_freq_sel	4	Input	`counter_freq_invariant` LUT index scaling the 1 us timer tick
cfg_axi_pkt_mask	16	Input	Packet type filter mask
cfg_axi_err_select	16	Input	Error selection mask
cfg_axi_error_mask	16	Input	Error event filter
cfg_axi_timeout_mask	16	Input	Timeout event filter

Port	Width	Direction	Description
cfg_axi_compl_mask	16	Input	Completion event filter
cfg_axi_thresh_mask	16	Input	Threshold event filter
cfg_axi_perf_mask	16	Input	Performance event filter
cfg_axi_addr_mask	16	Input	Address event filter
cfg_axi_debug_mask	16	Input	Debug event filter

Detection-cone enables: `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` turn on the completion, threshold, and debug reporter sub-blocks respectively. `cfg_debug_enable` gates the **debug cone** — the 6th reporter sub-block (`axi_monitor_reporter_debug`). In this wrapper the debug module's `cfg_debug_level` (4) and `cfg_debug_mask` (16) inputs are tied to `4'h0` / `16'h0`; `cfg_active_trans_threshold` is driven from the `ACTIVE_TRANS_THRESHOLD` parameter (default `MAX_TRANSACTIONS/2`). None are wrapper ports.

18.3.5 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	`monitor_packet_t` (see format below)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
debug_block_ready	1	Output	Observability tap for the block_ready gating net (drives nothing internally; leave unconnected if unused)

Port	Width	Direction	Description
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

18.3.6 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Lifetime count of error+timeout packets actually emitted (reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
transaction_count	32	Output	Lifetime count of completion packets actually emitted (zero-extended 16-bit reporter perf counter; reads 0 when `ENABLE_PERF_LOGIC=0` or `USE_MONITOR=0`)
cfg_conflict_error	1	Output	Configuration conflict detected

18.3.7 Packet filters

Two independent runtime filters cut monbus traffic without touching the datapath. Both are inert until driven, which is the failure to watch for: the build-time parameter only decides whether the logic is SYNTHESISED, and a design that sets the parameter but leaves the `cfg_*` ports tied low filters nothing and looks like the feature is broken.

Address-range filter — `ADDR_FILTER_ENABLE` (bit, default 0) builds it.

Port	Width	Description
cfg_addr_filter_enable	1	High: suppress packets for transactions outside the window. Low: inert, whatever the parameter says
cfg_addr_filter_low	ADDR_WIDTH	Window base, inclusive
cfg_addr_filter_high	ADDR_WIDTH	Window limit, inclusive

The verdict is latched per table entry at ALLOCATION, from the command address, and held for that entry's life. Widening the window mid-flight does not un-filter entries already admitted — which is exactly what makes it safe to reprogram live, since an entry's fate is decided when it is admitted and the retire accounting cannot be corrupted afterwards. Contrast the address-range CHECKER (N_ADDR_RANGES), which re-evaluates per command.

Runtime ID filter — overrides the ID_MATCH_BASE/ID_MATCH_COUNT elaboration constants so an instance can be retargeted at a different master without a rebuild.

Port	Width	Description
cfg_id_filter_enable	1	High: use the window below. Low: use the parameters, bit-identical to a build without this feature
cfg_id_match_base	ID_WIDTH	First ID owned
cfg_id_match_count	ID_WIDTH+1	How many; 0 means ALL, matching the parameter rule so a zeroed register block does not silently filter everything away

The window is half-open: [base, base + count).

18.4 Functional Description

18.4.1 Monitor Backpressure (block_ready)

`block_ready` is the monitor's throttle signal, and the wrapper deliberately exports it as the `debug_block_ready` output port so the gating contract stays observable (the `_mon_cg` wrapper ties it off rather than forwarding it). It goes low when the monitor's transaction-table occupancy reaches its blocking threshold — a function of `MAX_TRANSACTIONS`; the reporter FIFO depth `INTR_FIFO_DEPTH` has no path to it. The wrapper ANDs it into the upstream-facing `s_axi_awready`, so a saturated monitor throttles new transactions at the handshake instead of dropping events.

- **Where the stall lands**: the upstream `s_axi_awready` is forced low until the monitor drains.
- **When `USE_MONITOR=0`**: `block_ready` is internally tied high, so the wrapper imposes no stall and runs at full bandwidth.
- **When `cfg_monitor_enable=0`**: the wrapper gate forces the upstream ready open, so a runtime-disabled monitor can never stall the datapath (the AXI4 monitors assert this as an in-RTL formal property, `ap_disabled_never_stalls`; this module has no `ifdef FORMAL` block of its own, so here the guarantee rests on the gate expression above, not a proof).
- **For axi5 slave variants**: the monitor watches the FUB-side handshake, so there is a `SKID_DEPTH_AW` cycle lag between block_ready going low and new events ceasing. `MAX_TRANSACTIONS` should be sized to cover this margin.

Recovery is guaranteed by the **saturation-recovery contract** — and this is the clever part. Command-originated table entries are capped at `MAX_TRANSACTIONS - cmd_entry_reserve(MAX_TRANSACTIONS)` (reserve = 4 for tables of 16 or more, 0 below; the function lives in `ocelus_common_pkg`), and `block_ready` re-asserts at occupancy < `MAX_TRANSACTIONS - (reserve - 1)` — a threshold strictly ABOVE the command cap — so a saturated table always drains back below the reopen point. Blocking throttles; it never deadlocks. Tables smaller than 16 keep full legacy allocation (small tables cannot spare slots) and trade the recovery guarantee for tracking capacity. The contract is verified by in-RTL formal properties (mutation-checked) and a 100-seed deliberately-undersized-table stream sweep; see axi_monitor_base for the canonical description.

Sizing note: a monitor on a bus shared by several channels/requesters must size `MAX_TRANSACTIONS` to cover `NUM_CHANNELS x per-channel outstanding` plus margin — the per-channel limit alone makes the monitor throttle the shared master. Tables deeper than 64 also need Verilator's `--unroll-count` raised (default 64) in sim builds.

18.4.2 Address-Range Checker

With `N_ADDR_RANGES > 0` the wrapper instantiates the shared allowlist checker (`axi_monitor_addr_check`). Each range carries a DEBUG/ERROR flavor:

- **DEBUG range** — a hit emits a `PktTypeAddrMatch` (`4'h8`) packet with event code `AXI_ADDR_RANGE_MATCH` (`8'h01`), gated by `cfg_debug_enable`.
- **ERROR range** — the enabled ERROR ranges form an allowlist; an address in NONE of them emits a `PktTypeError` (`4'h0`) packet with event code `AXI_ERR_ADDR_RANGE` (`8'h0D`), gated by `cfg_error_enable`.

This wrapper does not expose `ADDR_RANGE_IS_ERROR`; all ranges default to the DEBUG (match) flavor.

Config inputs (active only when `N_ADDR_RANGES > 0`): - `cfg_addr_check_enable` — master on/off for the checker. - `cfg_addr_range_enable[N-1:0]` — per-range enable bit. - `cfg_addr_range_low/high[N-1:0][AXI_ADDR_WIDTH-1:0]` — inclusive bounds.

Event encoding: `event_data[63:60]` = `range_index` (matching DEBUG range, or `4'hF` sentinel on an ERROR miss); `event_data[59:0]` = full address. **Filtering:** AddrMatch is dropped by `cfg_axi_addr_mask[1]`, the ADDR_RANGE error by `cfg_axi_error_mask[13]`. See the checker page for coalescing + formal properties.

18.4.3 Monitor Packet Format

The 128-bit `monbus_packet` (paired with the 64-bit `monbus_timestamp` side-band signal) follows the standardized AMBA monitor bus format:

```
Bits [127:124] - Packet Type:
  0x0 = ERROR      Error events (SLVERR, DECERR, protocol violations)
  0x1 = COMPL      Completion events (transaction finished)
  0x2 = THRESH     Threshold events
  0x3 = TIMEOUT    Timeout events
  0x4 = PERF       Performance metrics
  0x8 = ADDR_MATCH Address match events
  0x9 = APB        APB-specific events
  0xF = DEBUG      Debug events
Bits [123:109] - Reserved (15 bits, forward-compat slack)
Bits [108:105] - Protocol (4 bits): 0x0=AXI, 0x1=AXIS, 0x2=APB,
0x3=ARB, 0x4=CORE
Bits [104:97]  - Event Code (8 bits, protocol-specific)
Bits [96:88]   - Channel ID (9 bits – AXI ID or channel index)
Bits [87:72]   - Agent ID (16 bits, from AGENT_ID parameter)
Bits [71:64]   - Unit ID (8 bits, from UNIT_ID parameter)
Bits [63:0]    - Event Data (64 bits – full address, latency, etc.)
```

18.4.4 Event Types

18.4.4.1 Error Packets (Type=0)

Event Code	Description	Event Data
0x1	SLVERR response	Zero-extended 32-bit ADDRESS
0x2	DECERR response	Zero-extended 32-bit ADDRESS
0x3	Orphan data/response	Zero-extended 32-bit ADDRESS
0x4	Protocol violation	Zero-extended 32-bit ADDRESS

(Event codes 0x5 “poison”, 0x6 “tag mismatch” and 0x7 “missing WLAST” were documented here but are NOT implemented — no poison/tag/WLAST signal reaches the monitor; these codes are never emitted.)

18.4.4.2 Completion Packets (Type=1)

Event Code	Description	Event Data
0x0	Write completion	Zero-extended 32-bit ADDRESS (the reporter cones carry address only – no ID/length/latency in event_data)

18.4.4.3 Timeout Packets (Type=3)

Event Code	Description	Event Data
0x1	AW channel timeout	Zero-extended 32-bit ADDRESS (no ID/cycle count in event_data)
0x2	W channel timeout	Zero-extended 32-bit ADDRESS
0x3	B channel timeout	Zero-extended 32-bit ADDRESS

18.4.4.4 Performance Packets (Type=4)

Event Code	Description	Event Data
(only two exist)	AXI_PERF_COMPLETED_COUNT	64’(completed-packet lifetime count)
	AXI_PERF_ERROR_COUNT	64’(error-packet lifetime count)

(High-latency / bandwidth-sample / outstanding-count packets were never implemented — the reporter’s own comment marks those states placeholders. Latency and utilization data come from the perfmon WINDOW outputs, not packets.)

18.4.5 Performance Monitoring

When performance tracking is compiled in (`ENABLE_PERF_LOGIC = 1`, the wrapper default, with `ENABLE_PERF_PACKETS` fixed to 1 inside the monitor instance) and `cfg_perf_enable` is asserted at runtime, the monitor runs a **measurement-window state machine** plus a bank of data-channel utilization counters. All counters accumulate **only while a window is open** (`window_active = 1`) and hold their values between windows so the host can read a completed window’s totals.

18.4.5.1 The Measurement Window

A window is opened by a **start event** and closed by an **end event**. The event sources are selected by `cfg_start_event_sel` / `cfg_end_event_sel` (3-bit; e.g. `3'b010` selects the `cfg_perf_enable` edge) and can also be fired directly by the `cfg_start_trigger` / `cfg_end_trigger` pulses (from an engine or CSR write). `cfg_window_force_close` is a software override that closes the window immediately. While the window is open:

- `window_active` is high.
- `window_cycles` free-runs, counting every clock elapsed inside the window.

18.4.5.2 Utilization Buckets (W Data Channel)

Every cycle inside the window is classified by the **W** data channel’s valid/ready handshake into exactly one of four buckets:

Output	Width	Condition	Meaning
`perf_prod_cycles`	32	`wvalid && wready`	productive beat transferred
`perf_bp_cycles`	32	`wvalid && !wready`	back-pressure (data offered, sink not ready)
`perf_starv_cycles`	32	`!wvalid && wready`	starvation (sink

Output	Width	Condition	Meaning
			ready, no data)
`perf_idle_cycles`	32	`!wvalid && !wready`	idle

The four buckets sum to `window_cycles - 1` (the start cycle seeds window_cycles to 1 while the buckets reset to 0); utilization = `perf_prod_cycles / window_cycles` is off by one count — negligible for long windows, use `window_cycles - 1` for exactness.

18.4.5.3 Throughput Counters

Output	Width	Meaning
`perf_beat_count`	32	W data beats transferred (= `perf_prod_cycles`, 1 beat/cycle)
`perf_byte_count`	64	bytes transferred = beats x (1 << latched AWSIZE), using the AWSIZE captured at the most recent AW address phase
`perf_burst_count`	32	AW address-phase handshakes

Average burst length is `perf_beat_count / perf_burst_count`. (Transaction completion is still tracked on the B channel; the throughput counters measure the W data phase.)

18.4.5.4 Performance Monitoring Ports

Port	Width	Direction	Description
cfg_perf_enable	1	Input	Enable performance-metric packet generation (also listed under Monitor Configuration)
cfg_start_event_sel	3	Input	Window **start** event source select
cfg_end_event_sel	3	Input	Window **end** event source select
cfg_start_trigger	1	Input	Pulse: open the measurement window
cfg_end_tri	1	Input	Pulse: close the

Port	Width	Direction	Description
gger			measurement window
cfg_window_force_close	1	Input	Software override: force the window closed
window_active	1	Output	High while a measurement window is open
window_cycles	32	Output	Cycles elapsed in the current window
perf_prod_cycles	32	Output	`wvalid && wready` cycles
perf_bp_cycles	32	Output	`wvalid && !wready` cycles (back-pressure)
perf_starv_cycles	32	Output	`!wvalid && wready` cycles (starvation)
perf_idle_cycles	32	Output	`!wvalid && !wready` cycles
perf_beat_count	32	Output	W data beats transferred
perf_byte_count	64	Output	bytes transferred
perf_burst_count	32	Output	AW address-phase handshakes

When `USE_MONITOR = 0` all perfmon outputs are tied to 0. With `USE_MONITOR = 1` and `ENABLE_PERF_LOGIC = 0`, only error_count / transaction_count read 0 — the window and bucket/throughput outputs remain LIVE.

18.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one gaxi_skid_buffer, which registers both rd_valid and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: aclk, reset aresetn (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

18.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_slave_wr_mon #(
    .SKID_DEPTH_AW          (2),
    .SKID_DEPTH_W           (4),
    .SKID_DEPTH_B           (2),
    .AXI_ID_WIDTH           (8),
    .AXI_ADDR_WIDTH         (32),
    .AXI_DATA_WIDTH         (32),
    .AXI_USER_WIDTH         (1),
    .AXI_ATOP_WIDTH         (6),
    .AXI_NSAID_WIDTH        (4),
    .AXI_MPAM_WIDTH         (11)
) u_axi5_slave_wr_mon (
    .aclk                   (aclk),
    .aresetn                (aresetn),
    .cam_clear              (cam_clear),
    .s_axi_awid             (s_axi_awid),
    .s_axi_awaddr           (s_axi_awaddr),
    .s_axi_awlen            (s_axi_awlen),
    .s_axi_awsize           (s_axi_awsize),
    .s_axi_awburst          (s_axi_awburst),
    .s_axi_awlock           (s_axi_awlock),
    .s_axi_awcache          (s_axi_awcache),
    .s_axi_awprot           (s_axi_awprot),
    .s_axi_awqos            (s_axi_awqos),
    .s_axi_awuser           (s_axi_awuser),
    .s_axi_awvalid          (s_axi_awvalid),
    .s_axi_awakeup          (s_axi_awakeup),
    .s_axi_awready          (s_axi_awready),
    .s_axi_awatop           (s_axi_awatop),
    .s_axi_awnsaid          (s_axi_awnsaid),
    .s_axi_awtrace          (s_axi_awtrace),
    .s_axi_awmpam           (s_axi_awmpam),
    .s_axi_awmecid          (s_axi_awmecid),
    .s_axi_awidunq         (s_axi_awidunq),
    .s_axi_awtagop          (s_axi_awtagop),
    .s_axi_wdata            (s_axi_wdata),
    .s_axi_wstrb            (s_axi_wstrb),
    .s_axi_wlast            (s_axi_wlast),
    .s_axi_wuser            (s_axi_wuser),
    .s_axi_wvalid           (s_axi_wvalid),
    .s_axi_wready           (s_axi_wready),
    .s_axi_wpoison          (s_axi_wpoison),
    .s_axi_wtag             (s_axi_wtag),
    .s_axi_wtagupdate       (s_axi_wtagupdate),
```

```
    .s_axi_bid             (s_axi_bid),
    .s_axi_bresp           (s_axi_bresp),
    .s_axi_buser           (s_axi_buser),
    .s_axi_bvalid          (s_axi_bvalid),
    .s_axi_bready          (s_axi_bready),
    .s_axi_btrace          (s_axi_btrace),
    .s_axi_bidunq          (s_axi_bidunq),
    .s_axi_btagmatch       (s_axi_btagmatch),
    .fub_axi_awid          (fub_axi_awid),
    .fub_axi_awaddr        (fub_axi_awaddr),
    .fub_axi_awlen         (fub_axi_awlen),
    .fub_axi_awsize        (fub_axi_awsize),
    .fub_axi_awburst       (fub_axi_awburst),
    .fub_axi_awlock        (fub_axi_awlock),
    .fub_axi_awcache       (fub_axi_awcache),
    .fub_axi_awprot        (fub_axi_awprot),
    .fub_axi_awqos         (fub_axi_awqos),
    .fub_axi_awuser        (fub_axi_awuser),
    .fub_axi_awvalid       (fub_axi_awvalid),
    .fub_axi_awakeup       (fub_axi_awakeup),
    .fub_axi_awready       (fub_axi_awready),
    .fub_axi_awatop        (fub_axi_awatop),
    .fub_axi_awnsaid       (fub_axi_awnsaid),
    .fub_axi_awtrace       (fub_axi_awtrace),
    .fub_axi_awmpam        (fub_axi_awmpam),
    .fub_axi_awmecid       (fub_axi_awmecid),
    .fub_axi_awidunq      (fub_axi_awidunq),
    .fub_axi_awtagop       (fub_axi_awtagop),
    .fub_axi_wdata         (fub_axi_wdata),
    .fub_axi_wstrb         (fub_axi_wstrb),
    .fub_axi_wlast         (fub_axi_wlast),
    .fub_axi_wuser         (fub_axi_wuser),
    .fub_axi_wvalid        (fub_axi_wvalid),
    .fub_axi_wready        (fub_axi_wready),
    .fub_axi_wpoison       (fub_axi_wpoison),
    .fub_axi_wtag          (fub_axi_wtag),
    .fub_axi_wtagupdate    (fub_axi_wtagupdate),
    .fub_axi_bid           (fub_axi_bid),
    .fub_axi_bresp         (fub_axi_bresp),
    .fub_axi_buser         (fub_axi_buser),
    .fub_axi_bvalid        (fub_axi_bvalid),
    .fub_axi_bready        (fub_axi_bready),
    .fub_axi_btrace        (fub_axi_btrace),
    .fub_axi_bidunq        (fub_axi_bidunq),
    .fub_axi_btagmatch     (fub_axi_btagmatch),
    .cfg_monitor_enable    (cfg_monitor_enable),
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
```

```
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .debug_block_ready     (debug_block_ready),
    .cfg_conflict_error    (cfg_conflict_error),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
```

```
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

18.7 Design Notes

18.7.1 W-Channel Wiring into the Monitor Core

The wrapper feeds the shared `axi_monitor_base` data channel from the W channel with `data_id = AWID` (the W channel itself carries no WID in AXI4/AXI5) and `data_resp = 2'b00` (write data carries no response; the response arrives on B). This matches the AXI4 write-monitor convention. Before commit `95c9490a` the AXI5 write monitors wired `data_id = BID` and `data_resp = BRESP` onto W beats (cross-channel), which mis-keyed write-data attribution; that defect is fixed.

18.7.2 Write-Specific Monitoring

Poison Detection: NOT implemented — WPOISON never reaches the monitor; no poison packet can be emitted.

Burst Completions: - Tracks AWLEN to verify complete bursts - (Missing-WLAST detection is NOT implemented — WLAST drives completion, not validation) - Monitors AW/W channel synchronization

Atomic Operations: - AWATOP is NOT monitored (ENABLE_ATOMIC shapes the transport only) - (Atomic latency is not separately tracked — atomics time out / complete like ordinary writes) - (Atomic failures surface only as ordinary SLVERR/DECERR — no ATOP-specific detection)

18.7.3 Best Practices

- Monitor packets provide real-time transaction visibility without external logic
- Filtering reduces monitor bus bandwidth — critical for high-throughput systems
- Transaction table size (MAX_TRANSACTIONS) must accommodate peak outstanding transactions
- Performance packets can generate high traffic — use sparingly or with filtering
- UNIT_ID and AGENT_ID identify this monitor in multi-agent systems
- Error count WRAPS at 16'hFFFF (plain increment, no saturation — the perfmon BUCKET counters are the ones that saturate)

18.8 Related Modules

- **AXI5 Slave Write** - Non-monitored version
- **AXI5 Slave Write Monitor CG** - Clock-gated variant
- **AXI5 Slave Read Monitor** - Read monitor
- **AXI Monitor Filtered** - Monitor core
- **Monitor Package Spec** - Packet format details

18.9 Testing

`val/amba/test_axi5_slave_wr_mon.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_wr_mon.py -v
```

18.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- The monitor’s own techniques (transaction table, timeouts, packet filtering) are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Maaref, M. *Architecting and Building High-Speed SoCs*, Packt, 2022, ch. 3 “Basic and Advanced On-Chip Busses and Interconnects” (“AXI bus protocol overview”) – textbook overview of AXI responses and ID ordering (secondary source).
- Indexed in `rtl/amba/references/README.md` (protocol specifications are citation only)

18.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index

19 AXI5 Slave Write Monitor with Clock Gating

Module: `axi5_slave_wr_mon_cg.sv` **Location:** `rtl/amba/axi5/` (protocol monitors live with their protocol; only the monitor CORE pieces are in `rtl/amba/ocelus/`) **Status:** Production Ready

---

19.1 Overview

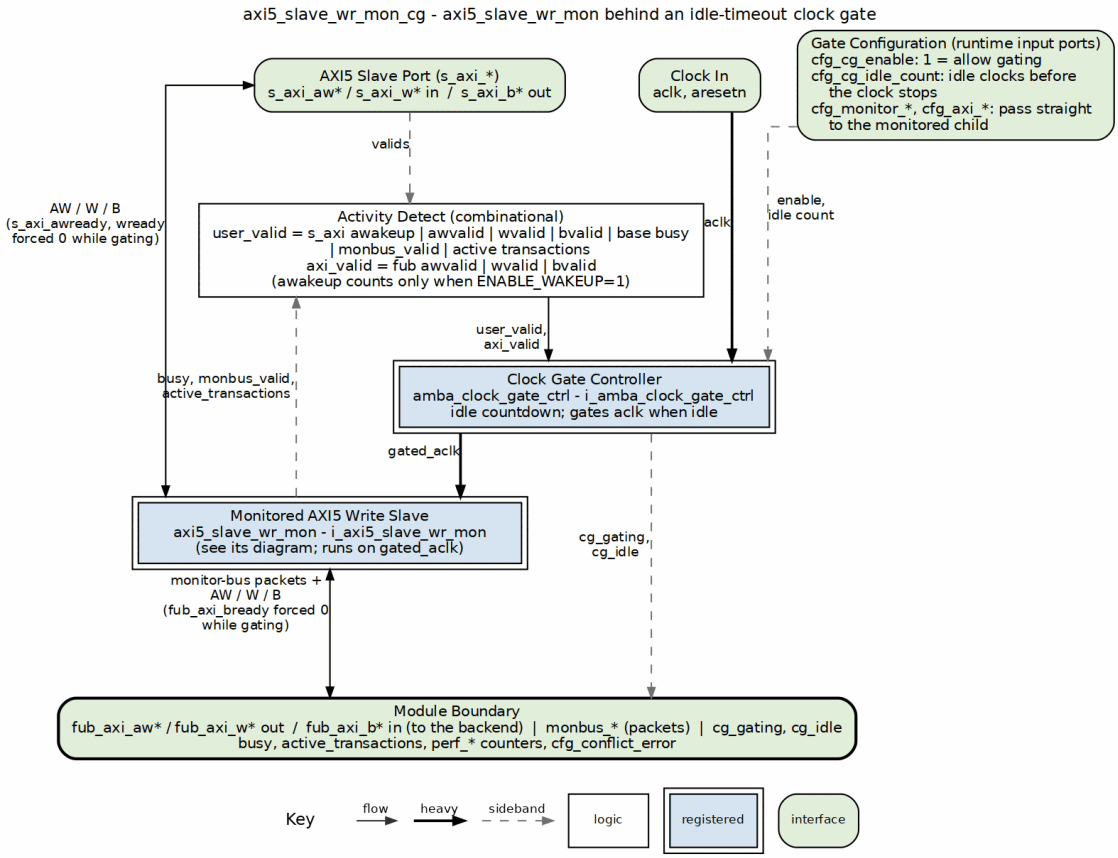
Same slave, same monitor — plus a clock gate. This module wraps `axi5_slave_wr_mon` with automatic clock gating logic, so you keep the integrated transaction monitoring and claw back dynamic power whenever the bus goes quiet.

Scope: this module transports AXI5 signals; it does not implement AXI5 transaction semantics. It performs no MTE tag checking, no chunk reassembly, no poison generation, and no atomic read-modify-write – `AWATOP` is transported, not executed. The monitor observes handshakes, responses and timing; it performs no protocol checking of handshake stability, ID width, burst length or address alignment. See Scope of This Implementation in the AXI5 index for the full coverage statement.

19.1.1 Key Features

- Full AMBA AXI5 slave write protocol compliance
- **Integrated filtered monitoring** — real-time transaction visibility
- **Integrated clock gating** for dynamic power reduction
- All AXI5 extensions supported (ATOMIC, NSAID, TRACE, MPAM, MECID, IDUNQ, MTE, POISON, WAKEUP)
- Configurable idle count before gating
- Transaction tracking with error detection
- Performance metrics and filtering
- Transparent gating — no protocol changes
- Gating status outputs for system monitoring
- **Power Savings:** traffic-dependent; unmeasured in this repo – treat any percentage as a placeholder until characterized

19.1.2 Block Diagram



axi5_slave_wr_mon_cg block diagram

The base `axi5_slave_wr_mon` is one box (see its own diagram). The clock is the heavy main column: `aclk` enters `amba_clock_gate_ctrl`, which counts down idle cycles and drives `gated_aclk` into the child. AXI traffic passes straight through; while the clock is gated the wrapper forces the incoming readies to 0 so no handshake is lost. Activity is `s_axi` valids, the child's busy, and `s_axi_awakeup` when `ENABLE_WAKEUP=1`, on the upstream side, plus the `fub_axi` valids. The monitored child's `monbus_valid` and active-transaction count also count as activity, so the clock never stops under a pending packet.

19.2 Parameters

A lite-monitor version of this clock-gated wrapper exists as `axi5_slave_wr_monlite_cg`: the same gating logic around `axi5_slave_wr_monlite` instead of `axi5_slave_wr_mon`, about a fifth of the monitor gates.

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel SKID buffer depth
SKID_DEPTH_W	int	4	W channel SKID buffer depth
SKID_DEPTH_B	int	2	B channel SKID buffer depth
AXI_ID_WIDTH	int	8	Transaction ID width
AXI_ADDR_WIDTH	int	32	Address bus width
AXI_DATA_WIDTH	int	32	Data bus width
AXI_USER_WIDTH	int	1	User signal width
AXI_ATOP_WIDTH	int	6	Atomic operation width
AXI_NSAID_WIDTH	int	4	Non-secure access ID width
AXI_MPAM_WIDTH	int	11	MPAM width
AXI_MECID_WIDTH	int	16	Memory encryption context width
AXI_TAG_WIDTH	int	4	Memory tag width per 16 bytes
AXI_TAGOP_WIDTH	int	2	Tag operation width
ENABLE_ATOMIC	bit	1	Enable atomic operations
ENABLE_NSAID	bit	1	Enable non-secure access ID
ENABLE_TRACE	bit	1	Enable trace signals
ENABLE_MPAM	bit	1	Enable memory partitioning
ENABLE_MECID	bit	1	Enable memory

Parameter	Type	Default	Description
			encryption
ENABLE_UNIQUE	bit	1	Enable the unique ID indicator (AxIDUNQ, and BIDUNQ/RIDUNQ on the responses)
ENABLE_MTE	bit	1	Enable Memory Tagging Extension
ENABLE_POISON	bit	1	Enable poison indicator
ENABLE_WAKEUP	bit	1	Enable AWAKEUP (IHI 0022L §A14.1.1); 0 ties the output low
UNIT_ID	int	1	Monitoring unit identifier
AGENT_ID	int	13	Agent identifier
MAX_TRANSACTIONS	int	16	Transaction table size
ENABLE_FILTERING	bit	1	Enable packet filtering: two active drop levels (packet type, then event code). Level 2 is reserved and routes nothing
ADD_PIPELINE_STAGE	bit	0	Add pipeline stage for timing
USE_MONITOR	bit	1	Synthesis-time monitor enable (forwarded to inner monitor).
N_ADDR_RANGES	int	0	Number of address-range comparators (forwarded to base module).
ENABLE_ERROR_LOGIC	bit	1	Synthesis-cone enable for error detection (forwarded to base module)

Parameter	Type	Default	Description
ENABLE_TIMEOUT_LOGIC	bit	1	Synthesis-cone enable for timeout detection (forwarded to base module)
ENABLE_COMPL_LOGIC	bit	1	Synthesis-cone enable for completion tracking (forwarded to base module)
ENABLE_THRESHOLD_LOGIC	bit	1	Synthesis-cone enable for threshold detection (forwarded to base module)
ENABLE_PERF_LOGIC	bit	1	Synthesis-cone enable for the REPORTER's legacy perf-packet cone and the two lifetime counters only – window FSM and meters are unconditional
ENABLE_DEBUG_LOGIC	bit	0	Synthesis-cone enable for the debug/trace cone (forwarded to base module)
`ACLK_MHZ`	int	100	Clock frequency in MHz. Builds the microsecond tick LUT in `counter_freq_invariant`. **Leave this at 100 on a 90 MHz part and every us-denominated timeout is wrong, silently** – it was unreachable through this wrapper until 2026-09-01.
`CFI_MIN_FREQ_MHZ`	int	`ACLK_MHZ`	Lowest frequency the

Parameter	Type	Default	Description
			tick LUT must cover (dynamic-frequency builds).
CFI_MAX_FREQ_MHZ	int	ACLK_MHZ	Highest frequency the tick LUT must cover.
USE_WDATA_ORDER_Q	bit	0	Write-data ordering queue. Required (=1) whenever NUM_BANKS > 1.
NUM_BANKS	int	1	Transaction-table banking. >**1 on this WRITE monitor requires USE_WDATA_ORDER_Q=1** – axi_monitor_trans_mgr fails elaboration otherwise (the WID-less fallback double-counts one W beat across banks).
ADDR_FILTER_ENABLE	bit	0	Synthesises the address-range report filter. **The parameter only decides whether the logic EXISTS** – a build that sets it and leaves cfg_addr_filter_enable low filters nothing and looks broken.
ID_FILTER_ENABLE	bit	0	Synthesises the ID report filter (see cfg_id_* for the runtime override).
ID_MATCH_BASE	int	0	First ID this instance owns.
ID_MATCH_COUNT	int	0	How many IDs; 0 means ALL, so a zeroed register

Parameter	Type	Default	Description
			block does not silently filter everything away.
CG_IDLE_COUNT_WIDTH	int	4	Clock gating idle counter width

19.2.1 Derived Parameters (do not override)

These are declared as `parameter` so the elaborator can compute them, not so callers can set them. Each defaults to an expression over the parameters above; overriding one desynchronises it from its source and the design fails to elaborate or silently mis-sizes a bus. Set the parameters they are derived FROM and leave these alone.

Derived parameter	Default expression
AXI_WSTRB_WIDTH	AXI_DATA_WIDTH / 8
DW	AXI_DATA_WIDTH
IW	AXI_ID_WIDTH
SW	AXI_WSTRB_WIDTH
UW	AXI_USER_WIDTH
NUM_TAGS	(AXI_DATA_WIDTH / 128) > 0 ? (AXI_DATA_WIDTH / 128) : 1
TW	AXI_TAG_WIDTH * NUM_TAGS

19.3 Ports

19.3.1 Filter configuration (forwarded)

These reach the inner monitor through this wrapper; before 2026-09-01 they did not.

Port	Direction	Width	Description
`cfg_addr_filter_enable`	Input	1	High: suppress packets for transactions outside the window. Low: inert, whatever `ADDR_FILTER_ENABLE` says
`cfg_addr_filter_low`	Input	ADDR_WIDTH	Window base, inclusive
`cfg_addr_filter_high`	Input	ADDR_WIDTH	Window limit, inclusive
`cfg_id_filter_enable`	Input	1	High: use the runtime window below instead of the `ID_MATCH_*` parameters
`cfg_id_match_base`	Input	ID_WIDTH	First ID to accept
`cfg_id_match_count`	Input	ID_WIDTH+1	How many; `0` means ALL

19.3.2 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock (ungated)
aresetn	1	Input	AXI active-low reset
cam_clear	1	Input	Synchronous clear of the monitor transaction CAM (do not leave unconnected)

19.3.3 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

19.3.4 Slave AXI5 Interface

Identical to `axi5_slave_wr` — see AXI5 Slave Write for the complete port list.

19.3.5 FUB Interface

Identical to `axi5_slave_wr` — see AXI5 Slave Write for the complete port list.

19.3.6 Monitor Configuration

Identical to `axi5_slave_wr_mon` — see AXI5 Slave Write Monitor for the complete list.

19.3.7 Monitor Bus Output

Port	Width	Direction	Description
monbus_valid	1	Output	Monitor packet valid
monbus_ready	1	Input	Monitor packet ready
monbus_packet	128	Output	`monitor_packet_t` (128-bit; [127:124] type, [104:97] event code – full layout on the base mon page)
monbus_timestamp	64	Output	`monbus_timestamp_t` paired atomically with `monbus_packet`
i_mon_time	64	Input	Free-running counter from `monbus_group_core`, sampled at packet emission

19.3.8 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy indicator
active_transactions	8	Output	Current outstanding transactions
error_count	16	Output	Errors whose packets were actually EMITTED – counts error AND timeout packets emitted – with cfg_error_enable off it still counts timeout packets; only ENABLE_PERF_LOGIC=0 (or USE_MONITOR=0) forces 0
transaction_count	32	Output	Completion packets actually EMITTED – with cfg_compl_enable off (this page's own perf-mode recipe) it reads 0 while transactions complete normally
cfg_conflict_error	1	Output	Configuration conflict detected
cg_gating	1	Output	Clock is currently gated
cg_idle	1	Output	Module is idle

19.4 Functional Description

19.4.1 Clock Gating Behavior

Activity Detection: - **user_valid:** Asserted when slave interface has activity (AWAKEUP, awvalid, wvalid, bvalid, internal busy, a monitor packet pending on the monitor bus, or monitor CAM entries still live — peer VALID, never peer READY) - **axi_valid:** Asserted when FUB interface has activity (awvalid, wvalid, bvalid)

Key Points: - Clock gating disabled when `cfg_cg_enable = 0` - Ready signals (awready, wready) forced to 0 when gated (prevents new transactions) - bready forced to 0 when gated (prevents accepting responses) - Gating only occurs after configured idle period - Any activity immediately ungates the clock - Monitor state IS a wake term: both a packet pending on the monitor bus (`w_monbus_valid`) and live monitor CAM entries (`|active_transactions`, which covers the reporter's few-cycle emission window) hold the block awake, and the external `monbus_valid` is masked with `!cg_gating` – so monitor-bus delivery is exactly-once across gating at any idle count (`val/amba/test_mon_cg_gating.py` phase 6)

19.4.2 Performance Monitoring

The clock-gated wrapper exposes the full perfmon interface of the base module and **forwards every port unchanged** to the inner `axi5_slave_wr_mon`. The measurement-window state machine, the four W-channel utilization buckets (productive / back-pressure / starvation / idle), and the beat/byte/burst throughput counters behave exactly as documented in the base module — see Performance Monitoring in axi5_slave_wr_mon for the full narrative and per-bit semantics.

Forwarded perfmon ports (identical width and direction to the base module):

- **Inputs:** `cfg_perf_enable`, `cfg_start_event_sel` (3), `cfg_end_event_sel` (3), `cfg_start_trigger`, `cfg_end_trigger`, `cfg_window_force_close`
- **Outputs:** `window_active`, `window_cycles` (32), `perf_prod_cycles` (32), `perf_bp_cycles` (32), `perf_starv_cycles` (32), `perf_idle_cycles` (32), `perf_beat_count` (32), `perf_byte_count` (64), `perf_burst_count` (32)

The `perf_burst_count` output tracks AW (write address) handshakes.

> **Warning:** an open measurement window is NOT a wake term — the window FSM and all counters run on the gated clock, so if the bus idles past `cfg_cg_idle_count` mid-window the counters FREEZE while wall-clock time passes, and trigger pulses arriving while gated are DROPPED. For exact wall-clock windows hold `cfg_cg_enable` low around the measurement, or use the base module.

Alongside perfmon, the wrapper forwards the `cfg_compl_enable`, `cfg_threshold_enable`, and `cfg_debug_enable` control inputs, plus the six `ENABLE_*_LOGIC` synthesis-cone parameters (see Parameters), straight through to the base monitor.

19.5 Timing Characteristics

Skid parameter	Default depth
SKID_DEPTH_AW	2 entries
SKID_DEPTH_W	4 entries
SKID_DEPTH_B	2 entries

Each channel traverses one `gaxi_skid_buffer`, which registers both `rd_valid` and its storage. The **1-cycle input-to-output latency therefore applies on every transfer, including the unstalled case** – there is no combinational bypass. Depth buys backpressure absorption, not throughput; full rate is sustained once the pipeline is primed. Legal range is 2..8 inclusive, odd values included.

Clocking: `aclk`, reset `aresetn` (active-low asynchronous).

No synthesis numbers are quoted here. Frequency and area depend on the target device and the parameters you elaborate with; run your own build.

19.6 Usage Examples

Every parameter and port below is read from the module declaration.

```
axi5_slave_wr_mon_cg #(
    .SKID_DEPTH_AW          (2),
    .SKID_DEPTH_W           (4),
    .SKID_DEPTH_B           (2),
    .AXI_ID_WIDTH           (8),
    .AXI_ADDR_WIDTH         (32),
    .AXI_DATA_WIDTH         (32),
    .AXI_USER_WIDTH         (1),
    .AXI_ATOP_WIDTH         (6),
    .AXI_NSAID_WIDTH        (4),
    .AXI_MPAM_WIDTH         (11)
) u_axi5_slave_wr_mon_cg (
    .aclk                   (aclk),
    .aresetn                (aresetn),
    .cam_clear              (cam_clear),
    .cfg_cg_enable          (cfg_cg_enable),
    .cfg_cg_idle_count      (cfg_cg_idle_count),
    .s_axi_awid             (s_axi_awid),
    .s_axi_awaddr           (s_axi_awaddr),
    .s_axi_awlen            (s_axi_awlen),
    .s_axi_awsize           (s_axi_awsize),
    .s_axi_awburst          (s_axi_awburst),
    .s_axi_awlock           (s_axi_awlock),
    .s_axi_awcache          (s_axi_awcache),
    .s_axi_awprot           (s_axi_awprot),
    .s_axi_awqos            (s_axi_awqos),
    .s_axi_awuser           (s_axi_awuser),
    .s_axi_awvalid          (s_axi_awvalid),
    .s_axi_awakeup          (s_axi_awakeup),
    .s_axi_awready          (s_axi_awready),
    .s_axi_awatop           (s_axi_awatop),
    .s_axi_awnsaid          (s_axi_awnsaid),
    .s_axi_awtrace          (s_axi_awtrace),
    .s_axi_awmpam           (s_axi_awmpam),
    .s_axi_awmecid          (s_axi_awmecid),
    .s_axi_awidunq         (s_axi_awidunq),
    .s_axi_awtagop          (s_axi_awtagop),
    .s_axi_wdata            (s_axi_wdata),
    .s_axi_wstrb            (s_axi_wstrb),
    .s_axi_wlast            (s_axi_wlast),
    .s_axi_wuser            (s_axi_wuser),
    .s_axi_wvalid           (s_axi_wvalid),
    .s_axi_wready           (s_axi_wready),
    .s_axi_wpoison          (s_axi_wpoison),
```

```
    .s_axi_wtag           (s_axi_wtag),
    .s_axi_wtagupdate     (s_axi_wtagupdate),
    .s_axi_bid            (s_axi_bid),
    .s_axi_bresp          (s_axi_bresp),
    .s_axi_buser          (s_axi_buser),
    .s_axi_bvalid         (s_axi_bvalid),
    .s_axi_bready         (s_axi_bready),
    .s_axi_btrace         (s_axi_btrace),
    .s_axi_bidunq         (s_axi_bidunq),
    .s_axi_btagmatch      (s_axi_btagmatch),
    .fub_axi_awid         (fub_axi_awid),
    .fub_axi_awaddr       (fub_axi_awaddr),
    .fub_axi_awlen        (fub_axi_awlen),
    .fub_axi_awsize       (fub_axi_awsize),
    .fub_axi_awburst      (fub_axi_awburst),
    .fub_axi_awlock       (fub_axi_awlock),
    .fub_axi_awcache      (fub_axi_awcache),
    .fub_axi_awprot       (fub_axi_awprot),
    .fub_axi_awqos        (fub_axi_awqos),
    .fub_axi_awuser       (fub_axi_awuser),
    .fub_axi_awvalid      (fub_axi_awvalid),
    .fub_axi_awakeup      (fub_axi_awakeup),
    .fub_axi_awready      (fub_axi_awready),
    .fub_axi_awatop       (fub_axi_awatop),
    .fub_axi_awnsaid      (fub_axi_awnsaid),
    .fub_axi_awtrace      (fub_axi_awtrace),
    .fub_axi_awmpam       (fub_axi_awmpam),
    .fub_axi_awmecid      (fub_axi_awmecid),
    .fub_axi_awidunq     (fub_axi_awidunq),
    .fub_axi_awtagop      (fub_axi_awtagop),
    .fub_axi_wdata        (fub_axi_wdata),
    .fub_axi_wstrb        (fub_axi_wstrb),
    .fub_axi_wlast        (fub_axi_wlast),
    .fub_axi_wuser        (fub_axi_wuser),
    .fub_axi_wvalid       (fub_axi_wvalid),
    .fub_axi_wready       (fub_axi_wready),
    .fub_axi_wpoison      (fub_axi_wpoison),
    .fub_axi_wtag         (fub_axi_wtag),
    .fub_axi_wtagupdate   (fub_axi_wtagupdate),
    .fub_axi_bid          (fub_axi_bid),
    .fub_axi_bresp        (fub_axi_bresp),
    .fub_axi_buser        (fub_axi_buser),
    .fub_axi_bvalid       (fub_axi_bvalid),
    .fub_axi_bready       (fub_axi_bready),
    .fub_axi_btrace       (fub_axi_btrace),
    .fub_axi_bidunq       (fub_axi_bidunq),
    .fub_axi_btagmatch    (fub_axi_btagmatch),
    .cfg_monitor_enable   (cfg_monitor_enable),
```

```
    .cfg_error_enable      (cfg_error_enable),
    .cfg_timeout_enable    (cfg_timeout_enable),
    .cfg_perf_enable       (cfg_perf_enable),
    .cfg_compl_enable      (cfg_compl_enable),
    .cfg_threshold_enable  (cfg_threshold_enable),
    .cfg_debug_enable      (cfg_debug_enable),
    .cfg_timeout_cycles    (cfg_timeout_cycles),
    .cfg_freq_sel          (cfg_freq_sel),
    .cfg_latency_threshold (cfg_latency_threshold),
    .cfg_axi_pkt_mask      (cfg_axi_pkt_mask),
    .cfg_axi_err_select    (cfg_axi_err_select),
    .cfg_axi_error_mask    (cfg_axi_error_mask),
    .cfg_axi_timeout_mask  (cfg_axi_timeout_mask),
    .cfg_axi_compl_mask    (cfg_axi_compl_mask),
    .cfg_axi_thresh_mask   (cfg_axi_thresh_mask),
    .cfg_axi_perf_mask     (cfg_axi_perf_mask),
    .cfg_axi_addr_mask     (cfg_axi_addr_mask),
    .cfg_axi_debug_mask    (cfg_axi_debug_mask),
    .cfg_addr_check_enable (cfg_addr_check_enable),
    .cfg_addr_range_enable (cfg_addr_range_enable),
    .cfg_addr_range_low    (cfg_addr_range_low),
    .cfg_addr_range_high   (cfg_addr_range_high),
    .cfg_addr_filter_enable(cfg_addr_filter_enable),
    .cfg_addr_filter_low   (cfg_addr_filter_low),
    .cfg_addr_filter_high  (cfg_addr_filter_high),
    .cfg_id_filter_enable  (cfg_id_filter_enable),
    .cfg_id_match_base     (cfg_id_match_base),
    .cfg_id_match_count    (cfg_id_match_count),
    .i_mon_time            (i_mon_time),
    .monbus_valid          (monbus_valid),
    .monbus_ready          (monbus_ready),
    .monbus_packet         (monbus_packet),
    .monbus_timestamp      (monbus_timestamp),
    .busy                  (busy),
    .active_transactions   (active_transactions),
    .error_count           (error_count),
    .transaction_count     (transaction_count),
    .cfg_conflict_error    (cfg_conflict_error),
    .cg_gating             (cg_gating),
    .cg_idle               (cg_idle),
    .cfg_start_event_sel   (cfg_start_event_sel),
    .cfg_end_event_sel     (cfg_end_event_sel),
    .cfg_start_trigger     (cfg_start_trigger),
    .cfg_end_trigger       (cfg_end_trigger),
    .cfg_window_force_close(cfg_window_force_close),
    .window_active         (window_active),
    .window_cycles         (window_cycles),
    .perf_prod_cycles      (perf_prod_cycles),
```

```
    .perf_bp_cycles        (perf_bp_cycles),
    .perf_starv_cycles     (perf_starv_cycles),
    .perf_idle_cycles      (perf_idle_cycles),
    .perf_beat_count       (perf_beat_count),
    .perf_byte_count       (perf_byte_count),
    .perf_burst_count      (perf_burst_count)
);
```

19.7 Design Notes

19.7.1 When to Use This Module

Ideal for: - Power-constrained systems needing visibility into write operations - Debug/validation builds with power budgets - Systems with sporadic write transaction patterns - SoC integration requiring both monitoring and power optimization - (NOT for poison detection — WPOISON never reaches the monitor)

Avoid when: - Interface is continuously active (minimal gating opportunities) - Monitoring overhead unacceptable (use non-monitored version) - Deterministic latency critical (use high idle count or disable gating) - Write bursts dominate traffic (reduces gating efficiency)

19.7.2 Configuration Recommendations

19.7.2.1 Low-Power Debug Mode

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd1),   // Aggressive gating
.cfg_monitor_enable (1'b1),   // Full monitoring
.cfg_error_enable   (1'b1),
.cfg_perf_enable    (1'b0)    // Reduce packet traffic
```

19.7.2.2 Performance Analysis Mode

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd4),   // Conservative gating
.cfg_monitor_enable (1'b1),   // Master gate MUST stay 1 (0 disables
ALL monitoring); disable completions via cfg_compl_enable
.cfg_perf_enable    (1'b1)    // Enable performance metrics
```

19.7.2.3 Data Integrity Validation

```
.cfg_cg_enable      (1'b1),
.cfg_cg_idle_count  (4'd2),   // Balanced
.cfg_error_enable   (1'b1),   // Catch all errors
.cfg_monitor_enable (1'b1),   // Master gate: monitor active
.ENABLE_POISON      (1),      // transport-only: carries WPOISON, no
detection
.ENABLE_MTE         (1)       // transport-only: carries tags, no
checking
```

19.7.3 Area and Timing Impact

- **Area:** not 5-8%. That figure was an unsourced estimate and is off by orders of magnitude. The monitor's transaction table alone is a counted floor of thousands of flops (see the resource table in the family README) against a few hundred for the unmonitored slave, so a monitored instance is a

multiple of the base module, not a few percent on top of it. Clock gating adds 5 flops. `MAX_TRANSACTIONS` is the knob and the cost is linear in it
- **Timing:** first-order estimate, not measured. The monitor is NOT automatically off the critical path – `axi_monitor_trans_mgr` is banked precisely because it was not (16 entries closed at WNS +1.018 ns where 40 entries came in at -25.183 ns), so size the transaction table for your clock
- **Power:** Net savings depends on activity factor (typically 30-50% at 50% duty cycle)

19.7.4 Power Optimization

Power Savings Estimation: - Base slave logic: ~40% of total power *(first-order estimates — no power/area analysis has been run)* - Monitor logic: ~60% of total power - With gating at 50% duty cycle: ~50% dynamic power savings - Actual savings depend on traffic pattern and idle_count setting

Write-Specific Power Considerations: - Burst writes keep module active longer - W channel SKID buffering consumes additional power - (No poison/tag checking exists — there is no such overhead.) - Atomic operations may extend active periods

19.7.5 Observability

Monitoring Capabilities: - Error detection (SLVERR, timeout, orphan — poison is NOT observable; see below) - Performance tracking (latency, throughput) - Transaction completion tracking - Protocol violation detection - (Atomic operations are NOT monitored — AWATOP never reaches the monitor) - (Tag-mismatch detection is NOT implemented — ENABLE_MTE shapes the transport only) - All monitoring continues when ungated

19.7.6 Write-Specific Considerations

Burst Handling: - Monitor tracks AW/W synchronization - (Missing-WLAST detection is NOT implemented) - Burst latency includes all beats - Gating should not occur mid-burst

Atomic Operations: - AWATOP is NOT tracked by the monitor (transport-only signal) - Atomic transaction latency measured end-to-end - Consider higher idle_count for atomic-heavy workloads

Poison Detection: NOT implemented. WPOISON flows through the transport data plane and never reaches the monitor — no poison event packet can be emitted (the 8'h5 decode some examples showed can never match).

19.8 Related Modules

- **AXI5 Slave Write** - Non-monitored, non-gated version
- **AXI5 Slave Write CG** - Clock-gated without monitoring
- **AXI5 Slave Write Monitor** - Monitored without clock gating
- **AXI5 Slave Read Monitor CG** - Read variant
- **AMBA Clock Gate Control** - Clock gating controller

19.9 Testing

`val/amba/test_axi5_slave_wr_mon_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axi5_slave_wr_mon_cg.py -v
```

19.10 References

- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A3.3.1 “Write response” and §A3.3.2 “Read response” (the SLVERR/DECERR codes the monitor reports), §A5.1 “Transaction identifiers” and §A5.3 “Request ordering” (per-ID tracking of outstanding transactions), §A2.3.2 “Dependencies between channel handshake signals” (the orderings a monitor timeout watches); the AXI5 payload is carried as on the base module.
- Benini, L., De Micheli, G. “Automatic Synthesis of Low-Power Gated-Clock Finite-State Machines.” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 15(6), June 1996. DOI: 10.1109/43.503933 – the latch-based clock-gating cell (`icg`, via `amba_clock_gate_ctrl` and `clock_gate_ctrl`) that gates this wrapper’s clock.
- Benini, L., Bogliolo, A., De Micheli, G. “A Survey of Design Techniques for System-Level Dynamic Power Management.” *IEEE Trans. VLSI Systems*, 8(3), June 2000. DOI: 10.1109/92.845896 – fixed-timeout shutdown: gate after `cfg_cg_idle_count` idle cycles, wake on activity.
- Arm. *AMBA AXI Protocol Specification*, ARM IHI 0022, Issue L, Aug. 2025. developer.arm.com/documentation/ihi0022/latest – §A14.1.1 “AWAKEUP rules and recommendations” in Issue L (§E1.7 “Wake-up Signaling” in Issue H) defines AWAKEUP, which lets a manager restart a gated interface before VALID. This wrapper counts AWAKEUP as an activity term (behind `ENABLE_WAKEUP`), so a manager that raises it a cycle ahead of VALID, as §A14.1.1 recommends, finds the clock running and its first request is accepted on the cycle it is presented; a manager that never drives it still wakes the block through the flopped OR of the VALIDs, with READY held low while gated (GitHub #102).
- The monitor’s own techniques are cited on the AXI Monitor Base page and in the monitor family of the archive.
- Indexed in `rtl/amba/references/README.md` (local PDF, archived under `rtl/common/references/`: `LatchBasedClockGating__Benini1996.pdf`, `DynamicPowerManagementTimeout__Benini2000.pdf`)

19.11 Navigation

- ← Back to AXI5 Index
- ← Back to rtl-amba Index
- ← Back to Main Documentation Index